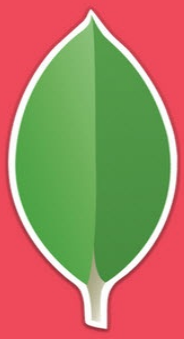


MongoDB

LEARN IN 1 DAY

KRISHNA RUNGTA

Learn MongoDB in 1 Day

By Krishna Rungta

Table Of Content

5. Install MongoDB Compass- MongoDB Management Tool
6. MongoDB Configuration, Import, and Export
7. Configuring MongoDB server with configuration file

Chapter 4: Install MongoDB in Cloud: AWS, Google, Azure

Chapter 5: How to Create Database & Collection in MongoDB

1. Creating a database using “use” command
2. Creating a Collection/Table using insert()
3. Adding documents using insert() command

Chapter 6: Add MongoDB Array using insert() with Example

Chapter 7: Mongodb Primary Key: Example to set _id field with ObjectId()

Chapter 8: MongoDB Query Document using find() with Example

Chapter 9: MongoDB Cursor Tutorial: Learn with EXAMPLE

Chapter 10: MongoDB order with Sort() & Limit() Query with Examples

1. What is Query Modifications?
2. MongoDB Limit Query Results
3. MongoDB Sort by Descending Order

Chapter 11: MongoDB Count() & Remove() Functions with Examples

Chapter 12: MongoDB Update() Document with Example

1. Basic document updates

dropindex() Example

1. Understanding Impact of Indexes
2. How to Create Indexes: createIndex()
3. How to Find Indexes: getindexes()
4. How to Drop Indexes: dropindex()

Chapter 19: MongoDB Regular Expression (Regex) with Examples

1. Using $regex operator for Pattern matching
2. Pattern Matching with $options
3. Pattern matching without the regex operator
4. Fetching last ‘n’ documents from a collection

Chapter 1: What is MongoDB? Introduction, Architecture, Features & Example

What is MongoDB?

MongoDB is a document-oriented NoSQL database used for high volume data storage. MongoDB is a database which came into light around the mid-2000s. It falls under the category of a NoSQL database.

MongoDB Features

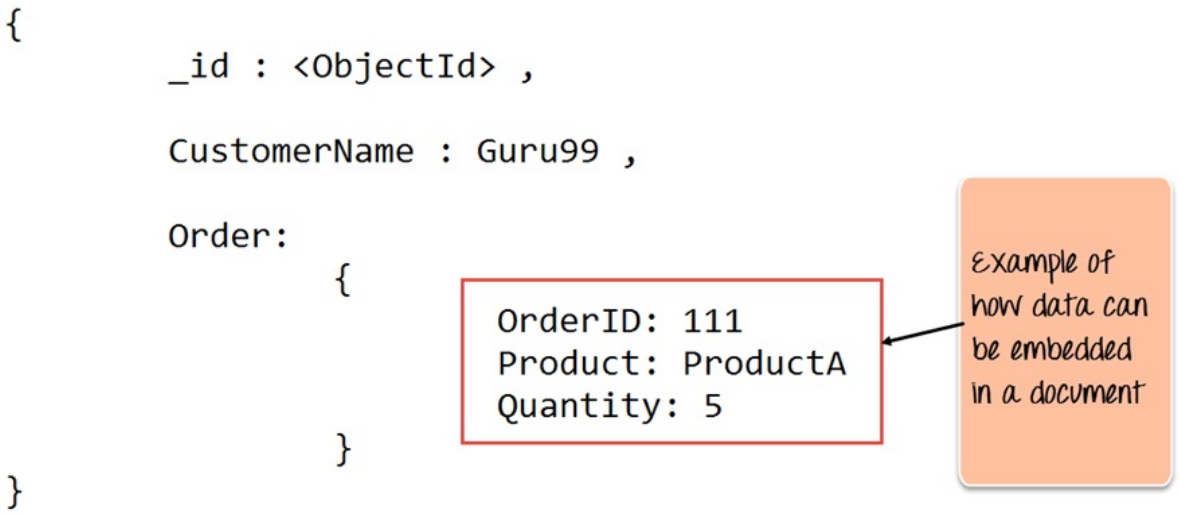
1. Each database contains collections which in turn contains documents. Each document can be different with a varying number of fields. The size and content of each document can be different from each other.
2. The document structure is more in line with how developers construct their classes and objects in their respective programming languages. Developers will often say that their classes are not rows and columns but have a clear structure with key-value pairs.
3. As seen in the introduction with NoSQL databases, the rows (or documents as called in MongoDB) doesn't need to have a schema defined beforehand. Instead, the fields can be created on the fly.
4. The data model available within MongoDB allows you to represent hierarchical relationships, to store arrays, and other more complex structures more easily.
5. Scalability – The MongoDB environments are very scalable. Companies across the world have defined clusters with some of them running 100+ nodes with around millions of documents

within the database

MongoDB Example

The below example shows how a document can be modeled in MongoDB.

1. The _id field is added by MongoDB to uniquely identify the document in the collection.
2. What you can note is that the Order Data (OrderID, Product, and Quantity) which in RDBMS will normally be stored in a separate table, while in MongoDB it is actually stored as an embedded document in the collection itself. This is one of the key differences in how data is modeled in MongoDB.



Key Components of MongoDB Architecture

Below are a few of the common terms used in MongoDB

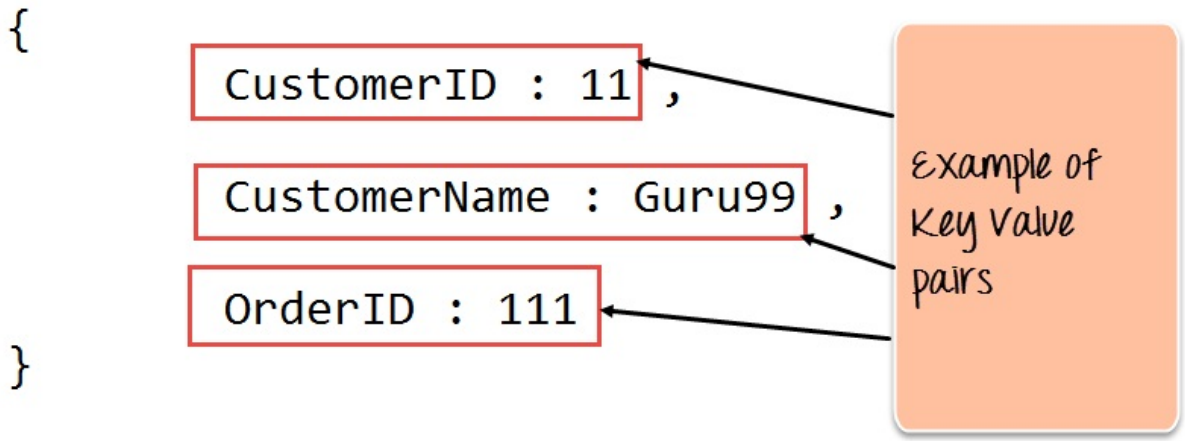
1. **_id** – This is a field required in every MongoDB document. The _id field represents a unique value in the MongoDB document. The _id field is like the document's primary key. If you create a

new document without an _id field, MongoDB will automatically create the field. So for example, if we see the example of the above customer table, Mongo DB will add a 24 digit unique identifier to each document in the collection.

_Id	CustomerID	CustomerName	OrderID
563479cc8a8a4246bd27d784	11	Guru99	111
563479cc7a8a4246bd47d784	22	Trevor Smith	222
563479cc9a8a4246bd57d784	33	Nicole	333

2. **Collection** – This is a grouping of MongoDB documents. A collection is the equivalent of a table which is created in any other RDMS such as Oracle or MS SQL. A collection exists within a single database. As seen from the introduction collections don't enforce any sort of structure.
3. **Cursor** – This is a pointer to the result set of a query. Clients can iterate through a cursor to retrieve results.
4. **Database** – This is a container for collections like in RDMS wherein it is a container for tables. Each database gets its own set of files on the file system. A MongoDB server can store multiple databases.
5. **Document** - A record in a MongoDB collection is basically called a document. The document, in turn, will consist of field name and values.
6. **Field** - A name-value pair in a document. A document has zero or more fields. Fields are analogous to columns in relational databases.

   The following diagram shows an example of Fields with Key value pairs. So in the example below CustomerID and 11 is one of the key value pair's defined in the document.



7. **JSON** – This is known as JavaScript Object Notation. This is a human-readable, plain text format for expressing structured data. JSON is currently supported in many programming languages.

Just a quick note on the key difference between the _id field and a normal collection field. The _id field is used to uniquely identify the documents in a collection and is automatically added by MongoDB when the collection is created.

Why Use MongoDB?

Below are the few of the reasons as to why one should start using MongoDB

1. Document-oriented – Since MongoDB is a NoSQL type database, instead of having data in a relational type format, it stores the data in documents. This makes MongoDB very flexible and adaptable to real business world situation and requirements.
2. Ad hoc queries - MongoDB supports searching by field, range queries, and regular expression searches. Queries can be made to return specific fields within documents.
3. Indexing - Indexes can be created to improve the performance of searches within MongoDB. Any field in a MongoDB document can be indexed.
4. Replication - MongoDB can provide high availability with replica

sets. A replica set consists of two or more mongo DB instances. Each replica set member may act in the role of the primary or secondary replica at any time. The primary replica is the main server which interacts with the client and performs all the read/write operations. The Secondary replicas maintain a copy of the data of the primary using built-in replication. When a primary replica fails, the replica set automatically switches over to the secondary and then it becomes the primary server.

5. Load balancing - MongoDB uses the concept of sharding to scale horizontally by splitting data across multiple MongoDB instances. MongoDB can run over multiple servers, balancing the load and/or duplicating data to keep the system up and running in case of hardware failure.

Data Modelling in MongoDB

As we have seen from the Introduction section, the data in MongoDB has a flexible schema. Unlike in SQL databases, where you must have a table's schema declared before inserting data, MongoDB's collections do not enforce document structure. This sort of flexibility is what makes MongoDB so powerful.

When modeling data in Mongo, keep the following things in mind

1. What are the needs of the application – Look at the business needs of the application and see what data and the type of data needed for the application. Based on this, ensure that the structure of the document is decided accordingly.
2. What are data retrieval patterns – If you foresee a heavy query usage then consider the use of indexes in your data model to improve the efficiency of queries.
3. Are frequent insert's, updates and removals happening in the database – Reconsider the use of indexes or incorporate sharding if required in your data modeling design to improve the efficiency of your overall MongoDB environment.

Difference between MongoDB & RDBMS

Below are some of the key term differences between MongoDB and RDBMS

RDBMS	MongoDB	Difference
Table	Collection	In RDBMS, the table contains the columns and rows which are used to store the data whereas, in MongoDB, this same structure is known as a collection. The collection contains documents which in turn contains Fields, which in turn are key-value pairs.
Row	Document	In RDBMS, the row represents a single, implicitly structured data item in a table. In MongoDB, the data is stored in documents.
Column	Field	In RDBMS, the column denotes a set of data values. These in MongoDB are known as Fields.
Joins	Embedded documents	In RDBMS, data is sometimes spread across various tables and in order to show a complete view of all data, a join is sometimes formed across tables to get the data. In MongoDB, the data is normally stored in a single collection, but separated by using Embedded documents. So there is no concept of joins in MongoDB.

Apart from the terms differences, a few other differences are shown below

1. Relational databases are known for enforcing data integrity. This is not an explicit requirement in MongoDB.
2. RDBMS requires that data be normalized first so that it can prevent orphan records and duplicates Normalizing data then has the requirement of more tables, which will then result in more table joins, thus requiring more keys and indexes.

   As databases start to grow, performance can start becoming an issue. Again this is not an explicit requirement in MongoDB. MongoDB is flexible and does not need the data to be normalized first.

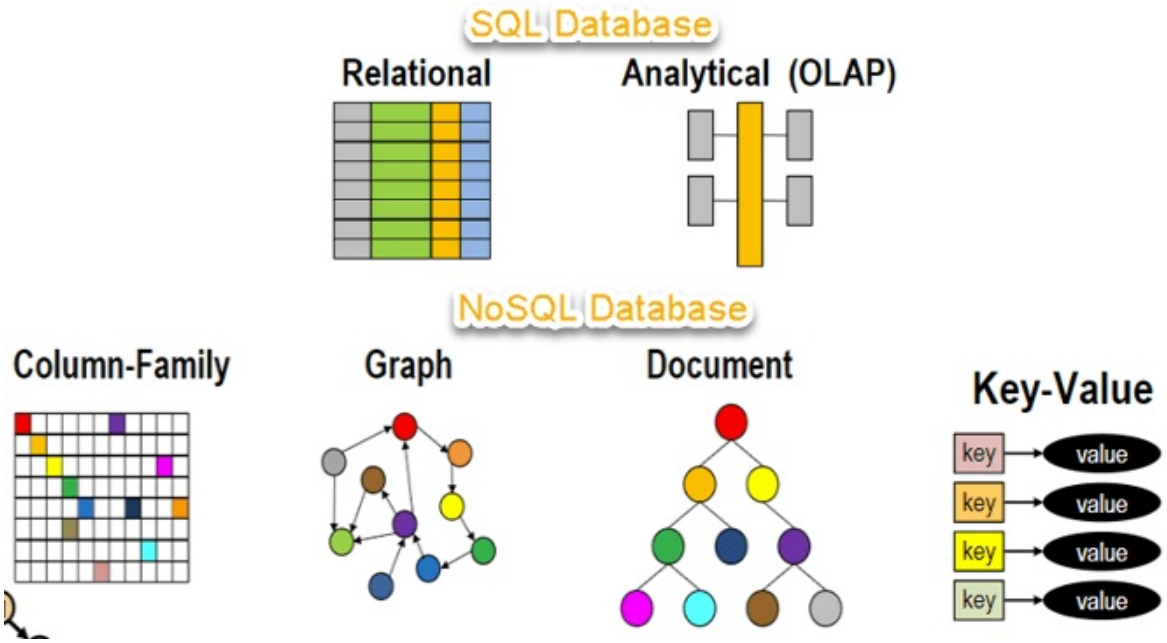
Chapter 2: NoSQL Tutorial: Learn NoSQL Features, Types, What is, Advantages

What is NoSQL?

NoSQL is a non-relational DMS, that does not require a fixed schema, avoids joins, and is easy to scale. NoSQL database is used for distributed data stores with humongous data storage needs. NoSQL is used for Big data and real-time web apps. For example, companies like Twitter, Facebook, Google that collect terabytes of user data every single day.

NoSQL database stands for “Not Only SQL” or “Not SQL.” Though a better term would NoREL NoSQL caught on. Carl Strozz introduced the NoSQL concept in 1998.

Traditional RDBMS uses SQL syntax to store and retrieve data for further insights. Instead, a NoSQL database system encompasses a wide range of database technologies that can store structured, semi-structured, unstructured and polymorphic data.

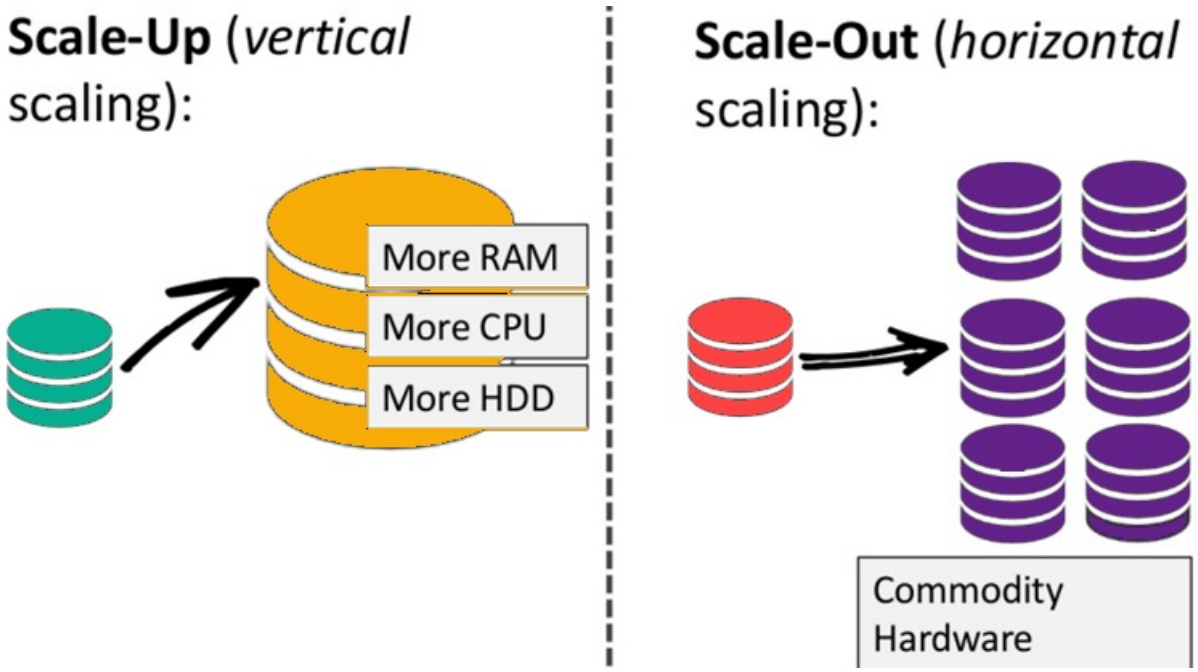


Why NoSQL?

The concept of NoSQL databases became popular with Internet giants like Google, Facebook, Amazon, etc. who deal with huge volumes of data. The system response time becomes slow when you use RDBMS for massive volumes of data.

To resolve this problem, we could “scale up” our systems by upgrading our existing hardware. This process is expensive.

The alternative for this issue is to distribute database load on multiple hosts whenever the load increases. This method is known as “scaling out.”



NoSQL database is non-relational, so it scales out better than relational databases as they are designed with web applications in mind.

Brief History of NoSQL Databases

- 1998- Carlo Strozzi use the term NoSQL for his lightweight, open-source relational database
- 2000- Graph database Neo4j is launched
- 2004- Google BigTable is launched
- 2005- CouchDB is launched
- 2007- The research paper on Amazon Dynamo is released
- 2008- Facebooks open sources the Cassandra project
- 2009- The term NoSQL was reintroduced

Features of NoSQL

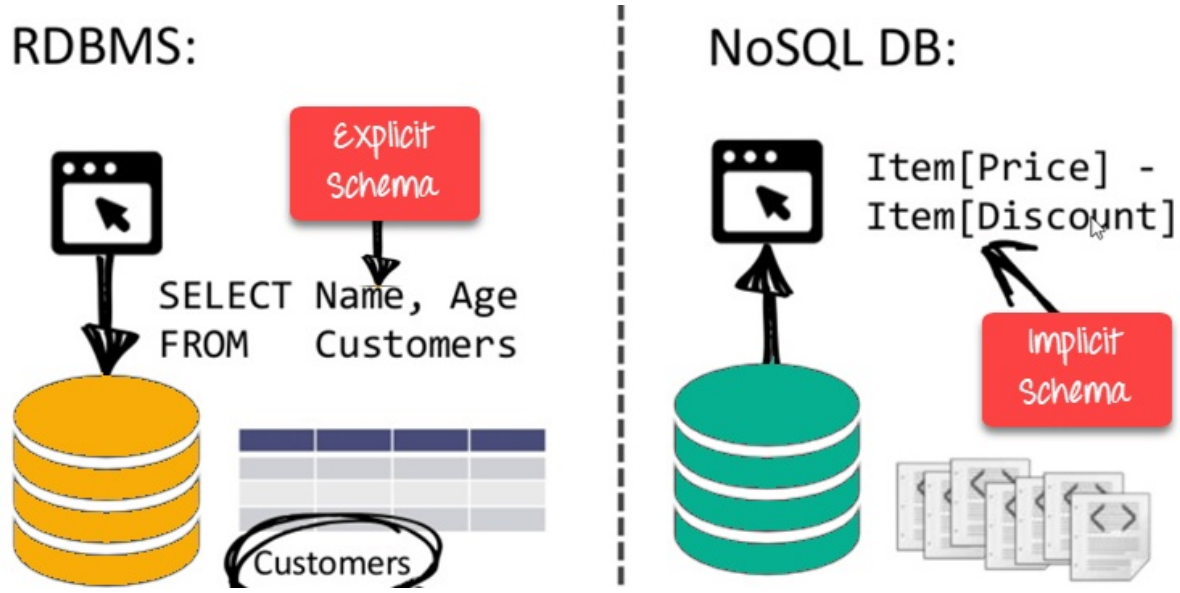
Non-relational

- NoSQL databases never follow the relational model
- Never provide tables with flat fixed-column records

- Work with self-contained aggregates or BLOBs
- Doesn’t require object-relational mapping and data normalization
- No complex features like query languages, query planners, referential integrity joins, ACID

Schema-free

- NoSQL databases are either schema-free or have relaxed schemas
- Do not require any sort of definition of the schema of the data
- Offers heterogeneous structures of data in the same domain



NoSQL is Schema-Free

Simple API

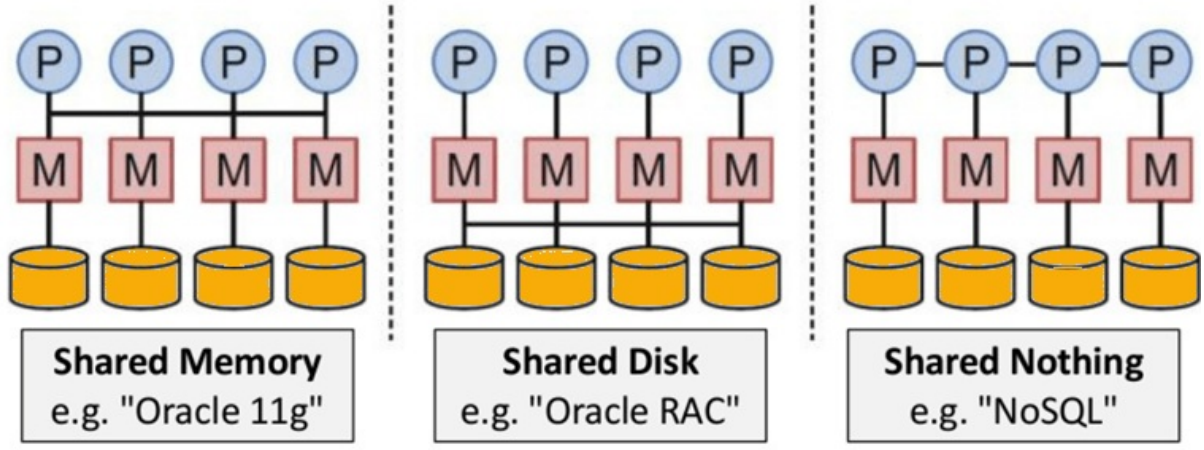
- Offers easy to use interfaces for storage and querying data provided
- APIs allow low-level data manipulation & selection methods
- Text-based protocols mostly used with HTTP REST with JSON
- Mostly used no standard based query language
- Web-enabled databases running as internet-facing services

Distributed

- Multiple NoSQL databases can be executed in a distributed

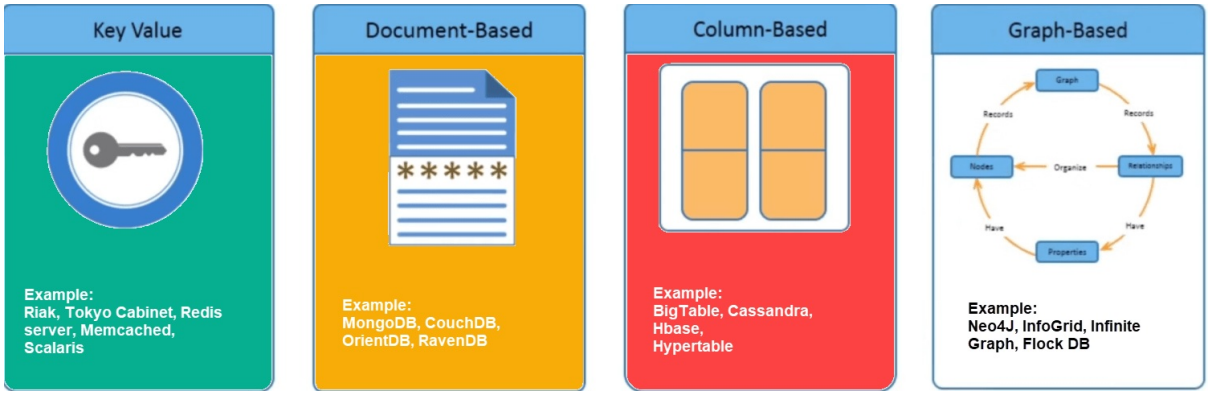
fashion

- Offers auto-scaling and fail-over capabilities
- Often ACID concept can be sacrificed for scalability and throughput
- Mostly no synchronous replication between distributed nodes Asynchronous Multi-Master Replication, peer-to-peer, HDFS Replication
- Only providing eventual consistency
- Shared Nothing Architecture. This enables less coordination and higher distribution.



NoSQL is Shared Nothing.

Types of NoSQL Databases



There are mainly four categories of NoSQL databases. Each of these categories has its unique attributes and limitations. No specific database is better to solve all problems. You should select a database

based on your product needs.

Let see all of them:

- Key-value Pair Based
- Column-oriented Graph
- Graphs based
- Document-oriented

Key Value Pair Based

Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.

Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB(Binary Large Objects), string, etc.

For example, a key-value pair may contain a key like “Website” associated with a value like “Guru99”.

Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

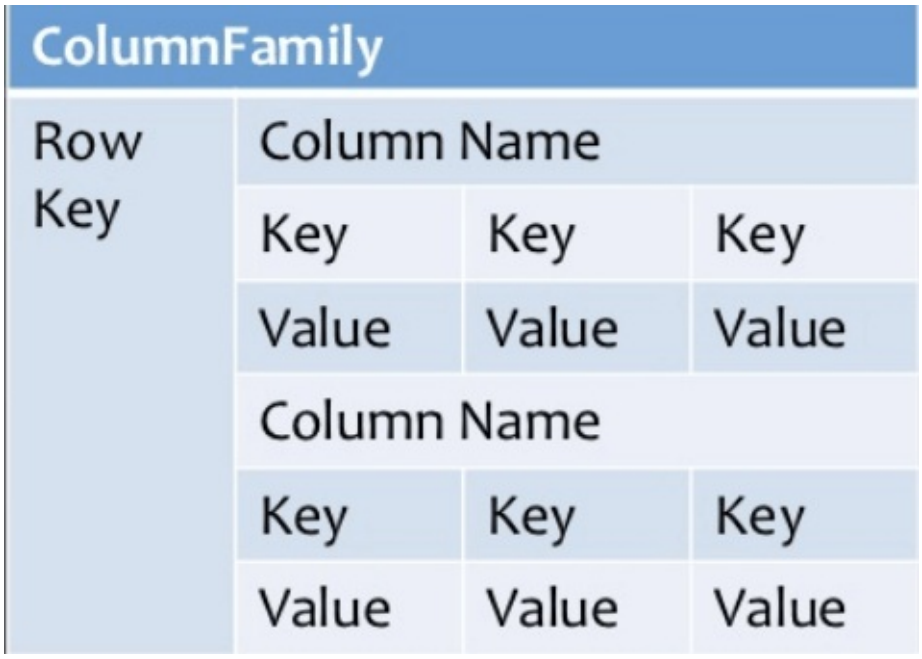
It is one of the most basic types of NoSQL databases. This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc. Key value stores help the developer to store schema-less data. They work best for shopping cart contents.

Redis, Dynamo, Riak are some examples of key-value store DataBases.

They are all based on Amazon's Dynamo paper.

Column-based

Column-oriented databases work on columns and are based on BigTable paper by Google. Every column is treated separately. Values of single column databases are stored contiguously.



Column based NoSQL database

They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.

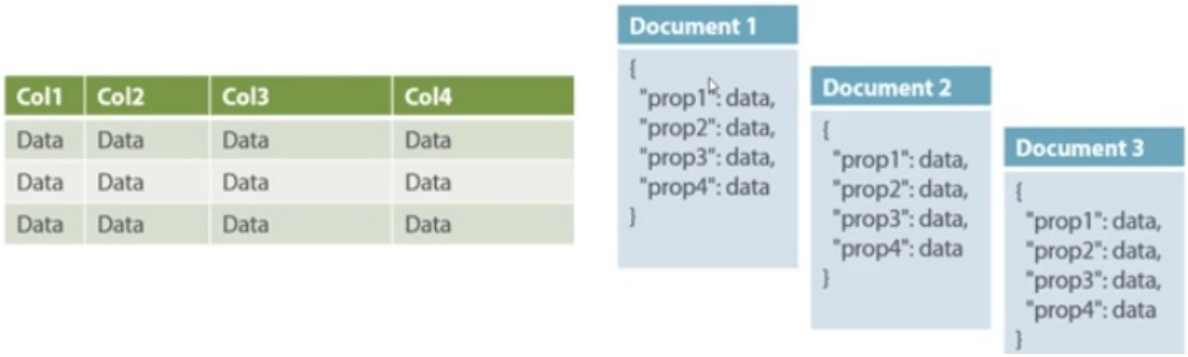
Column-based NoSQL databases are widely used to manage data warehouses, business intelligence, CRM, Library card catalogs,

HBase, Cassandra, HBase, Hypertable are examples of column based database.

Document-Oriented:

Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document. The document is stored in JSON or XML formats. The value is understood by the DB and can

be queried.



Relational Vs. Document

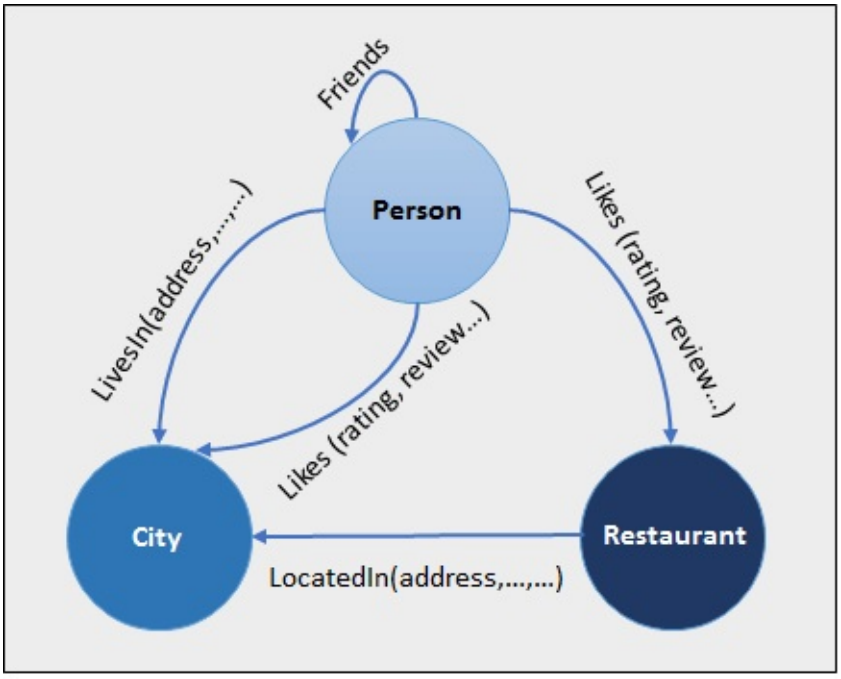
In this diagram on your left you can see we have rows and columns, and in the right, we have a document database which has a similar structure to JSON. Now for the relational database, you have to know what columns you have and so on. However, for a document database, you have data store like JSON object. You do not require to define which make it flexible.

The document type is mostly used for CMS systems, blogging platforms, real-time analytics & e-commerce applications. It should not use for complex transactions which require multiple operations or queries against varying aggregate structures.

Amazon SimpleDB, CouchDB, MongoDB, Riak, Lotus Notes, MongoDB, are popular Document originated DBMS systems.

Graph-Based

A graph type database stores entities as well the relations amongst those entities. The entity is stored as a node with the relationship as edges. An edge gives a relationship between nodes. Every node and edge has a unique identifier.



Compared to a relational database where tables are loosely connected, a Graph database is a multi-relational in nature. Traversing relationship is fast as they are already captured into the DB, and there is no need to calculate them.

Graph base database mostly used for social networks, logistics, spatial data.

Neo4J, Infinite Graph, OrientDB, FlockDB are some popular graph-based databases.

Query Mechanism tools for NoSQL

The most common data retrieval mechanism is the REST-based retrieval of a value based on its key/ID with GET resource

Document store Database offers more difficult queries as they understand the value in a key-value pair. For example, CouchDB allows defining views with MapReduce

What is the CAP Theorem?

CAP theorem is also called brewer's theorem. It states that is impossible for a distributed data store to offer more than two out of three guarantees

1. Consistency
2. Availability
3. Partition Tolerance

Consistency:

The data should remain consistent even after the execution of an operation. This means once data is written, any future read request should contain that data. For example, after updating the order status, all the clients should be able to see the same data.

Availability:

The database should always be available and responsive. It should not have any downtime.

Partition Tolerance:

Partition Tolerance means that the system should continue to function even if the communication among the servers is not stable. For example, the servers can be partitioned into multiple groups which may not communicate with each other. Here, if part of the database is unavailable, other parts are always unaffected.

Eventual Consistency

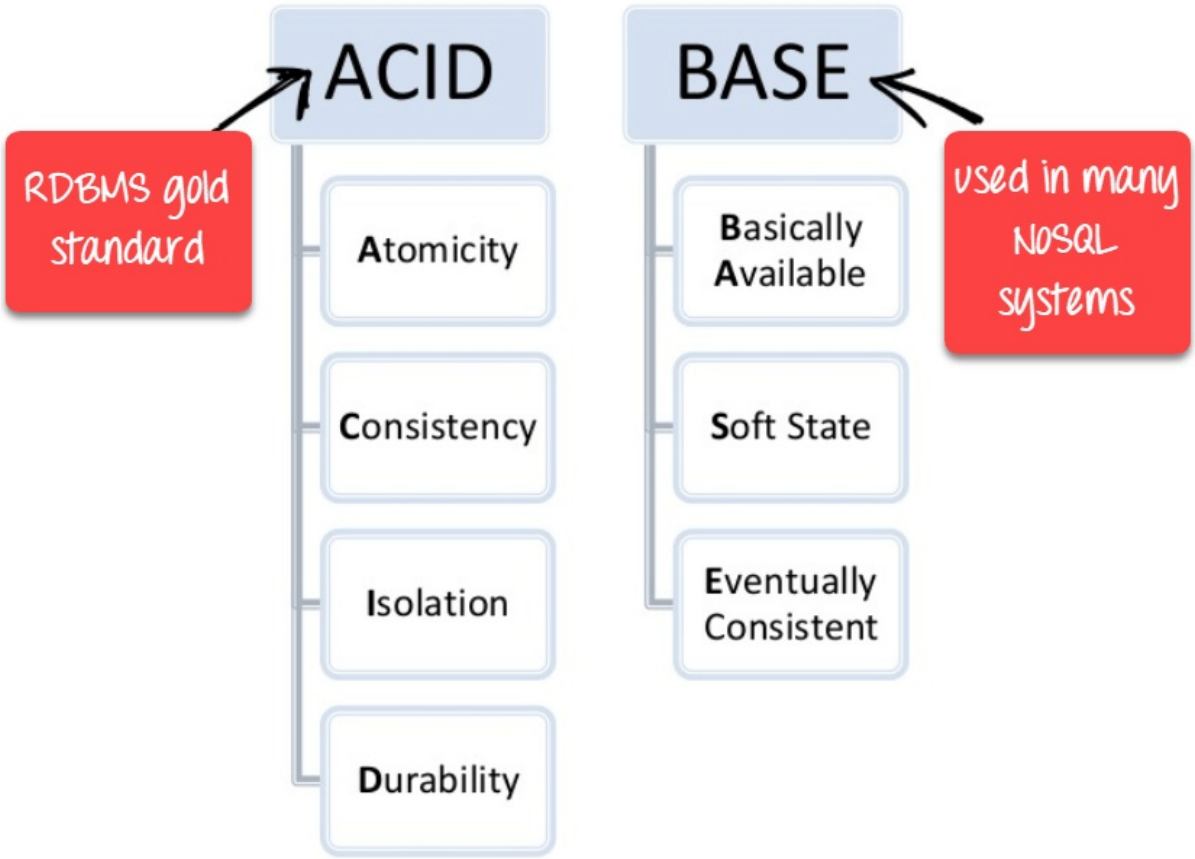
The term "eventual consistency" means to have copies of data on multiple machines to get high availability and scalability. Thus, changes made to any data item on one machine has to be propagated to other replicas.

Data replication may not be instantaneous as some copies will be updated immediately while others in due course of time. These copies may be mutually, but in due course of time, they become consistent.

Hence, the name eventual consistency.

BASE: **B**asically **A**vailable, **S**oft state, **E**ventual consistency

- Basically, available means DB is available all the time as per CAP theorem
- Soft state means even without an input; the system state may change
- Eventual consistency means that the system will become consistent over time



Advantages of NoSQL

- Can be used as Primary or Analytic Data Source
- Big Data Capability
- No Single Point of Failure
- Easy Replication
- No Need for Separate Caching Layer

- It provides fast performance and horizontal scalability.
- Can handle structured, semi-structured, and unstructured data with equal effect
- Object-oriented programming which is easy to use and flexible
- NoSQL databases don’t need a dedicated high-performance server
- Support Key Developer Languages and Platforms
- Simple to implement than using RDBMS
- It can serve as the primary data source for online applications.
- Handles big data which manages data velocity, variety, volume, and complexity
- Excels at distributed database and multi-data center operations
- Eliminates the need for a specific caching layer to store data
- Offers a flexible schema design which can easily be altered without downtime or service disruption

Disadvantages of NoSQL

- No standardization rules
- Limited query capabilities
- RDBMS databases and tools are comparatively mature
- It does not offer any traditional database capabilities, like consistency when multiple transactions are performed simultaneously.
- When the volume of data increases it is difficult to maintain unique values as keys become difficult
- Doesn’t work as well with relational data
- The learning curve is stiff for new developers
- Open source options so not so popular for enterprises.

Summary

- NoSQL is a non-relational DMS, that does not require a fixed schema, avoids joins, and is easy to scale

- The concept of NoSQL databases beccame popular with Internet giants like Google, Facebook, Amazon, etc. who deal with huge volumes of data
- In the year 1998- Carlo Strozzi use the term NoSQL for his lightweight, open-source relational database
- NoSQL databases never follow the relational model it is either schema-free or has relaxed schemas
- Four types of NoSQL Database are 1).Key-value Pair Based 2).Column-oriented Graph 3). Graphs based 4).Document-oriented
- NOSQL can handle structured, semi-structured, and unstructured data with equal effect
- CAP theorem consists of three words Consistency, Availability, and Partition Tolerance
- BASE stands for **B**asically **A**vailable, **S**oft state, **E**ventual consistency
- The term “eventual consistency” means to have copies of data on multiple machines to get high availability and scalability
- NOSQL offer limited query capabilities

Chapter 3: How to Download & Install MongoDB on Windows

The installers for MongoDB are available in both the 32-bit and 64-bit format. The 32-bit installers are good for development and test environments. But for production environments you should use the 64-bit installers. Otherwise, you can be limited to the amount of data that can be stored within MongoDB.

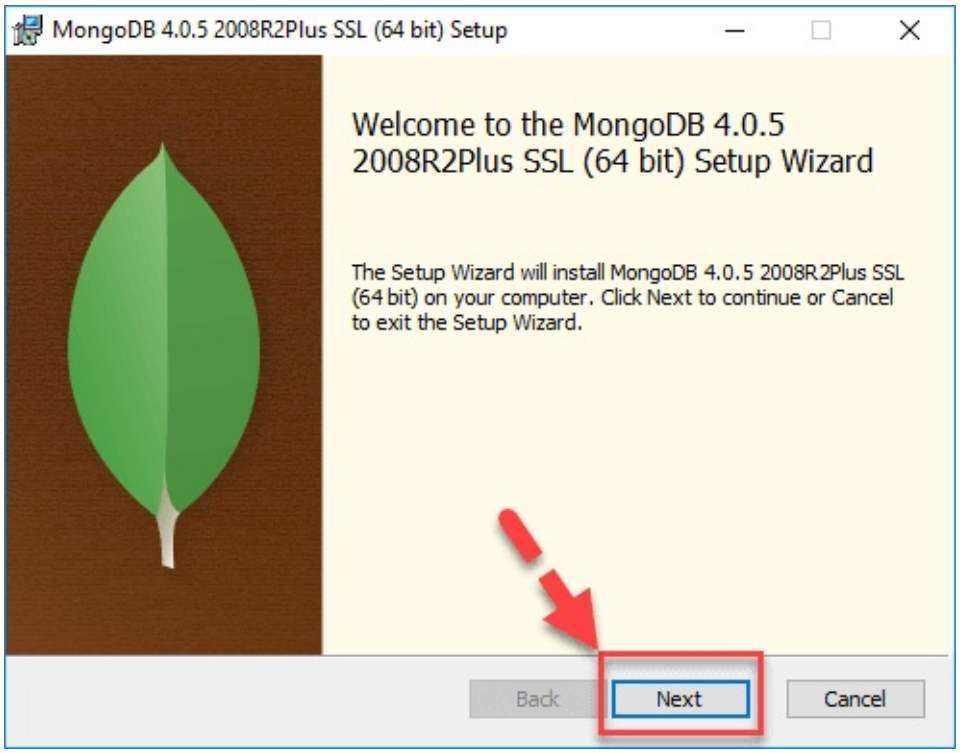
It is advisable to always use the stable release for production environments.

Download & Install MongoDB on Windows

The following steps can be used to install MongoDB on Windows 10

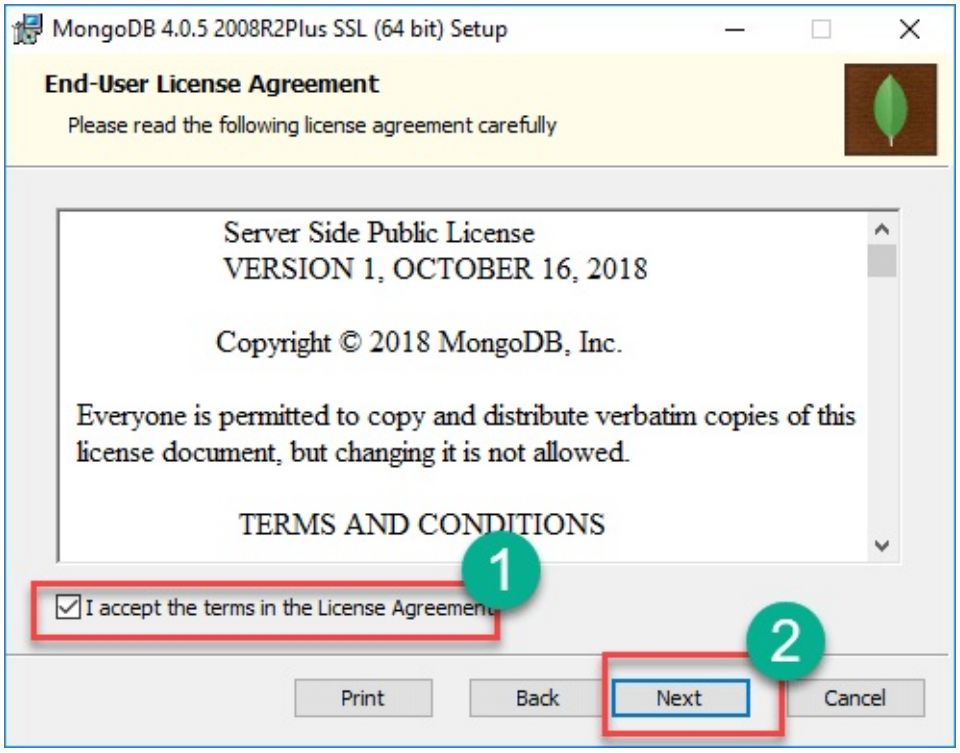
Step 1) Go to link and Download MongoDB Community Server. We will install the 64-bit version for Windows.

Step 2) Once download is complete open the msi file. Click Next in the start up screen

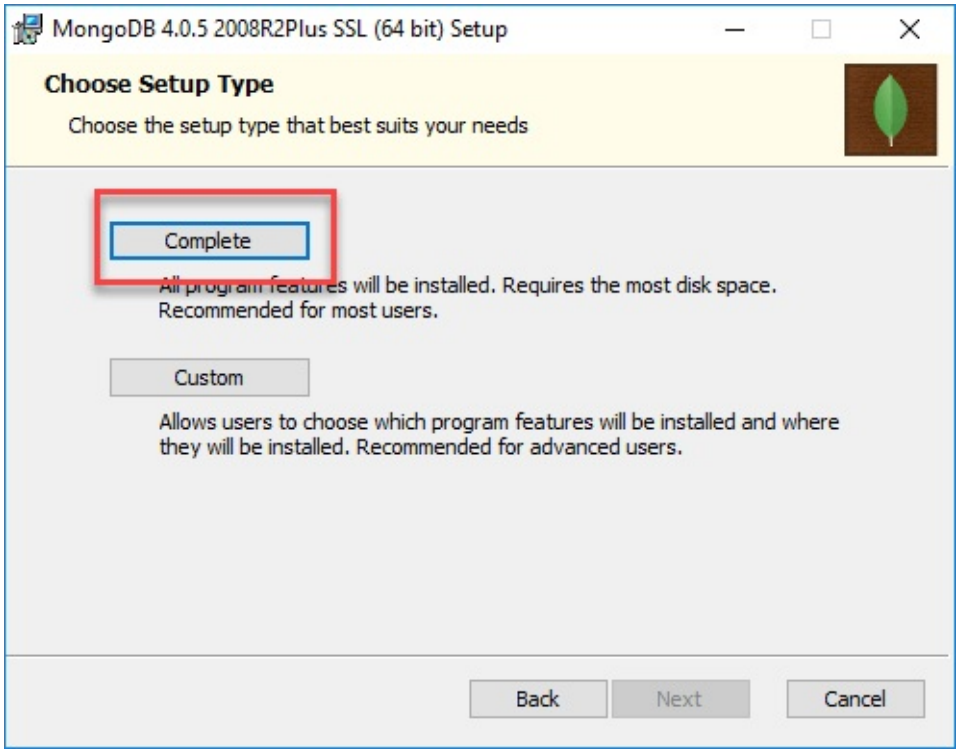


Step 3)

1. Accept the End-User License Agreement
2. Click Nest



Step 4) Click on the "complete" button to install all of the components. The custom option can be used to install selective components or if you want to change the location of the installation.

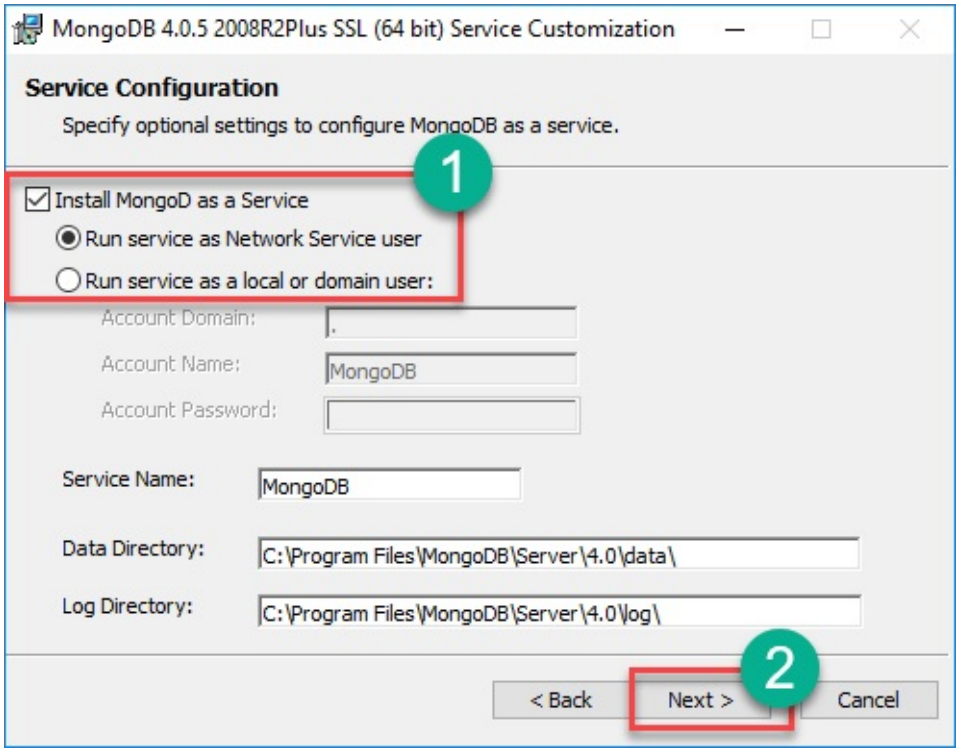


Step 5)

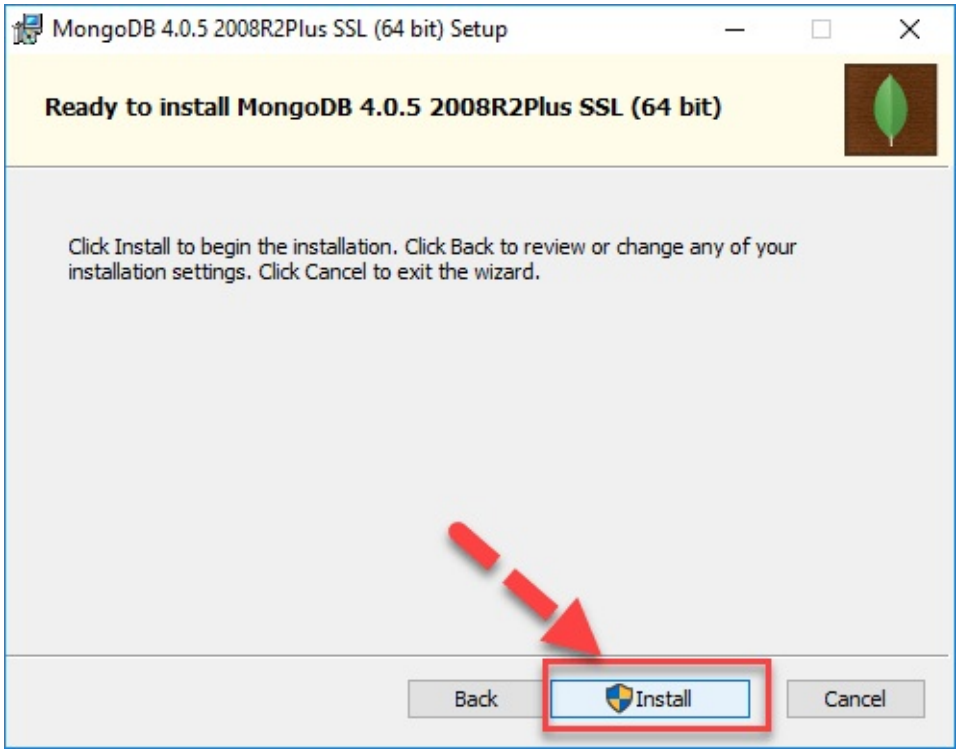
1. Select "Run service as Network Service user". make a note of the

data directory, we'll need this later.

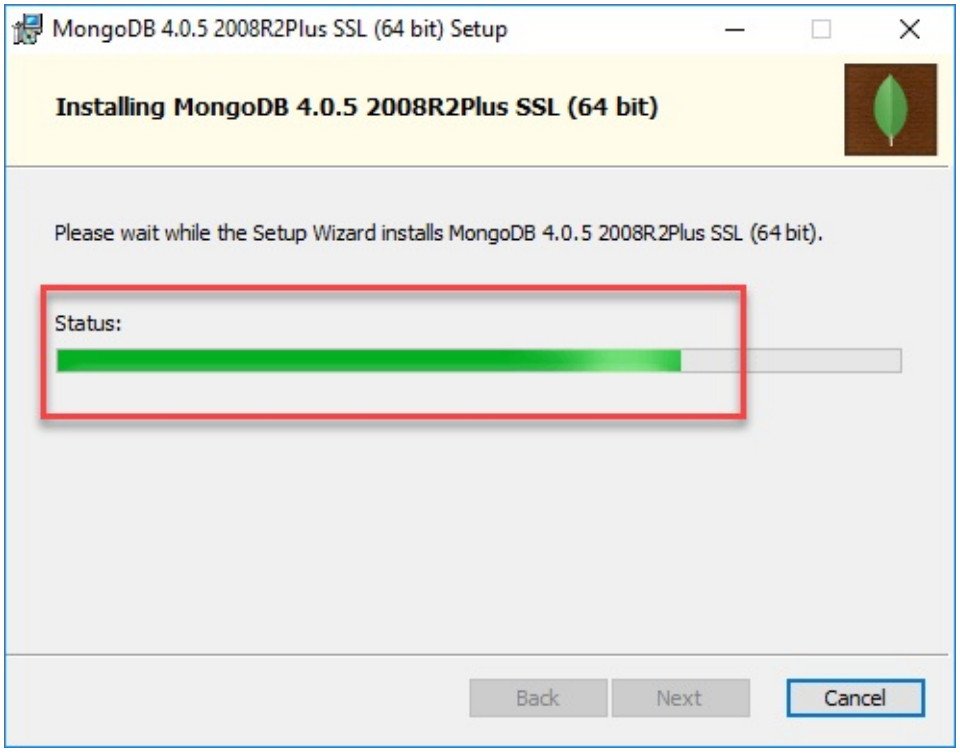
2. Click Next



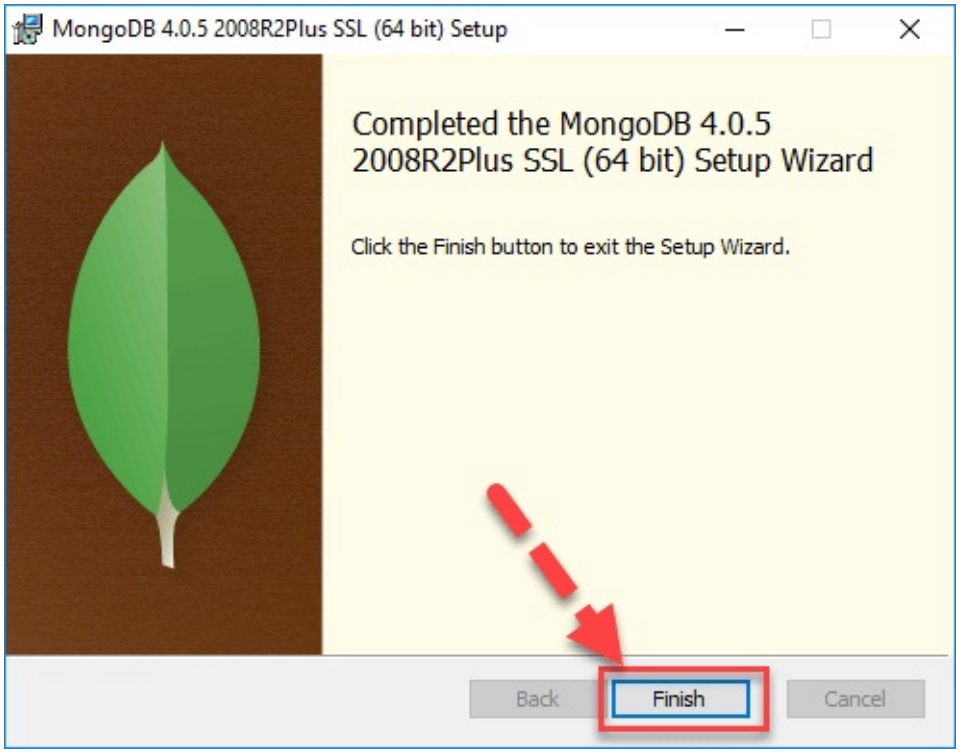
Step 6) Click on the Install button to start the installation.



Step 7) Installation begins. Click Next once completed



Step 8) Click on the Finish button to complete the installation

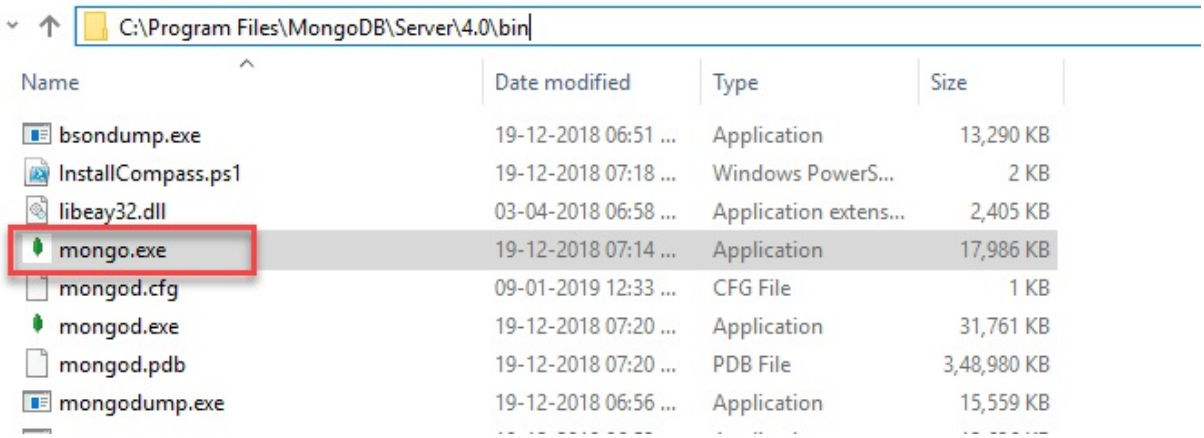


Hello World MongoDB: JavaScript Driver

Drivers in MongoDB are used for connectivity between client applications and the database. For example, if you had Java program and required it to connect to MongoDB then you would require to download and integrate the Java driver so that the program can work with the MongoDB database.

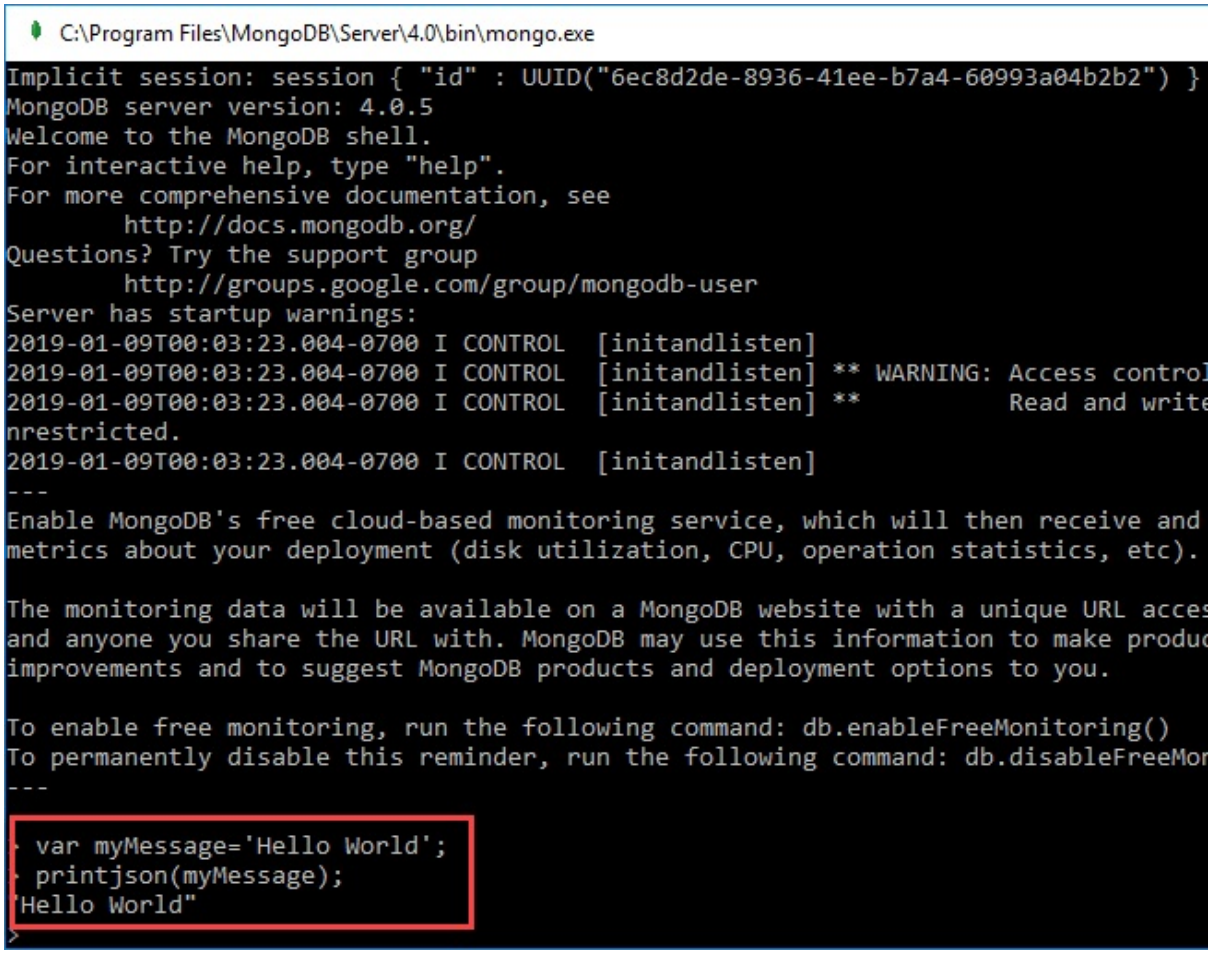
The driver for JavaScript comes out of the box. The MongoDB shell which is used to work with MongoDB database is actually a javascript shell. To access it

Step 1) Go to " C:\Program Files\MongoDB\Server\4.0\bin" and double click on mongo.exe. Alternatively, you can also click on the MongoDB desktop item



Step 2) Enter following program into shell

```
var myMessage='Hello World';
printjson(myMessage);
```



Code Explanation:

1. We are just declaring a simple Javascript variable to store a string called 'Hello World.'
2. We are using the printjson method to print the variable to the screen.

Install Python Driver

Step 1) Ensure Python is installed on the system

Step 2) Install the mongo related drivers by issuing the below command

```
pip install pymongo
```

Install Ruby Driver

Step 1) Ensure Ruby is installed on the system

Step 2) Ensure gems is updated by issuing the command

```
gem update -system
```

Step 3) Install the mongo related drivers by issuing the below command

```
gem install mong
```

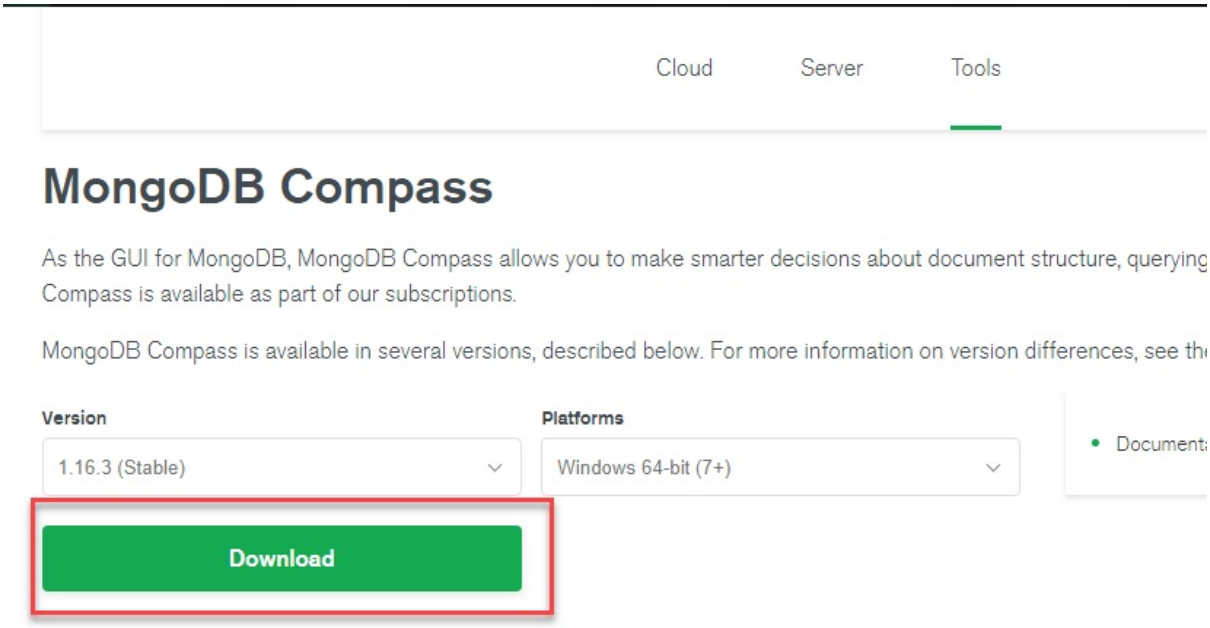
Install MongoDB Compass- MongoDB Management Tool

There are tools in the market which are available for managing MongoDB. One such non-commercial tool is MongoDB Compass.

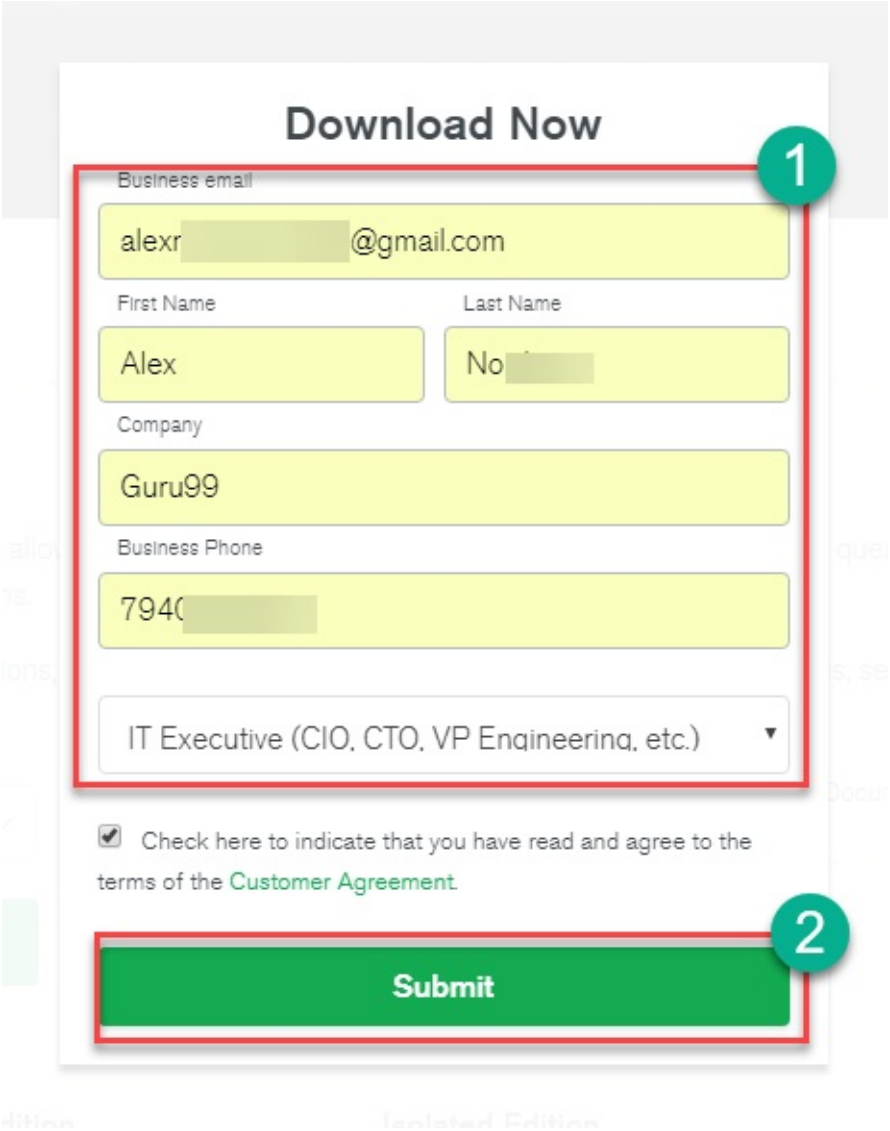
Some of the features of Compass are given below:

1. Full power of the Mongoshell
2. Multiple shells
3. Multiple results

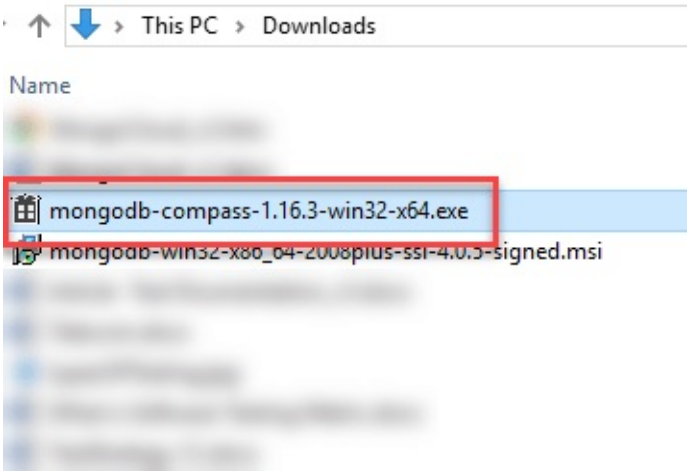
Step 1) Go to link and click download



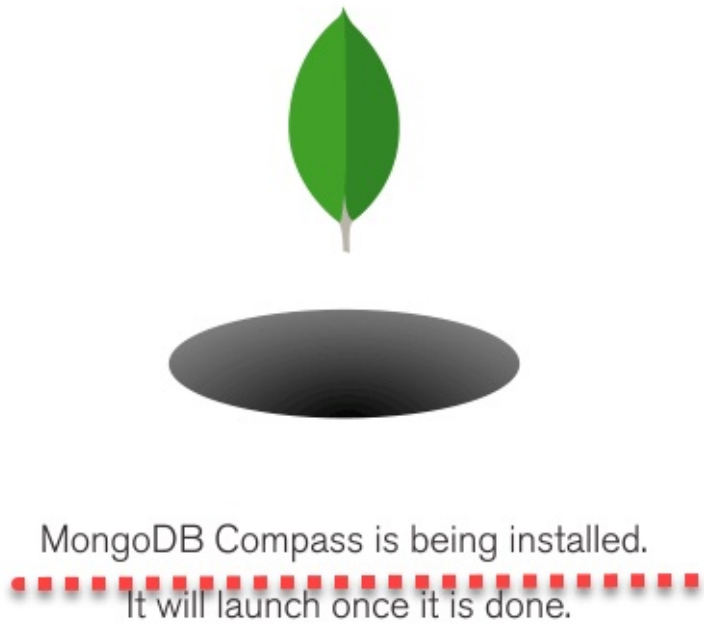
Step 2) Enter details in the popup and click submit



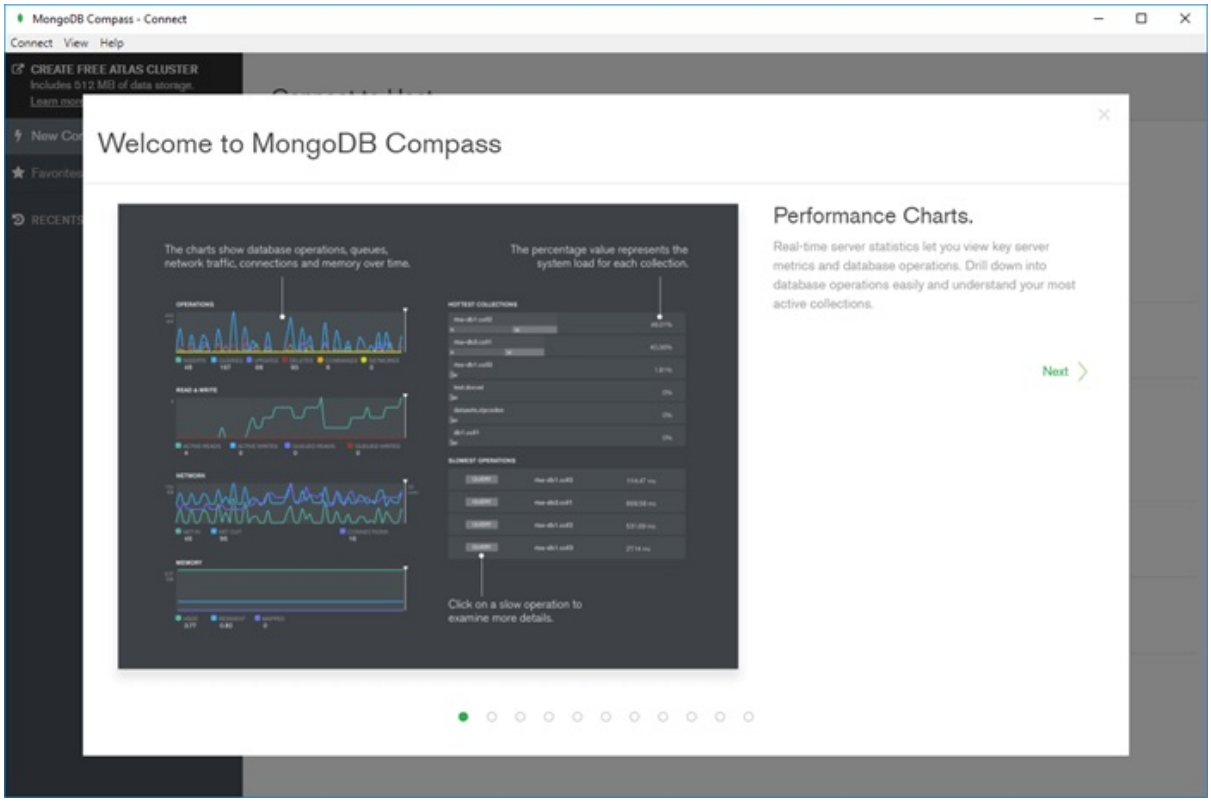
Step 3) Double click on the downloaded file



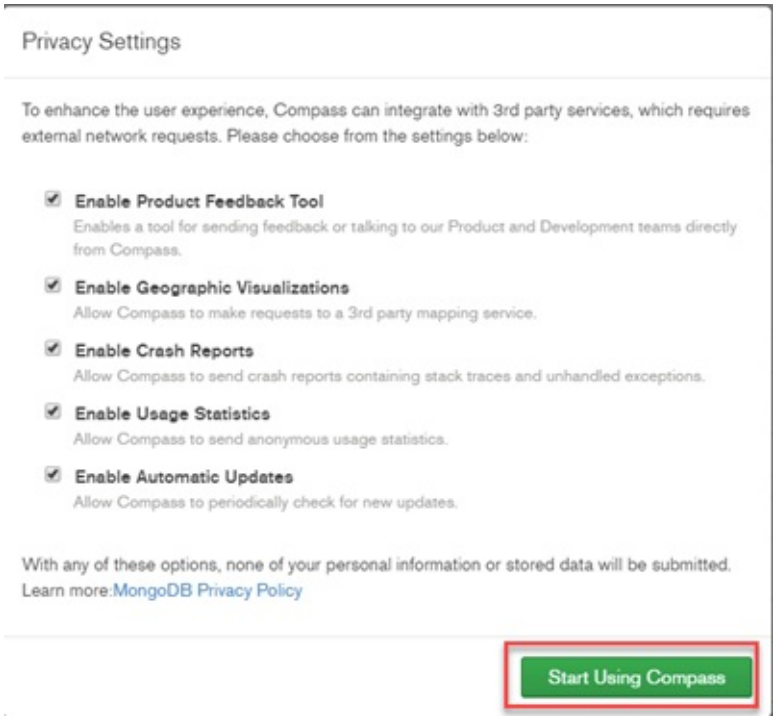
Step 4) Installation will auto-start



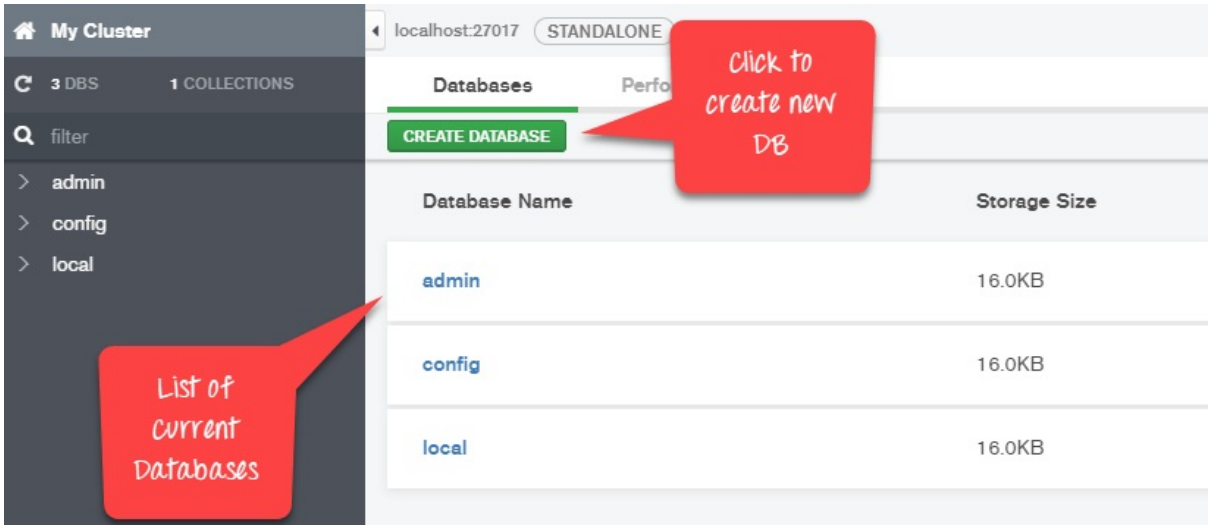
Step 5) Compass will launch with a Welcome screen



Step 6) Keep the privacy settings as default and Click “Start Using Compass”

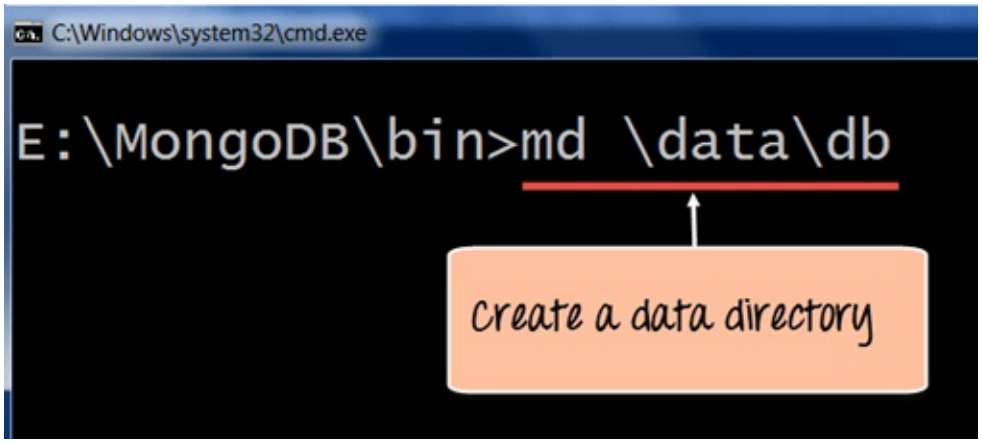


Step 7) You will see homescreen with list of current databases.



MongoDB Configuration, Import, and Export

Before starting the MongoDB server, the first key aspect is to configure the data directory where all the MongoDB data will be stored. This can be done in the following way



The above command 'md \data\db' makes a directory called \data\db in your current location.

MongoDB will automatically create the databases in this location, because this is the default location for MongoDB to store its information. We are just ensuring the directory is present, so that MongoDB can find it when it starts.

The import of data into MongoDB is done using the "mongoimport" command. The following example shows how this can be done.

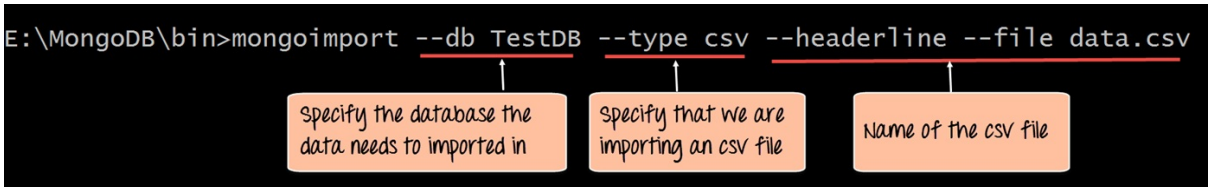
Step 1) Create a CSV file called data.csv and put the following data in it

Employeeid,EmployeeName

1. Guru99
2. Mohan
3. Smith

So in the above example, we are assuming we want to import 3 documents into a collection called data. The first row is called the header line which will become the Field names of the collection.

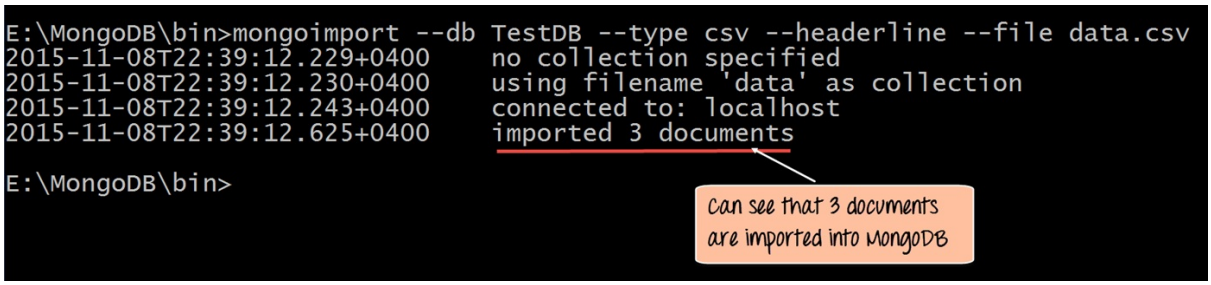
Step 2) Issue the mongo import command



Code Explanation:

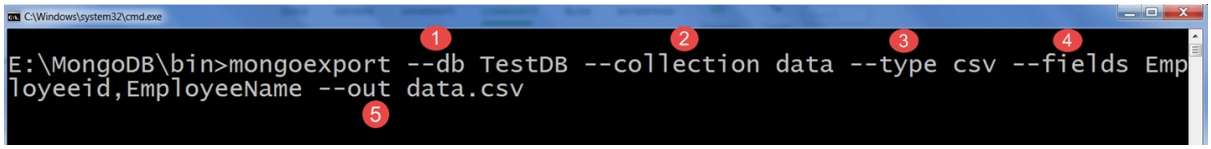
1. We are specifying the db option to say which database the data should be imported to
2. The type option is to specify that we are importing a csv file
3. Remember that the first row is called the header line which will become the Field names of the collection, that is why we specify the –headerline option. And then we specify our data.csv file.

Output



The output clearly shows that 3 documents were imported into MongoDB.

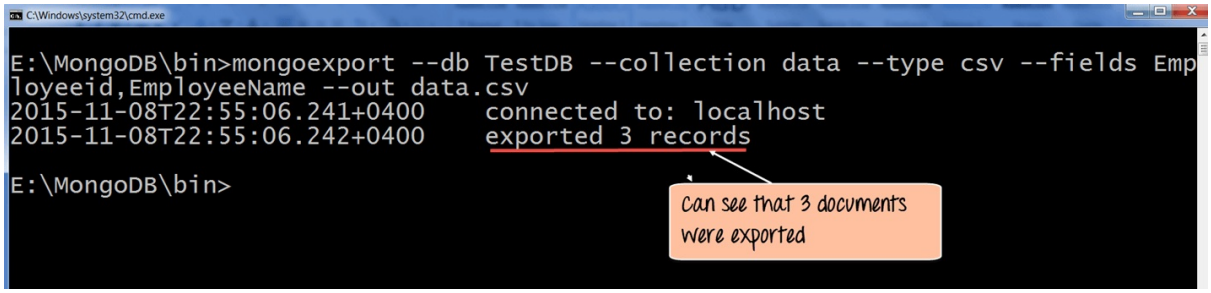
Exporting MongoDB is done by using the mongoexport command



Code Explanation:

1. We are specifying the db option to say which database the data should be exported from.
2. We are specifying the collection option to say which collection to use
3. The third option is to specify that we want to export to a csv file
4. The fourth is to specify which fields of the collection should be exported.
5. The –out option specifies the name of the csv file to export the data to.

Output



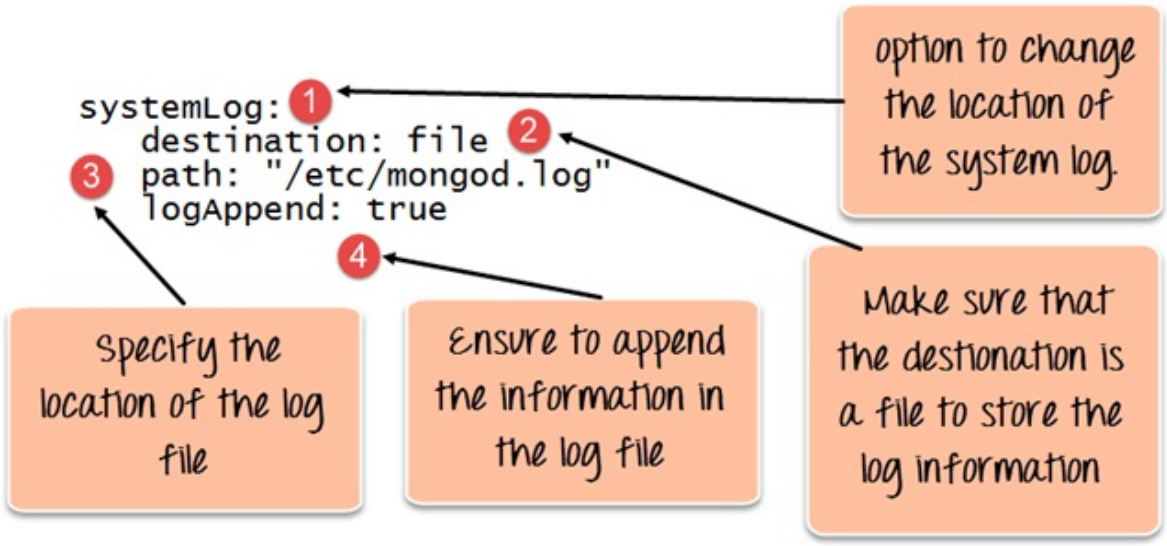
The output clearly shows that 3 records were exported from MongoDB.

Configuring MongoDB server with configuration file

One can configure the mongod server instance to startup with a configuration file. The configuration file contains settings that are equivalent to the mongod command-line options.

For example, supposed you wanted MongoDB to store all its logging information to a custom location then follow the below steps

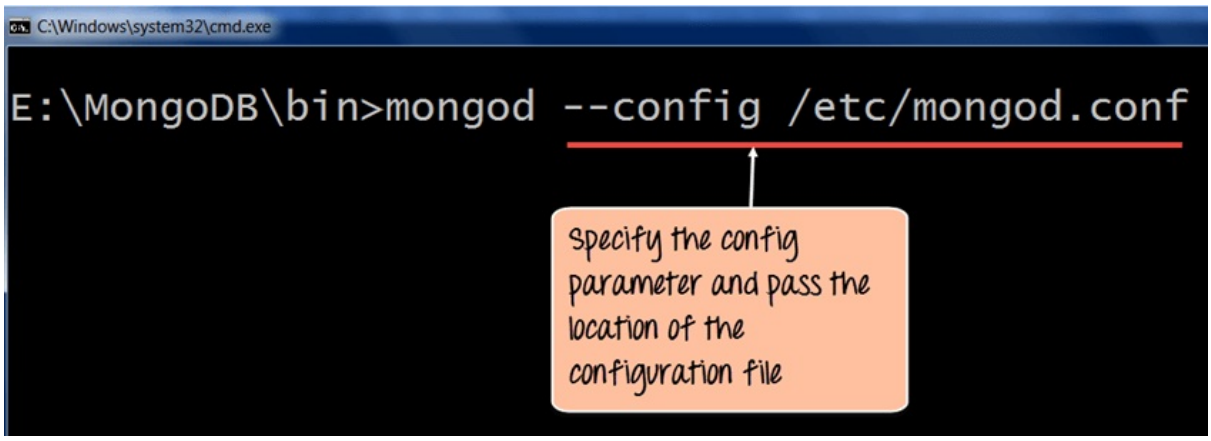
Step 1) Create a file called, “mongod.conf” and store the below information in the file



1. The first line of the file specifies that we want to add configuration for the system log file, that is where the information about what the server is doing in a custom log file.

2. The second option is to mention that the location will be a file.
3. This mentions the location of the log file
4. The logAppend: “true” means to ensure that the log information keeps on getting added to the log file. If you put the value as “false”, then the file would be deleted and created fresh whenever the server starts again.

Step 2) Start the mongod server process and specify the above created configuration file as a parameter. The screenshot of how this can be done is shown below



Once the above command is executed, the server process will start using this configuration file, and if you go to the /etc. directory on your system, you will see the mongod.log file created.

The below snapshot shows an example of what a log file would look like.

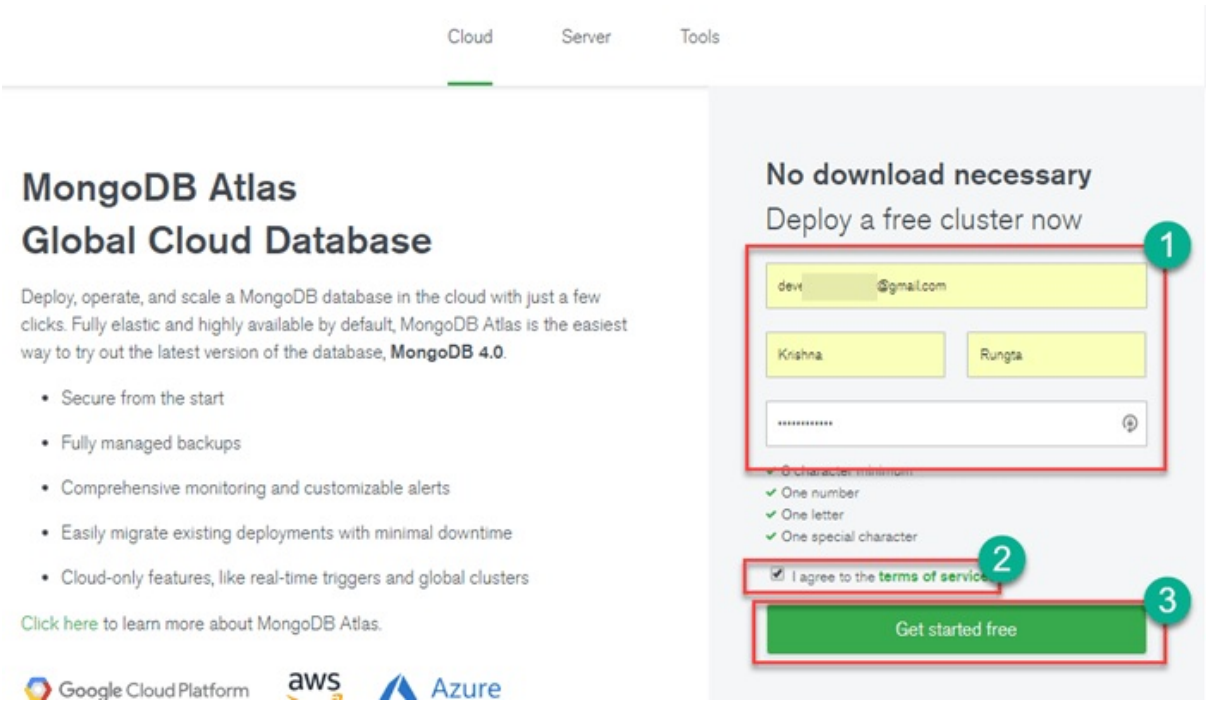
```
2015-11-18T17:09:39.892+0400 I CONTROL  Hotfix KB2731284 or later update is installed, no need to zero-out data files
2015-11-18T17:09:39.983+0400 I CONTROL  [initandlisten] MongoDB starting : pid=4328 port=27017 dbpath=E:\data\db\ 32-bit
host=Babuli-PC
2015-11-18T17:09:39.983+0400 I CONTROL  [initandlisten] ** NOTE: This is a 32-bit MongoDB binary running on a 64-bit operating
2015-11-18T17:09:39.983+0400 I CONTROL  [initandlisten] **       system. Switch to a 64-bit build of MongoDB to
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten] **       support larger databases.
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten]
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten]
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten] ** NOTE: This is a 32 bit MongoDB binary.
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten] **       32 bit builds are limited to less than 2GB of data (or less with --journal).
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten] **       Note that journaling defaults to off for 32 bit and is currently off.
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten] **       See http://dochub.mongodb.org/core/32bit
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten]
2015-11-18T17:09:39.984+0400 I CONTROL  [initandlisten] targetMinOS: Windows XP SP3
2015-11-18T17:09:39.985+0400 I CONTROL  [initandlisten] db version v3.0.7
2015-11-18T17:09:39.985+0400 I CONTROL  [initandlisten] git version: 6ce7cbe8c6b899552dadd907604559806aa2e9bd
2015-11-18T17:09:39.985+0400 I CONTROL  [initandlisten] build info: windows sys.getwindowsversion(major=6, minor=1, build=7601,
platform=2, service_pack='Service Pack 1') BOOST_LIB_VERSION=1_49
2015-11-18T17:09:39.985+0400 I CONTROL  [initandlisten] allocator: tcmalloc
2015-11-18T17:09:39.985+0400 I CONTROL  [initandlisten] options: { config: "E:\mongo.conf", systemLog: { destination: "file", logAppend:
true, path: "mongod.log" } }
2015-11-18T17:09:41.605+0400 I NETWORK  [initandlisten] waiting for connections on port 27017
2015-11-18T17:10:09.219+0400 I NETWORK  [initandlisten] connection accepted from 127.0.0.1:51310 #1 (1 connection now open)
```

Chapter 4: Install MongoDB in Cloud: AWS, Google, Azure

You do not need install the MongoDB server and configure it. You can deploy MongoDB Atlas server on the cloud on platforms like AWS, Google Cloud, Azure and connect to the instance using a client. Below are the detailed steps

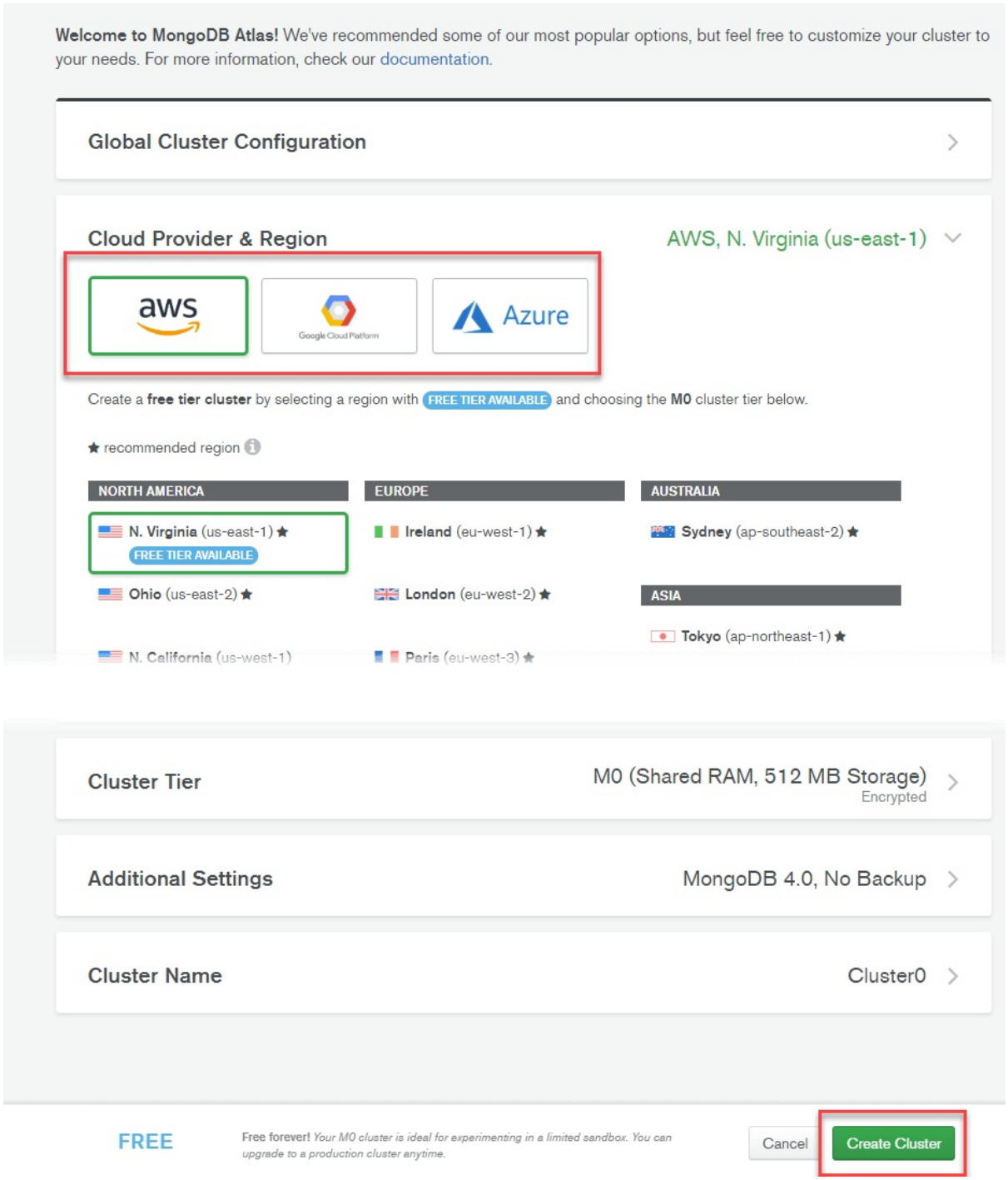
Step 1) Go to the link

1. Enter Personal Details
2. Agree to terms
3. Click button “Get Started Free”

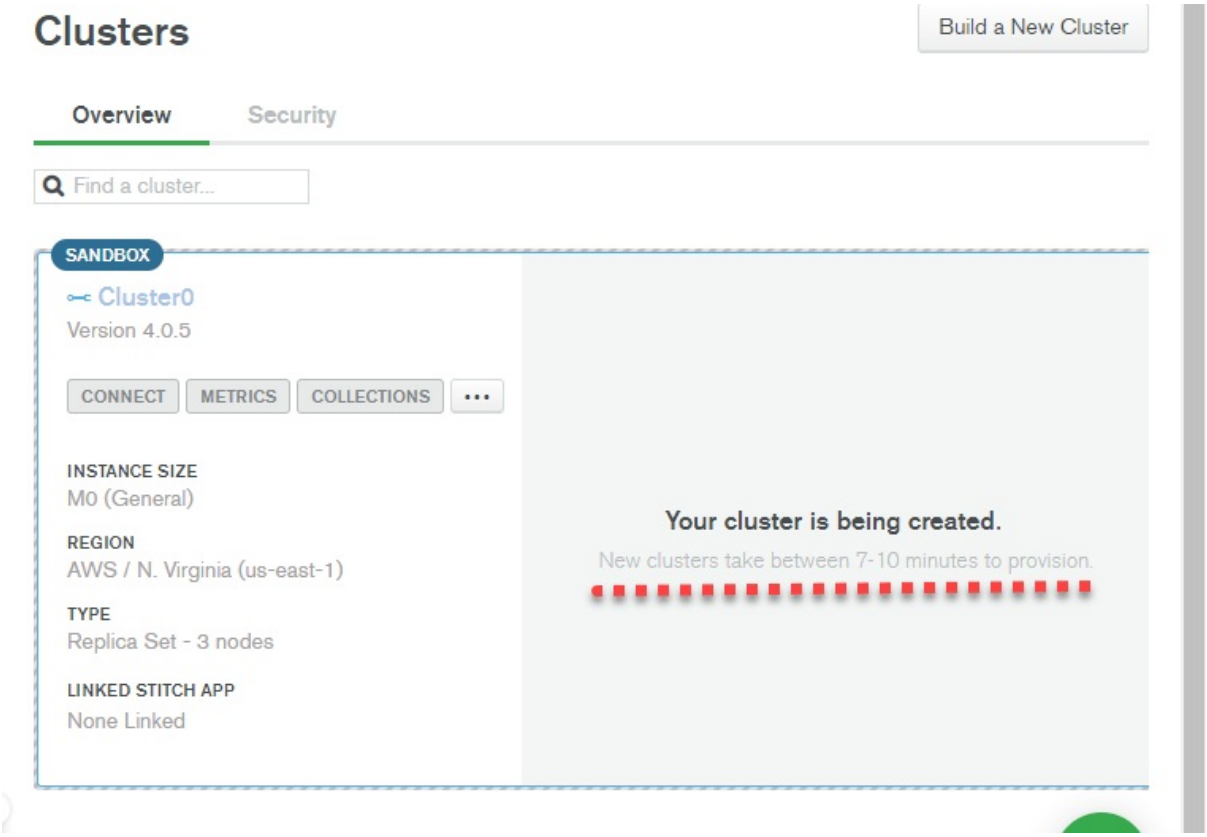


Step 2) Click “Build my first cluster”

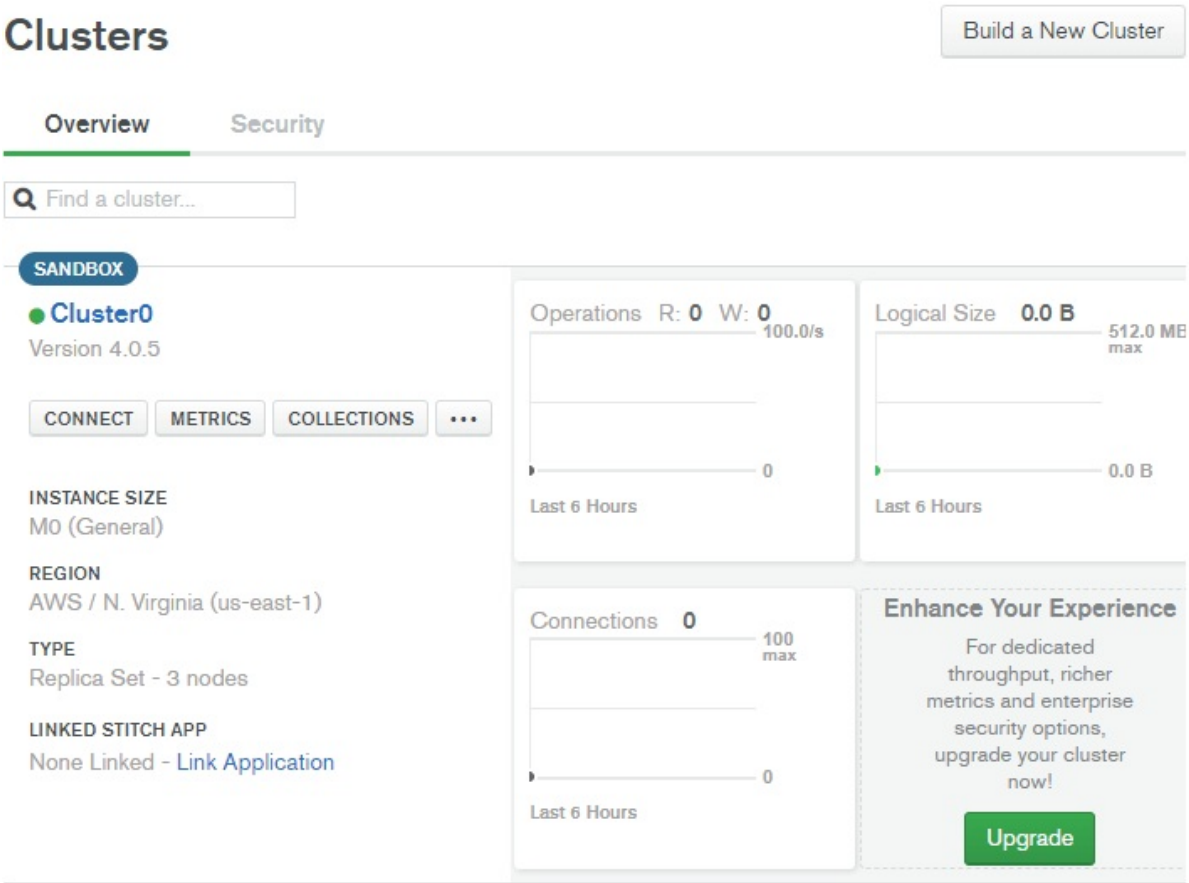
Step 3) You can select between AWS, Google Cloud, Azure as your cloud provider. In this tutorial, we will use AWS which set be default. Make no other changes on the page and click “Create Cluster.”



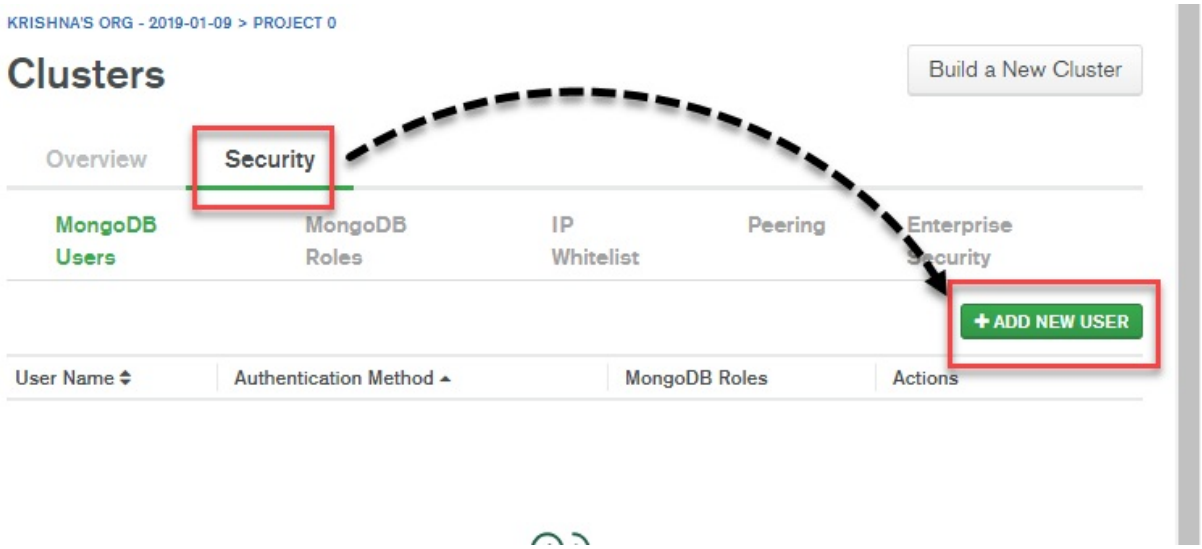
Step 4) Cluster creation takes some time:



Step 5) After some time you will see

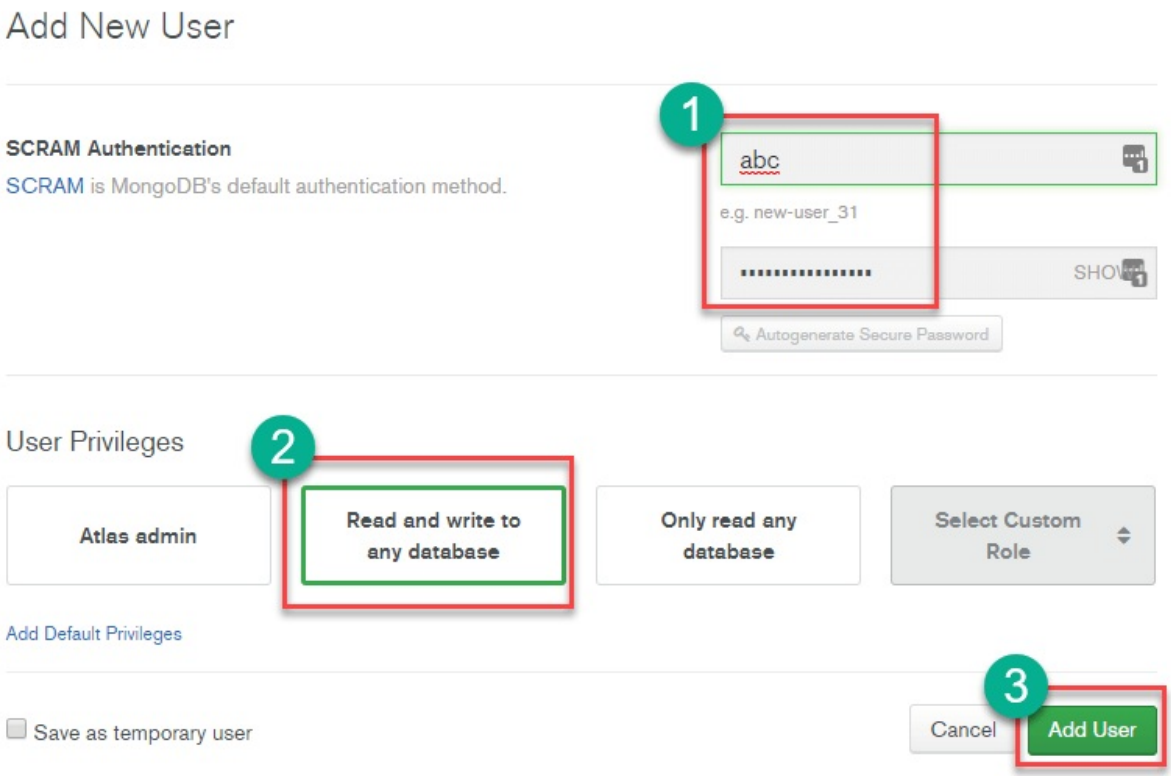


Step 6) Click Security > Add new user



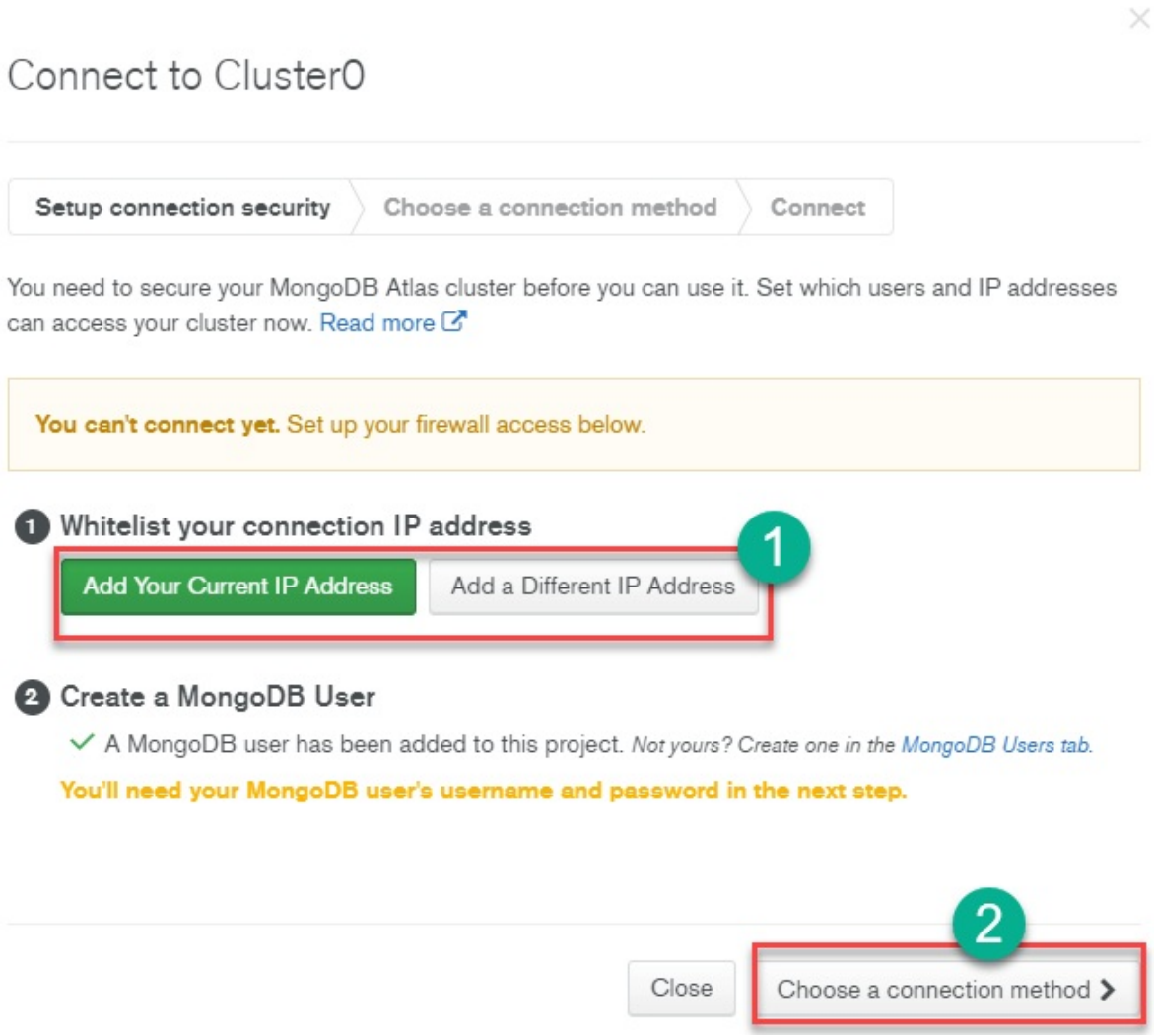
Step 7) In next screen,

1. Enter user credentials
2. Assign privileges
3. Click Add User button

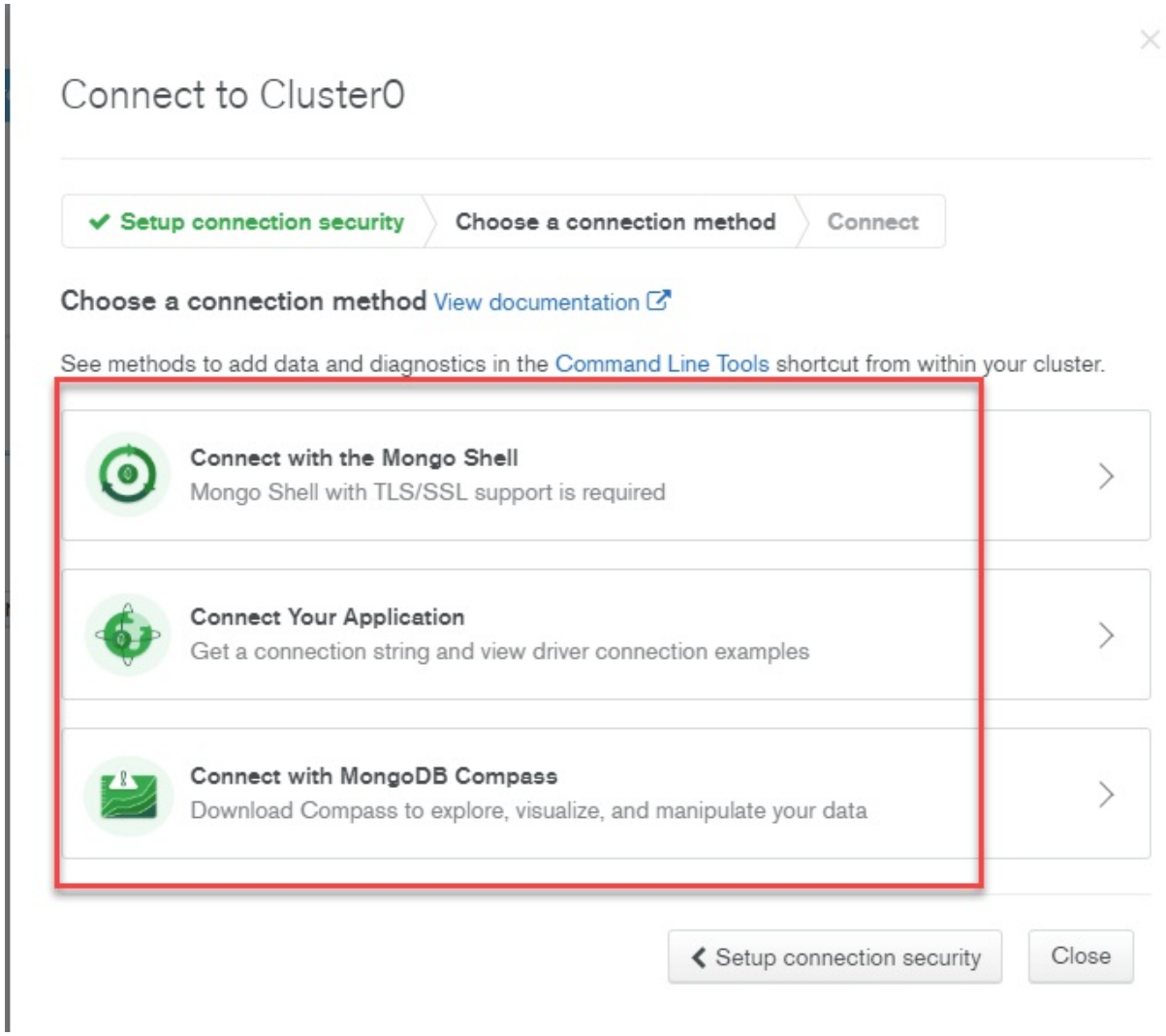


Step 8) In the dashboard, click connect button

1. Whitelist your IP connection
2. Choose the connection method



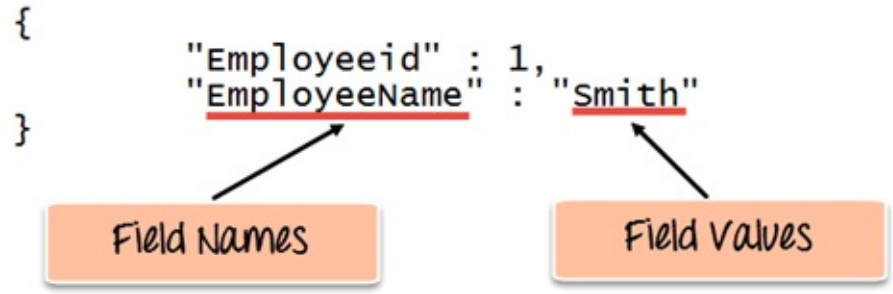
Step 9) Select the connection method of your choice to connect to MongoDB server


Connect to Cluster0
Setup connection security
Choose a connection method
Connect
Choose a connection method View documentation
See methods to add data and diagnostics in the Command Line Tools shortcut from within your cluster.
Connect with the Mongo Shell
Mongo Shell with TLS/SSL support is required
Connect Your Application
Get a connection string and view driver connection examples
Connect with MongoDB Compass
Download Compass to explore, visualize, and manipulate your data
Setup connection security
Close

Chapter 5: How to Create Database & Collection in MongoDB

In MongoDB, the first basic step is to have a database and collection in place. The database is used to store all of the collections, and the collection in turn is used to store all of the documents. The documents in turn will contain the relevant Field Name and Field values.

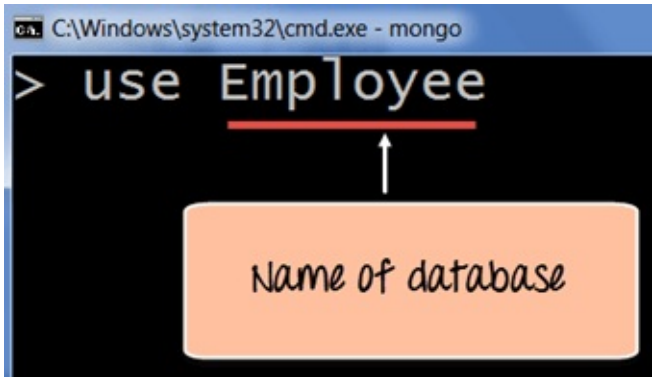
The snapshot below shows a basic example of how a document would look like.



The Field Names of the document are “Employeeid” and “EmployeeName” and the Field values are “1” and “Smith’ respectively. A bunch of documents would then make up a collection in MongoDB.

Creating a database using “use” command

Creating a database in MongoDB is as simple as issuing the “**using**” command. The following example shows how this can be done.

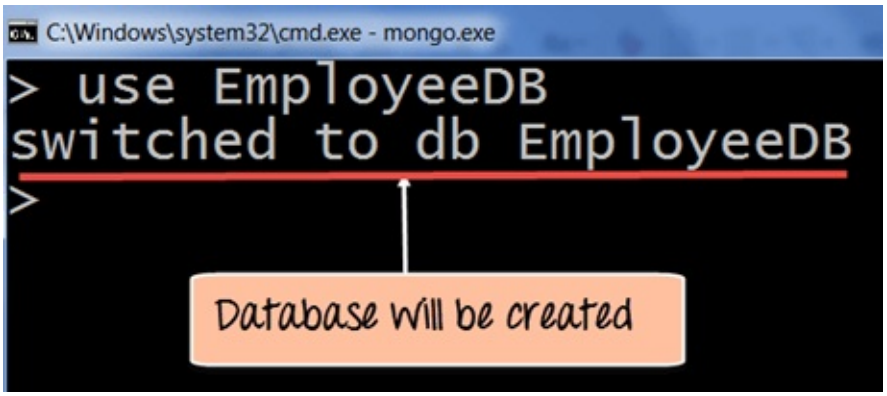


Code Explanation:

1. The **"use"** command is used to create a database in MongoDB. If the database does not exist a new one will be created.

If the command is executed successfully, the following Output will be shown:

Output:



MongoDB will automatically switch to the database once created.

Creating a Collection/Table using insert()

The easiest way to create a collection is to insert a record (which is nothing but a document consisting of Field names and Values) into a collection. If the collection does not exist a new one will be created.

The following example shows how this can be done.

```
db.Employee.insert
```

```
(
        {
                "Employeeid" : 1,
                "EmployeeName" : "Martin"
        }
)
```

Code Explanation:

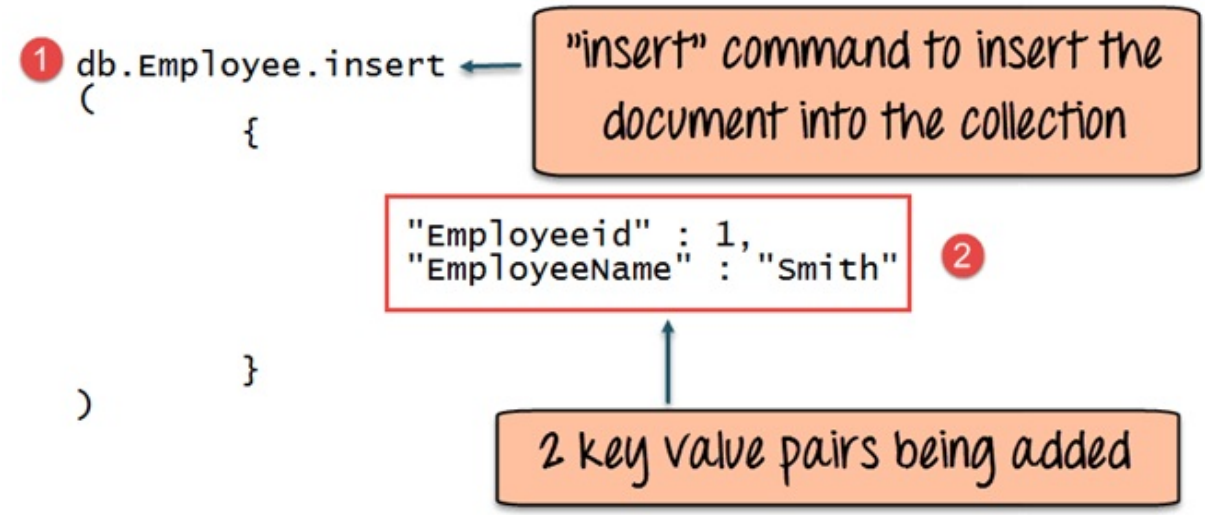
As seen above, by using the **"insert"** command the collection will be created.

Adding documents using insert() command

MongoDB provides the **insert () command** to insert documents into a collection. The following example shows how this can be done.

Step 1) Write the "insert" command

Step 2) Within the "insert" command, add the required Field Name and Field Value for the document which needs to be created.



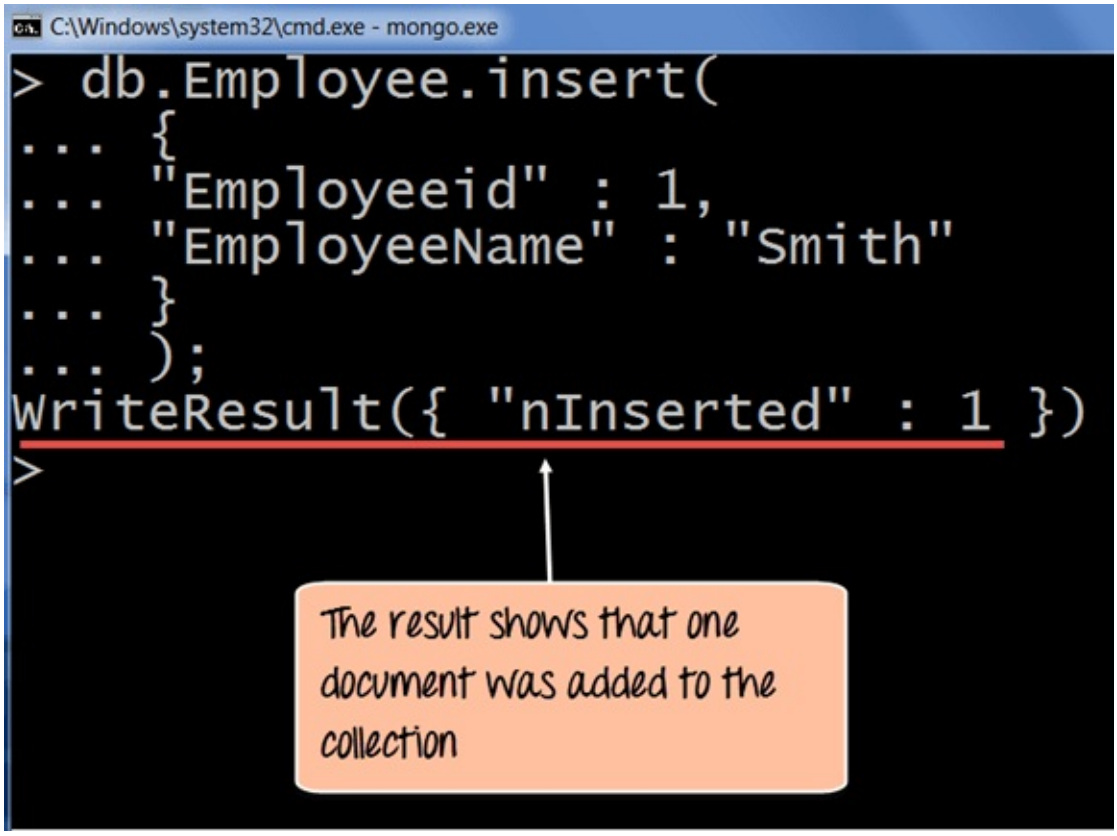
Code Explanation:

1. The first part of the command is the "**insert statement**" which is the statement used to insert a document into the collection.
2. The second part of the statement is to add the Field name and the

Field value, in other words, what is the document in the collection going to contain.

If the command is executed successfully, the following Output will be shown

Output:



The output shows that the operation performed was an insert operation and that one record was inserted into the collection.

Chapter 6: Add MongoDB Array using insert() with Example

The “insert” command can also be used to insert multiple documents into a collection at one time. The below code example can be used to insert multiple documents at a time.

The following example shows how this can be done,

Step 1) Create a JavaScript variable called myEmployee to hold the array of documents

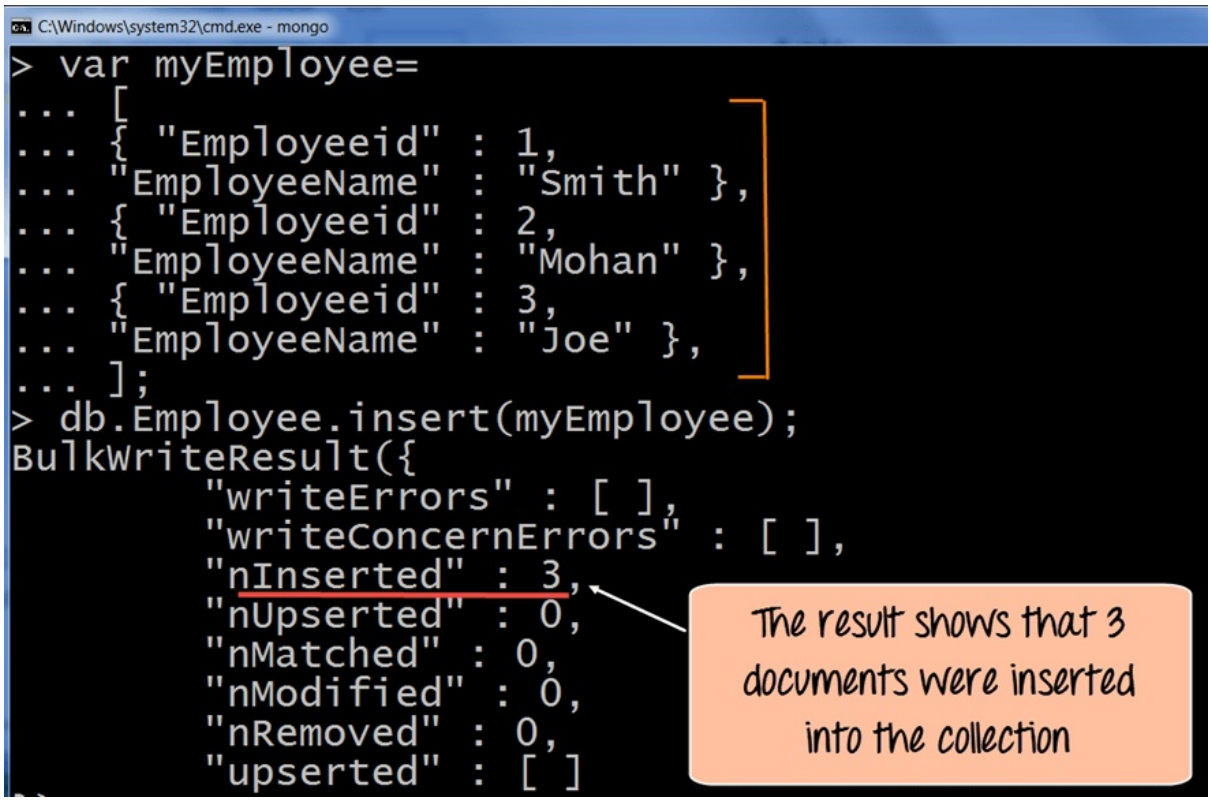
Step 2) Add the required documents with the Field Name and values to the variable

Step 3) Use the insert command to insert the array of documents into the collection

```
var myEmployee=
        [

                {
                        "Employeeid" : 1,
                        "EmployeeName" : "Smith"
                },
                {
                        "Employeeid"   : 2,
                        "EmployeeName" : "Mohan"
                },
                {
                        "Employeeid"   : 3,
                        "EmployeeName" : "Joe"
                },

        ];

        db.Employee.insert(myEmployee);
```

If the command is executed successfully, the following Output will be

shown



The output shows that those 3 documents were added to the collection.

Printing in JSON format

JSON is a format called **JavaScript Object Notation**, and is just a way to store information in an organized, easy-to-read manner. In our further examples, we are going to use the JSON print functionality to see the output in a better format.

Let's look at an example of printing in JSON format

```
db.Employee.find().forEach(printjson)
```

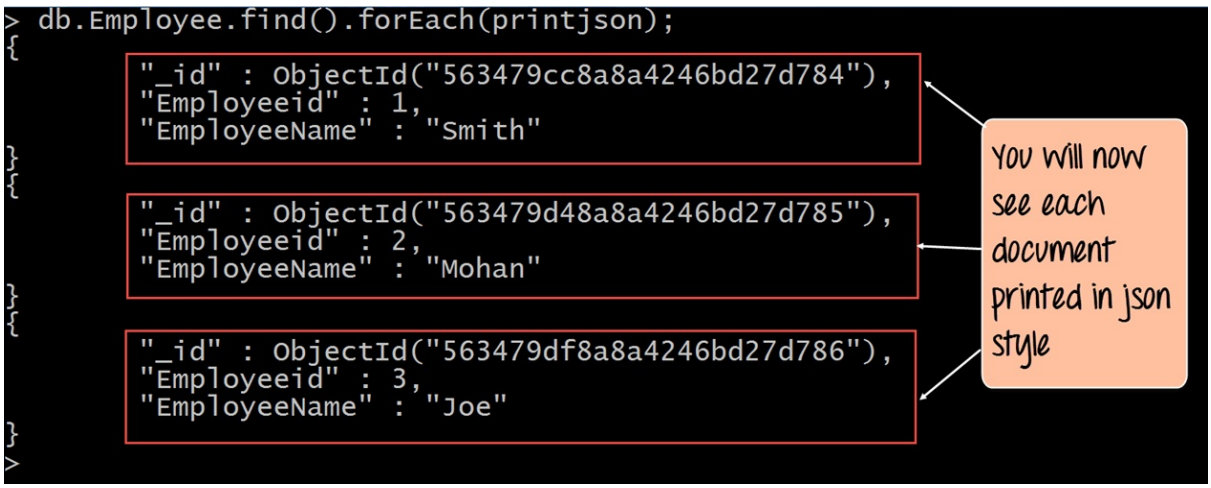
Code Explanation:

1. The first change is to append the function called for Each() to the find() function. What this does is that it makes sure to explicitly go through each document in the collection. In this way, you have more control of what you can do with each of the documents in the collection.

2. The second change is to put the printjson command to the forEach statement. This will cause each document in the collection to be displayed in JSON format.

If the command is executed successfully, the following Output will be shown

Output:



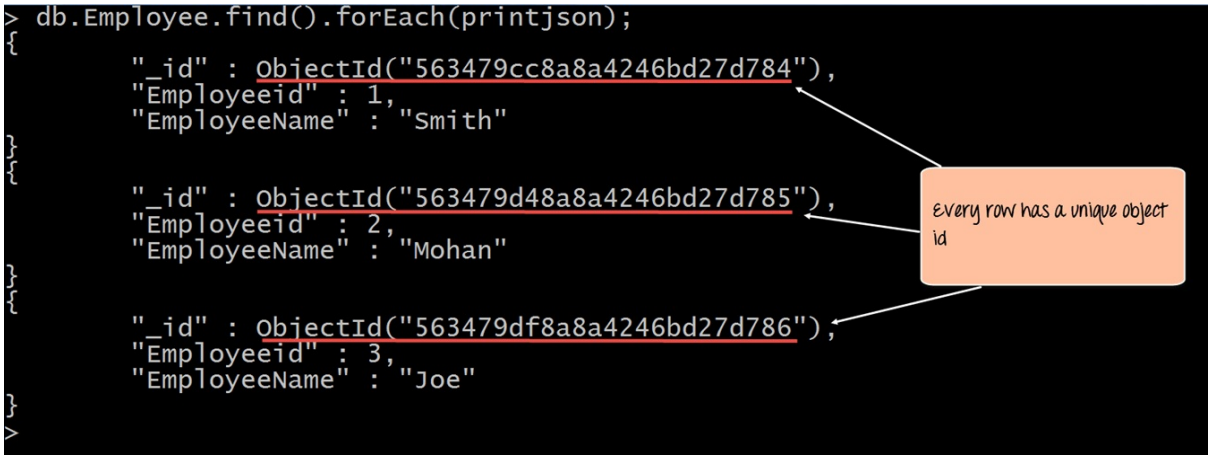
The output clearly shows that all of the documents are printed in JSON style.

Chapter 7: Mongodb Primary Key: Example to set _id field with ObjectId()

What is Primary Key in MongoDB?

In MongoDB, _id field as the primary key for the collection so that each document can be uniquely identified in the collection. The _id field contains a unique ObjectID value.

By default when inserting documents in the collection, if you don't add a field name with the _id in the field name, then MongoDB will automatically add an Object id field as shown below



When you query the documents in a collection, you can see the ObjectId for each document in the collection.

If you want to ensure that MongoDB does not create the _id Field when the collection is created and if you want to specify your own id as the _id of the collection, then you need to explicitly define this while creating the collection.

When explicitly creating an id field, it needs to be created with _id in its name.

Let's look at an example on how we can achieve this.

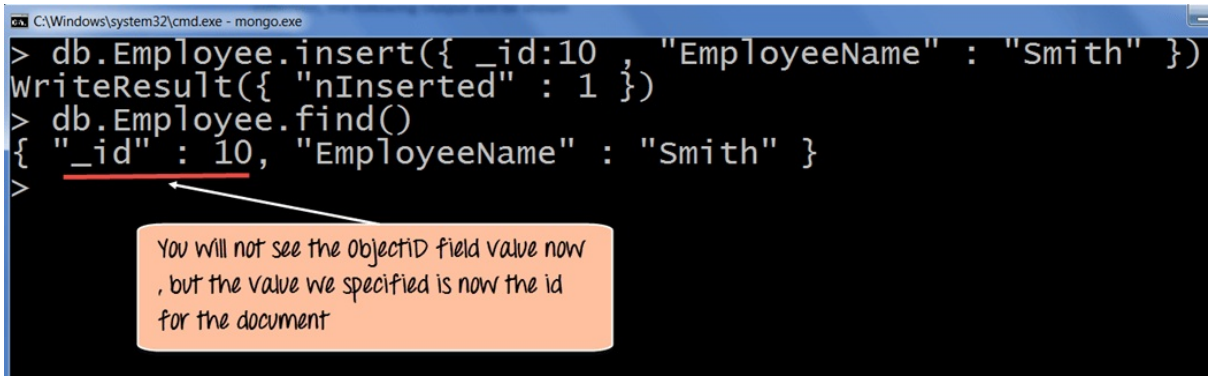
```
db.Employee.insert({_id:10, "EmployeeName" : "Smith"})
```

Code Explanation:

1. We are assuming that we are creating the first document in the collection and hence in the above statement while creating the collection, we explicitly define the field _id and define a value for it.

If the command is executed successfully and now use the find command to display the documents in the collection, the following Output will be shown

Output:



The output clearly shows that the _id field we defined while creating the collection is now used as the primary key for the collection.

Chapter 8: MongoDB Query Document using find() with Example

The method of fetching or getting data from a MongoDB database is carried out by using queries. While performing a query operation, one can also use criteria's or conditions which can be used to retrieve specific data from the database.

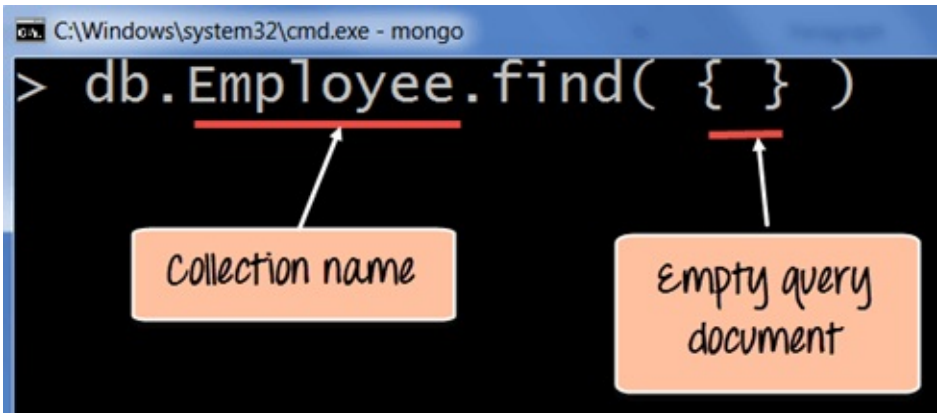
MongoDB provides a function called **db.collection.find ()** which is used for retrieval of documents from a MongoDB database.

During the course of this tutorial, you will see how this function is used in various ways to achieve the purpose of document retrieval.

Basic query operations

The basic query operations cover the simple operations such as getting all of the documents in a MongoDB collection. Let's look at an example of how we can accomplish this.

All of our code will be run in the MongoDB JavaScript command shell. Consider that we have a collection named 'Employee' in our MongoDB database and we execute the below command.

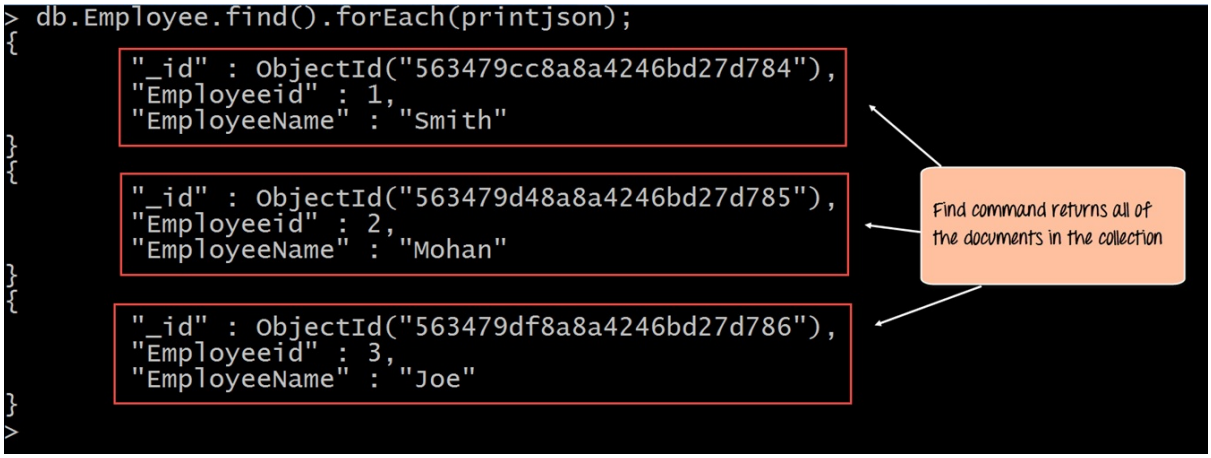


Code Explanation:

1. Employee is the collection name in the MongoDB database
2. The find command is an in-built function which is used to retrieve the documents in the collection.

If the command is executed successfully, the following Output will be shown

Output:



The output shows all the documents which are present in the collection.

We can also add criteria to our queries so that we can fetch documents based on certain conditions.

Example 1

Let's look at a couple of examples of how we can accomplish this.

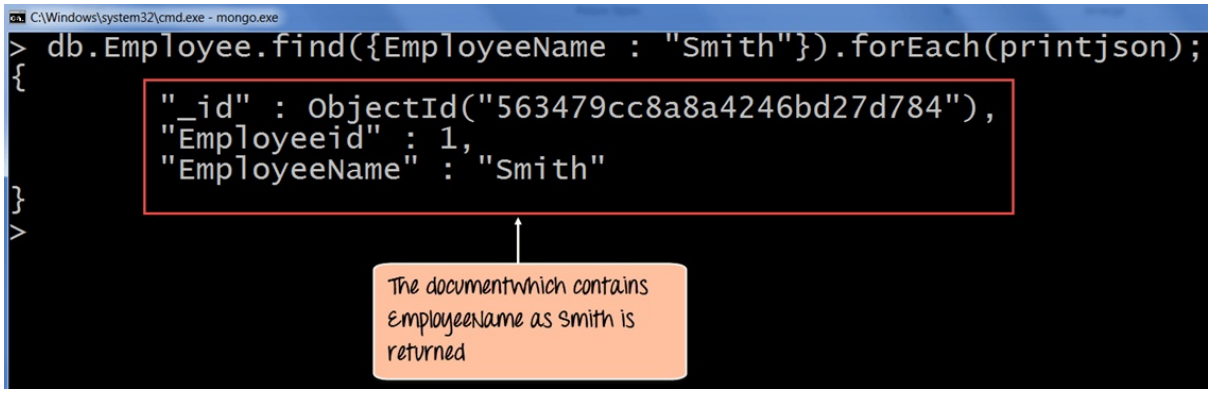
```
db.Employee.find({EmployeeName : "Smith"}).forEach(printjson);
```

Code Explanation:

1. Here we want to find for an Employee whose name is "Smith" in the collection , hence we enter the filter criteria as EmployeeName : "Smith"

If the command is executed successfully, the following Output will be shown

Output:



The output shows that only the document which contains “Smith” as the Employee Name is returned.

Example 2

Now, let’s take a look at another code example which makes use of the greater than search criteria. When this criteria is included, it actually searches those documents where the value of the field is greater than the specified value.

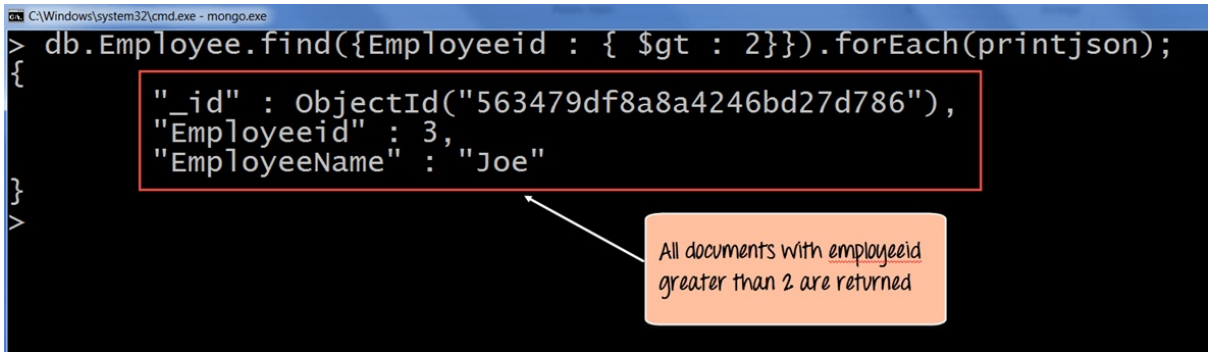
```
db.Employee.find({Employeeid : {$gt:2}}).forEach(printjson);
```

Code Explanation:

1. Here we want to find for all Employee’s whose id is greater than 2. The $gt is called a query selection operator, and what is just means is to use the greater than expression.

If the command is executed successfully, the following Output will be shown

Output:



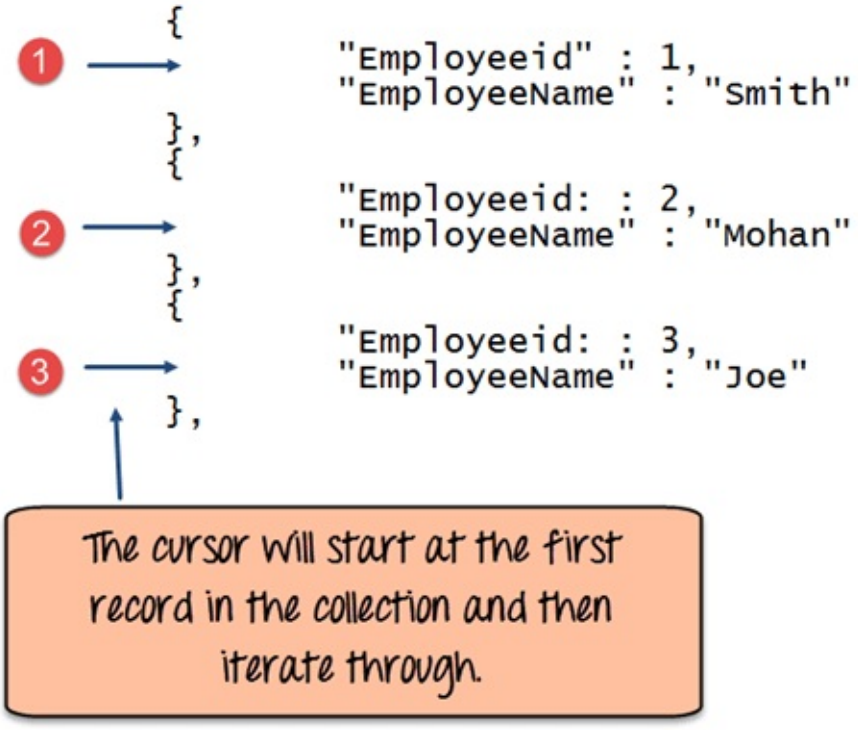
All of the documents wherein the Employee id is greater than 2 is returned.

Chapter 9: MongoDB Cursor Tutorial: Learn with EXAMPLE

What is Cursor in MongoDB?

When the **db.collection.find ()** function is used to search for documents in the collection, the result returns a pointer to the collection of documents returned which is called a cursor.

By default, the cursor will be iterated automatically when the result of the query is returned. But one can also explicitly go through the items returned in the cursor one by one. If you see the below example, if we have 3 documents in our collection, the cursor object will point to the first document and then iterate through all of the documents of the collection.



The following example shows how this can be done.

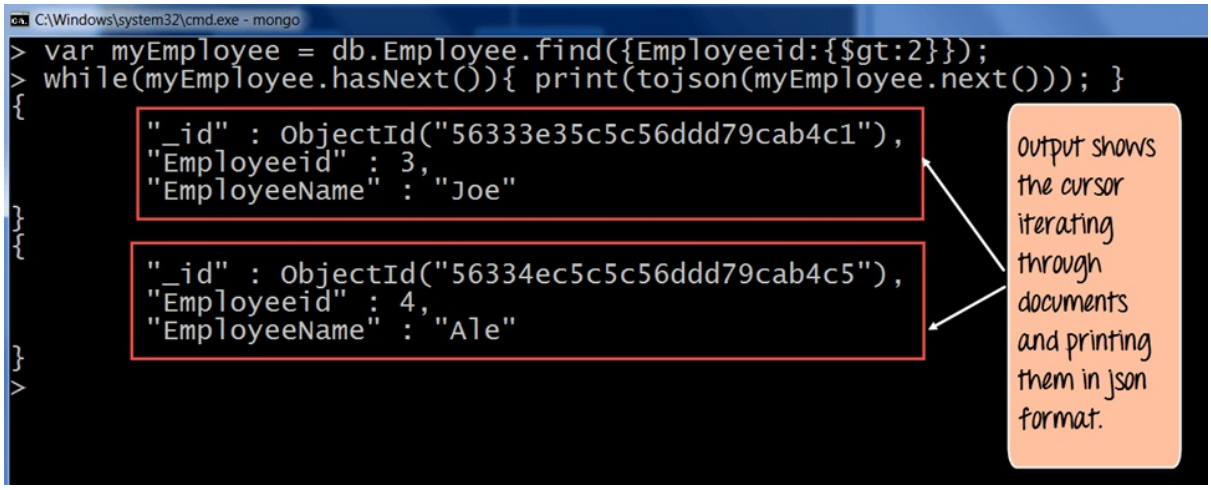
```
var myEmployee = db.Employee.find( { Employeeid : { $gt:2 }});

        while(myEmployee.hasNext())

        {

                print(tojson(myEmployee.next()));

        }
```

Code Explanation:

1. First we take the result set of the query which finds the Employee's whose id is greater than 2 and assign it to the JavaScript variable 'myEmployee'
2. Next we use the while loop to iterate through all of the documents which are returned as part of the query.
3. Finally for each document, we print the details of that document in JSON readable format.

If the command is executed successfully, the following Output will be shown

Output:



Chapter 10: MongoDB order with Sort() & Limit() Query with Examples

What is Query Modifications?

Mongo DB provides query modifiers such as the 'limit' and 'Orders' clause to provide more flexibility when executing queries. We will take a look at the following query modifiers

MongoDB Limit Query Results

This modifier is used to limit the number of documents which are returned in the result set for a query. The following example shows how this can be done.

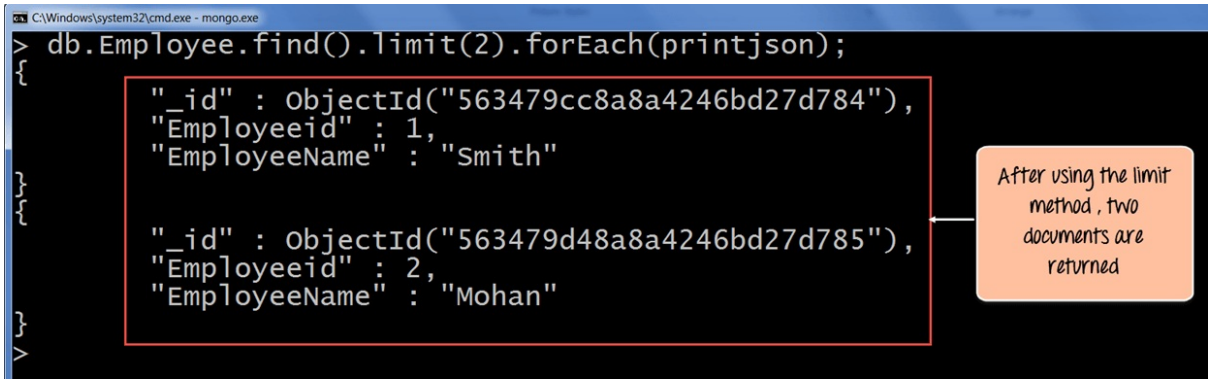
```
db.Employee.find().limit(2).forEach(printjson);
```

Code Explanation:

1. The above code takes the find function which returns all of the documents in the collection but then uses the limit clause to limit the number of documents being returned to just 2.

Output:

If the command is executed successfully, the following Output will be shown



The output clearly shows that since there is a limit modifier, so at most just 2 records are returned as part of the result set based on the ObjectId in ascending order.

MongoDB Sort by Descending Order

One can specify the order of documents to be returned based on ascending or descending order of any key in the collection. The following example shows how this can be done.

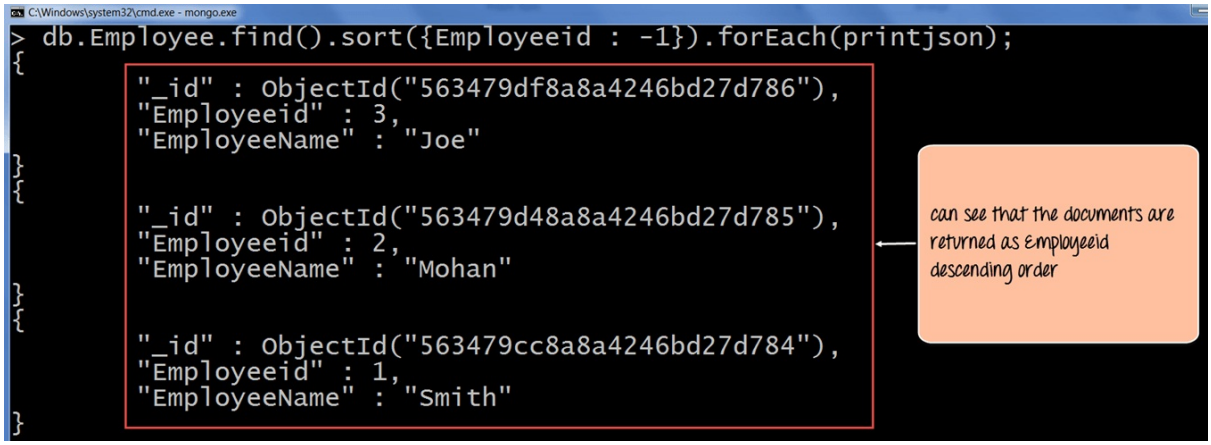
```
db.Employee.find().sort({Employeeid:-1}).forEach(printjson)
```

Code Explanation:

1. The above code takes the sort function which returns all of the documents in the collection but then uses the modifier to change the order in which the records are returned. Here the -1 indicates that we want to return the documents based on the descending order of Employee id.

If the command is executed successfully, the following Output will be shown

Output:



The output clearly shows the documents being returned in descending order of the Employeeid.

Ascending order is defined by value 1.

Chapter 11: MongoDB Count() & Remove() Functions with Examples

The concept of aggregation is to carry out a computation on the results which are returned in a query. For example, suppose you wanted to know what is the count of documents in a collection as per the query fired, then MongoDB provides the count() function.

Let's look at an example of this.

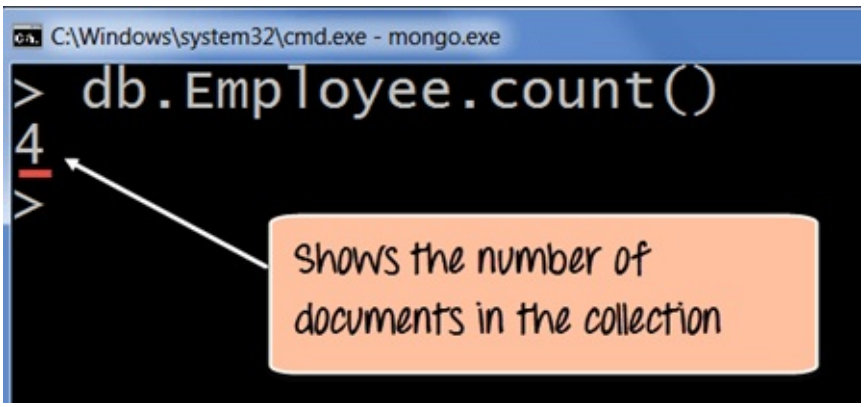
```
db.Employee.count()
```

Code Explanation:

1. The above code executes the count function.

If the command is executed successfully, the following Output will be shown

Output:



The output clearly shows that 4 documents are there in the collection.

Performing Modifications

The other two classes of operations in MongoDB are the update and remove statements.

The update operations allow one to modify existing data, and the remove operations allow the deletion of data from a collection.

Deleting Documents

In MongoDB, the **db.collection.remove ()** method is used to remove documents from a collection. Either all of the documents can be removed from a collection or only those which matches a specific condition.

If you just issue the remove command, all of the documents will be removed from the collection.

The following code example demonstrate how to remove a specific document from the collection.

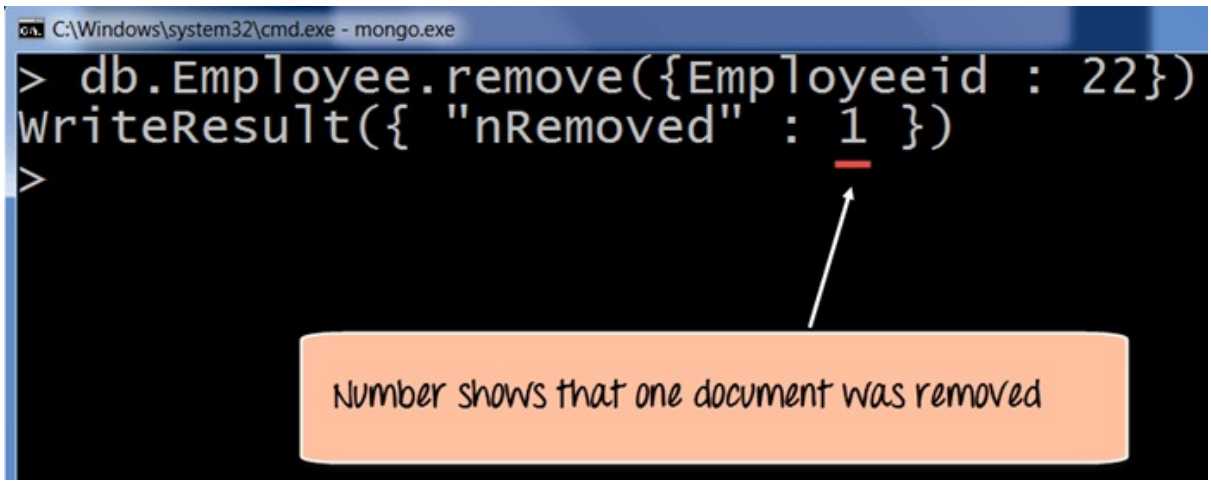
```
db.Employee.remove({Employeeid:22})
```

Code Explanation:

1. The above code use the remove function and specifies the criteria which in this case is to remove the documents which have the Employee id as 22.

If the command is executed successfully, the following Output will be shown

Output:



The output will show that 1 document was modified.

Chapter 12: MongoDB Update() Document with Example

Basic document updates

MongoDB provides the update() command to update the documents of a collection. To update only the documents you want to update, you can add a criteria to the update statement so that only selected documents are updated.

The basic parameters in the command is a condition for which document needs to be updated, and the next is the modification which needs to be performed.

The following example shows how this can be done.

Step 1) Issue the update command

Step 2) Choose the condition which you want to use to decide which document needs to be updated. In our example, we want to update the document which has the Employee id 22.

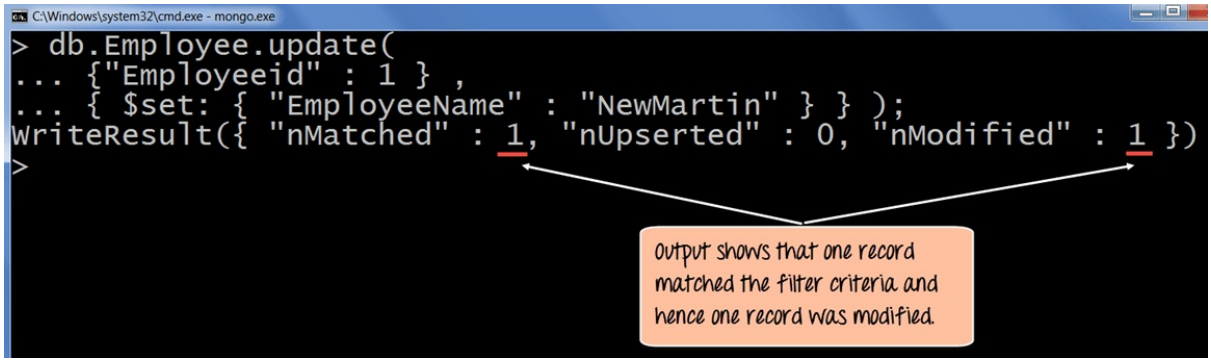
Step 3) Use the set command to modify the Field Name

Step 4) Choose which Field Name you want to modify and enter the new value accordingly.

```
db.Employee.update(
{"Employeeid" : 1},
{$set: { "EmployeeName" : "NewMartin"}});
```

If the command is executed successfully, the following Output will be shown

Output:



The output clearly shows that one record matched the condition and hence the relevant field value was modified.

Updating Multiple Values

To ensure that multiple/bulk documents are updated at the same time in MongoDB you need to use the multi option because otherwise by default only one document is modified at a time.

The following example shows how to update many documents.

In this example, we are going to first find the document which has the Employee id as “1” and change the Employee name from “Martin” to “NewMartin”

Step 1) Issue the update command

Step 2) Choose the condition which you want to use to decide which document needs to be updated. In our example, we want the document which has the Employee id of “1” to be updated.

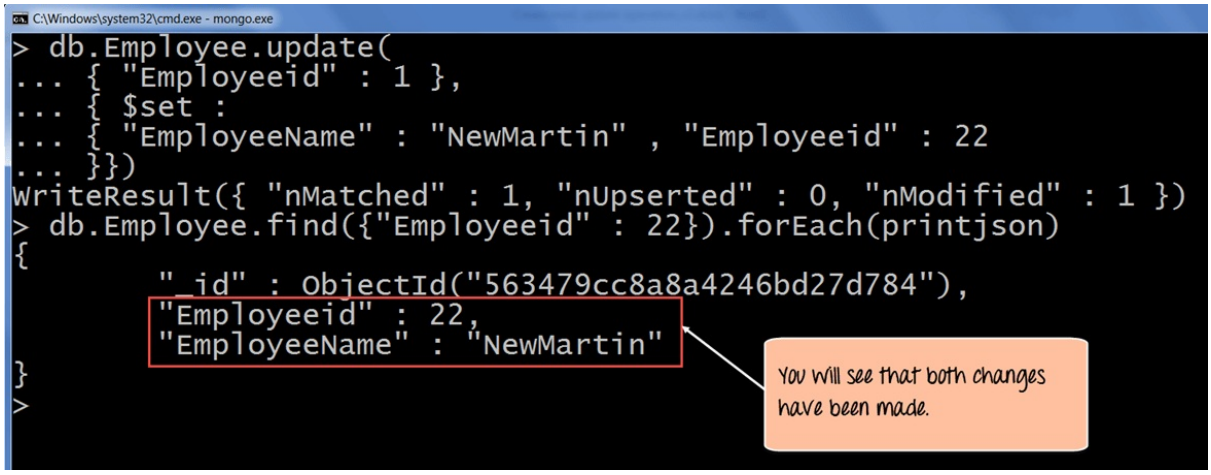
Step 3) Choose which Field Name’s you want to modify and enter their new value accordingly.

```
db.Employee.update
(
        {
                Employeeid : 1
        },
        {
                $set :
                {
                        "EmployeeName" : "NewMartin"
```

```
                        "Employeeid" : 22
                }
        }
)
```

If the command is executed successfully and if you run the **"find"** command to search for the document with Employee id as 22 you will see the following Output will be shown

Output:



The output clearly shows that one record matched the condition and hence the relevant field value was modified.

Chapter 13: MongoDB Security, Backup & Monitoring

One of the key concepts in MongoDB is the management of databases. Important aspects such as security, backup, access to databases are all important concepts when it comes to database administration.

MongoDB Security Overview

MongoDB has the ability to define security mechanisms to databases. By default one wouldn't want everyone to have an open access to every database in MongoDB, hence the requirement for having some sort of security mechanism in MongoDB is important.

Following are the best practices when implementing security in databases

1. Enable access control – Create users so that all applications and users are enforced to have some sort of authentication mechanism when accessing databases on MongoDB.
2. Configure role based access control – Sometimes there can be a logical grouping of permissions which may be required, which can be clubbed in roles. Users can then be assigned to these roles.
3. Try to configure MongoDB to use some sort of encryption protocol such as TLS or SSL. These protocols can be used to encrypt the traffic which flows between the client and the mongo DB environment.
4. Configure auditing – Administrators normally need to know who is doing what, which helps in analyzing problems later on. The best way is to enable auditing in MongoDB.

5. Run MongDB server instance with a separate user id which has access to the required resources on the server environment.

Mongodb Backup Procedures

When working with MongDB it is important to always ensure a backup procedure is in place in case the data within MongoDB gets corrupted for any reason.

Below are the backup mechanisms available from within MongoDB

1. **Backup by Copying Underlying Data Files** – This is probably the easiest mechanism , all that needs to be done is to copy the data files on which MongoDB resides and copy it to another location which ideally should be another server.
2. **Backup a Database with mongodump** - The mongodump tool reads data from a MongoDB database and creates high fidelity BSON files. What needs to be taken into consideration is that if the data set is large in volume, then mongodump can be very resource intensive, so then to mitigate this problem, the utility should be run on a secondary server.
3. **MongoDB Cloud Manager Backup** - MongoDB Cloud Manager continually backs up MongoDB replica sets and sharded clusters by reading the oplog data from the MongoDB environment. MongoDB Cloud Manager can create a point in time recovery by storing oplog data so that it can create a restore at any moment in time for a particular replica set or sharded cluster.

Mongodb Monitoring

Monitoring is one of the most critical administrative activities in MongoDB. This is because you can be more proactive by monitoring the environment for possible issues which could crop up.

Below are some of the ways of implementing monitoring

1. mongostat will tell you how many time database operations such as insert, query, update, delete, etc. actually occur on the server. This will give a good idea on how much the load the server is handling and will indicate whether you need additional resources on the server or maybe additional servers to distribute the load.
2. mongotop tracks and reports the current read and write activity of a MongoDB instance, and reports these statistics on a per collection basis.
3. MongoDB provides a web interface that exposes diagnostic and monitoring information in a simple web page. One can browse to the below url on your local server to open the web administration utility http://localhost:28017
4. The serverStatus command, or db.serverStatus() from the shell, returns an overview of the status of the database, with details on the disk usage, memory use, connections established to the MongoDB environment, etc.

MongoDB Indexing and Performance Considerations

1. Indexes are very important in any database and can be used to improve the efficiency of search queries in MongoDB. If you are continually performing searches in your document, then it's better to add indexes on the fields of the document that are used in the search criteria.
2. Try to always limit the number of query results returned. Suppose you have 2 field names in a document, but you just want to see 2 fields from the document. Then make sure your query only targets to display the 2 fields you require and not all of the fields.
3. If you want to view certain field values, then only use those fields in the query. Do not query for all the fields in the collection if they are not required.

Summary:

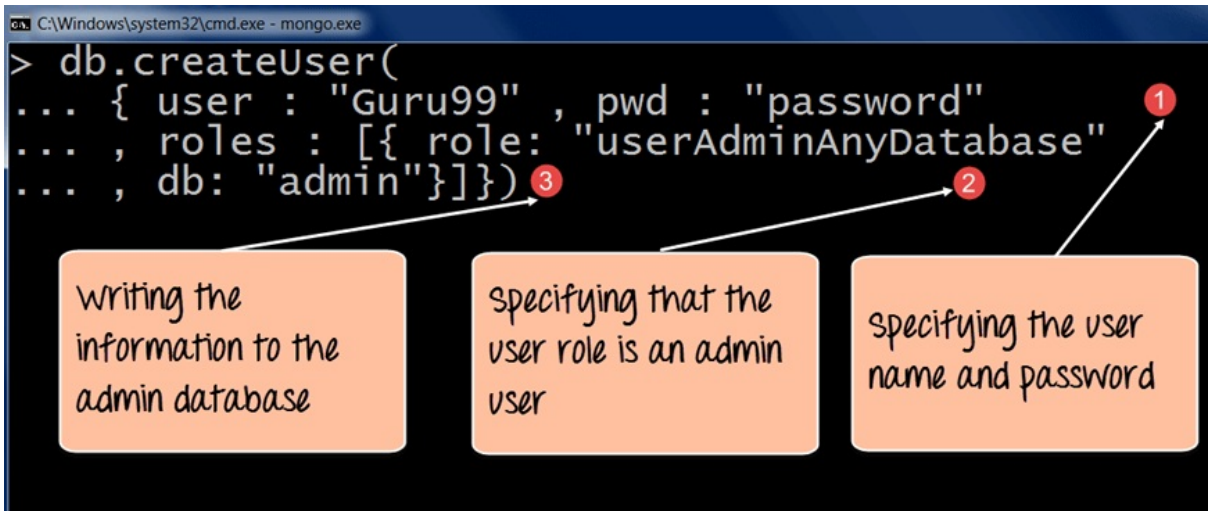
- It is very important to implement security in databases to ensure that the data in the database is kept safe.
- Users can be created in the database with the createUser command. Specific roles can be assigned to the users to give them specific permissions on the database itself.
- Administrators can be added for all databases are for just specific databases. This is achieved by giving either the userAdmin or the userAdminAnyDatabase role.
- Always backup your MongoDB environment so that in that in the case of any disaster, the data would be easily recoverable.
- Always monitor your MongoDB environment for being more proactive and see issues before they occur.

Chapter 14: How to Create User & add Role in MongoDB

MongoDB Create Administrator User

Creating a user administrator in MongoDB is done by using the createUser method. The following example shows how this can be done.

```
db.createUser(
{       user: "Guru99",
        pwd: "password",

        roles:[{role: "userAdminAnyDatabase" , db:"admin"}]})
```



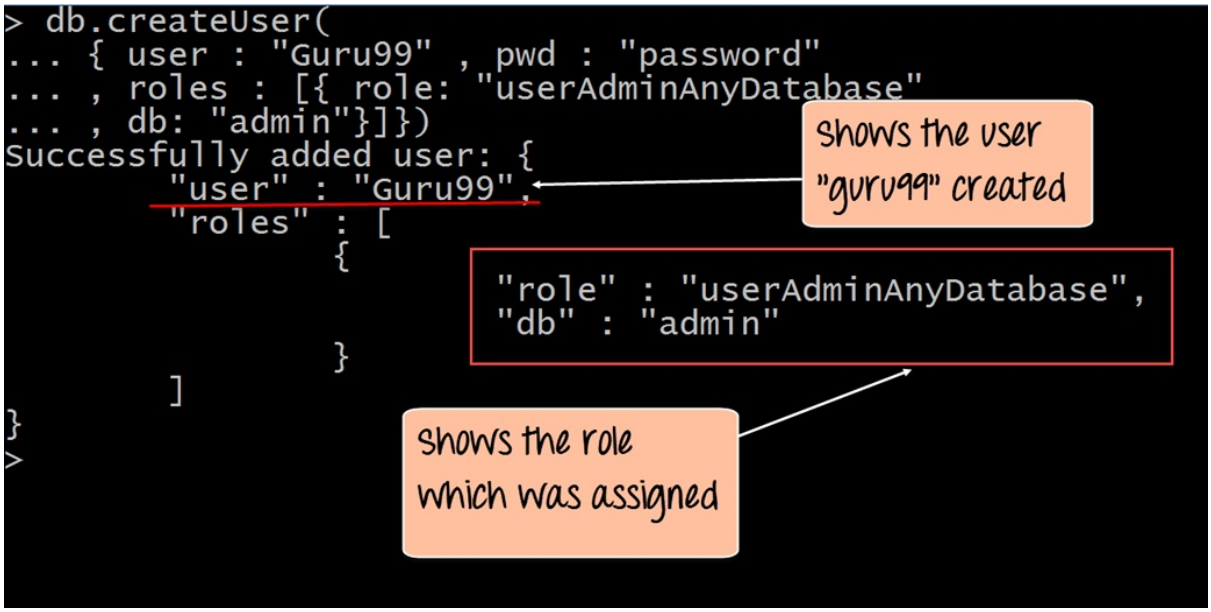
Code Explanation:

1. The first step is to specify the “username” and “password” which needs to be created.
2. The second step is to assign a role for the user. Since it needs to be a database administrator in which case we have assigned to the “userAdminAnyDatabase” role. This role allows the user to have administrative privileges to all databases in MongoDB.
3. The db parameter specifies the admin database which is a special

Meta database within MongoDB which holds the information for this user.

If the command is executed successfully, the following Output will be shown:

Output:

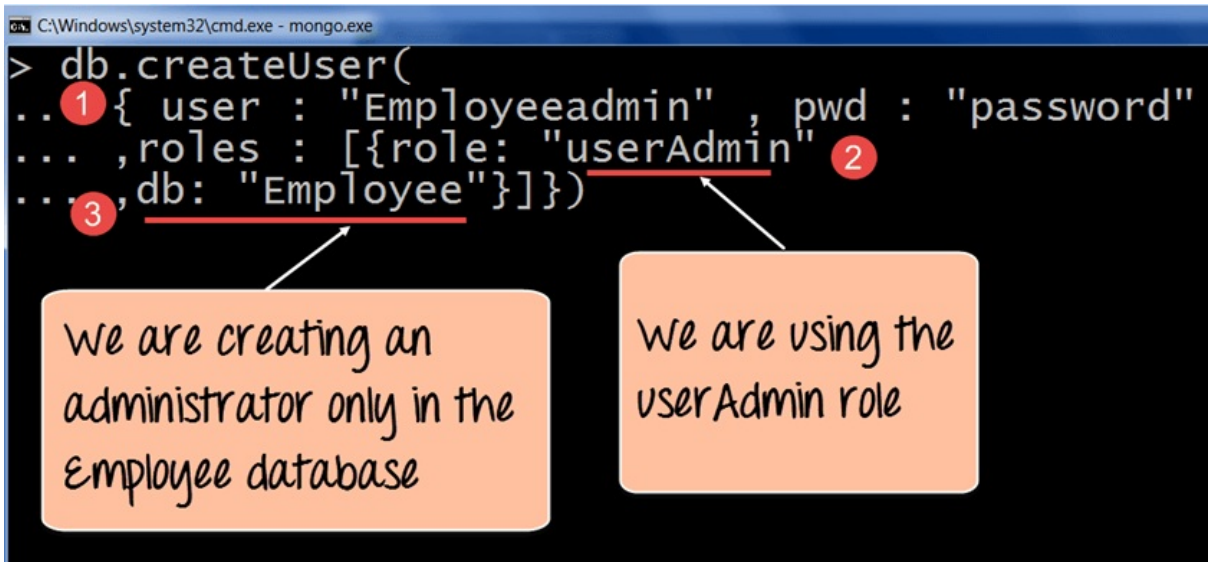


The output shows that a user called “Guru99” was created and that user has privileges over all the databases in MongoDB.

MongoDB Create User for Single Database

To create a user who will manage a single database, we can use the same command as mentioned above but we need to use the “userAdmin” option only.

The following example shows how this can be done;



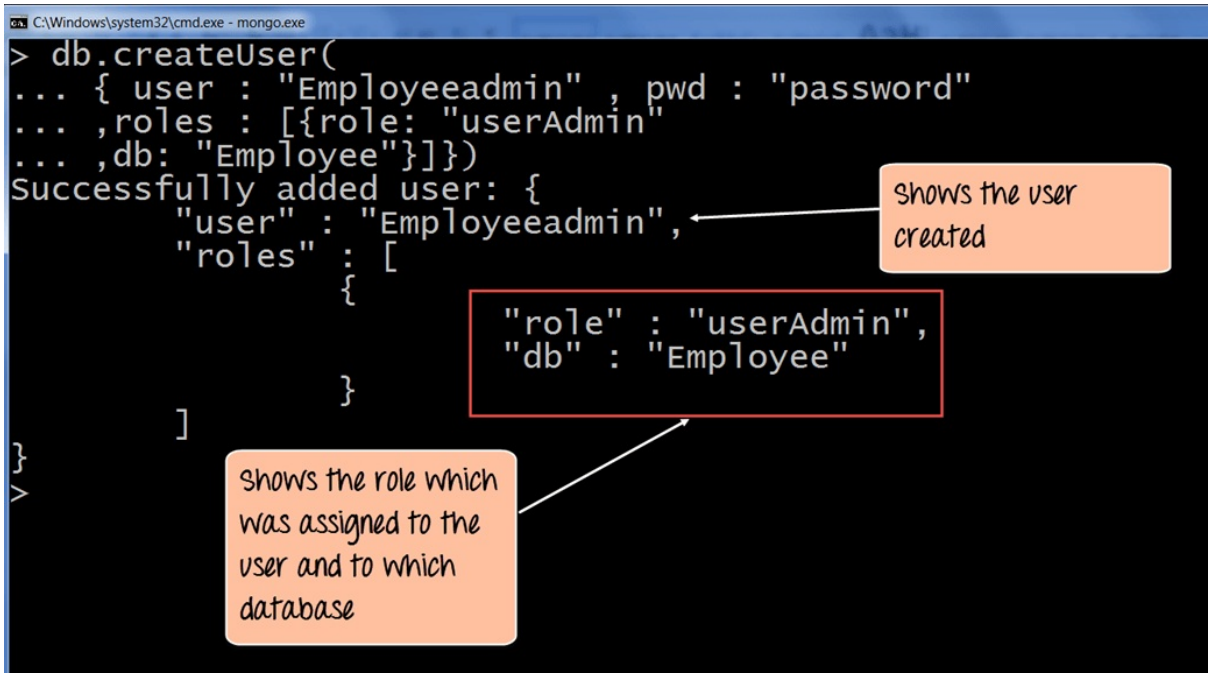

```
db.createUser(
{
        user: "Employeeadmin",

        pwd: "password",

        roles:[{role: "userAdmin" , db:"Employee"}]})
```

Code Explanation:

1. The first step is to specify the “username” and “password” which needs to be created.
2. The second step is to assign a role for the user which in this case since it needs to be a database administrator is assigned to the “userAdmin” role. This role allows the user to have administrative privileges only to the database specified in the db option.
3. The db parameter specifies the database to which the user should have administrative privileges on.

If the command is executed successfully, the following Output will be shown:

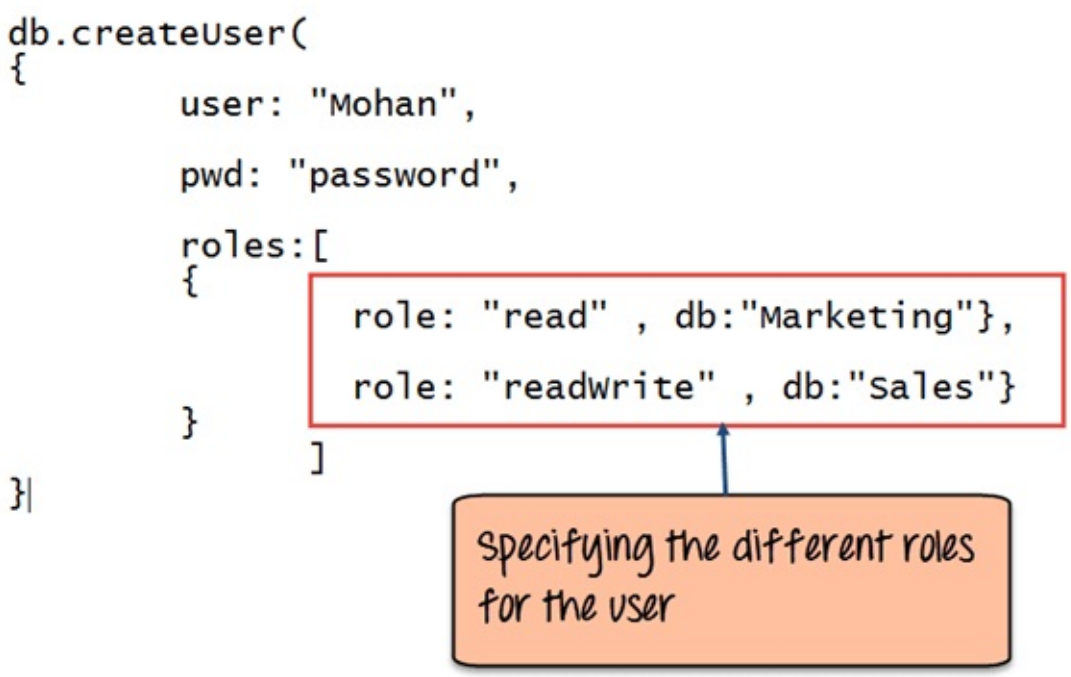
Output:



The output shows that a user called “Employeeadmin” was created and that user has privileges only on the “Employee” database.

Managing users

First understand the roles which you need to define. There is a whole list of role available in MongoDB. For example, there is a the “read role” which only allows read only access to databases and then there is the “readwrite” role which provides read and write access to the database , which means that the user can issue the insert, delete and update commands on collections in that database.




```
db.createUser(
{
        user: "Mohan",

        pwd: "password",

        roles:[
        {
                role: "read" , db:"Marketing"},

                role: "readWrite" , db:"Sales"}
        }
                  ]
})
```

The above code snippet shows that a user called Mohan is created, and he is assigned multiple roles in multiple databases. In the above example, he is given read only permission to the “Marketing” database and readWrite permission to the “Sales” database.

Chapter 15: Configure MongoDB with Kerberos Authentication: X.509 Certificates

While authorization looks at ensuring the client access to the system, the authentication checks what type of access the client has in MongoDB, once they have been authorized into the system.

There are various authentication mechanisms, below are just a few of them.

MongoDB Authentication using x.509 Certificates

Use x.509 Certificates to authenticate the client – A certificate is basically a trusted signature between the client and the MongoDB Server.

So instead of entering a user name and password to connect to the server, a certificate is passed between the client and the MongoDB Server. The client will basically have a client certificate which will be passed to the server to authenticate into the server. Each client certificate corresponds to single MongoDB user. So each user from MongoDB has to have their own certificate in order to authenticated to the MongoDB server.

To ensure this works, the following steps must be followed;

1. A valid certificate must be bought from a valid third party authority and install it on the MongoDB Server.
2. The Client certificate must have the following properties (A single

Certificate Authority (CA) must issue the certificates for both the client and the server . The Client certificates must contain the following fields – keyUsage and extendedKeyUsage.

3. Each user who connects to the MongDB Server needs to have a separate certificate.

Mongodb Authentication with Kerberos

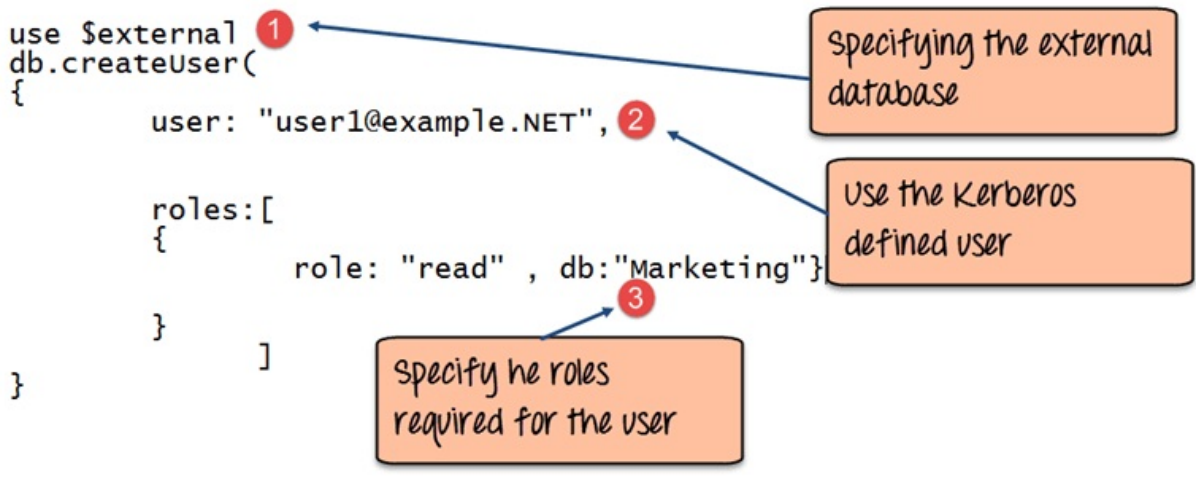
Step 1) Configure MongoDB with Kerberos Authentication on windows – Kerberos is an authentication mechanism used in large client-server environments.

It is a very secure mechanism wherein the password is only allowed if it is encrypted. Well, MongoDB has the facility to authenticate against an existing Kerberos based system.

Step 2) Start the mongod.exe server process.

Step 3) Start the mongo.exe client process and connect to the MongoDB server.

Step 4) Add a user in MongoDB, which is basically a Kerberos principal name to the $external database. The $external database is a special database which tells MongoDB to authenticate this user against a Kerberos system instead of its own internal system.



```
use $external
db.createUser(
{
        user: "user1@example.NET",

        roles:[
        {
                role: "read" , db:"Marketing"}

        }
                  ]
}
```

Step 5) Start mongod.exe with Kerberos support by using the following command

```
mongod.exe –auth –setParameter authenticationMechanisms=GSSAPI
```

And then you can now connect with the Kerberos user and Kerberos authentication to the database.

Summary:

- There are various authentication mechanisms to provide better security in databases. One example is the usage of certificates to authenticate users instead of using usernames and passwords.

Chapter 16: MongoDB Replica Set Tutorial: Step by Step Replication Example

What is MongoDB Replication?

Replication is referred to the process of ensuring that the same data is available on more than one Mongo DB Server. This is sometimes required for the purpose of increasing data availability.

Because if your main MongoDB Server goes down for any reason, there will be no access to the data. But if you had the data replicated to another server at regular intervals, you will be able to access the data from another server even if the primary server fails.

Another purpose of replication is the possibility of load balancing. If there are many users connecting to the system, instead of having everyone connect to one system, users can be connected to multiple servers so that there is an equal distribution of the load.

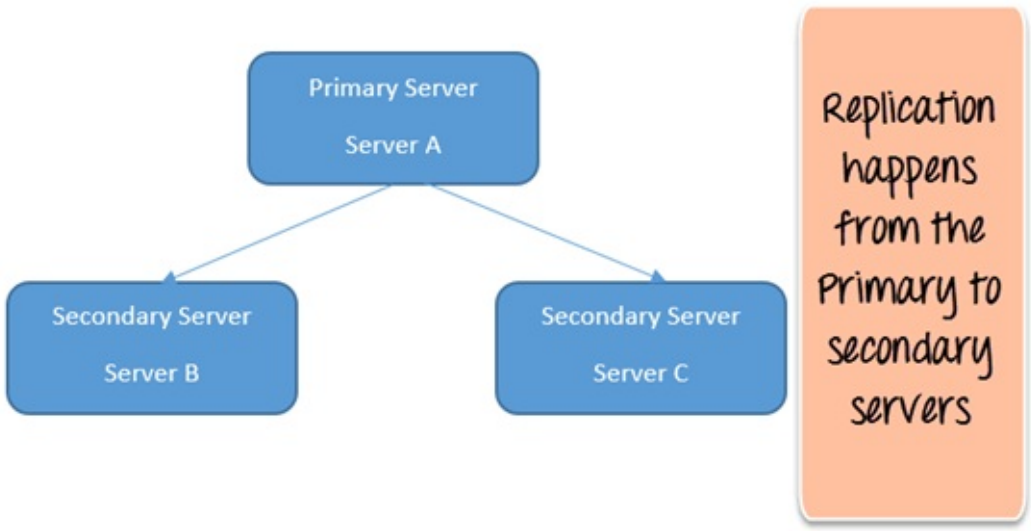
In MongoDB, multiple MongDB Servers are grouped in sets called Replica sets. The Replica set will have a primary server which will accept all the write operation from clients. All other instances added to the set after this will be called the secondary instances which can be used primarily for all read operations.

Replica Set: Adding the First Member using rs.initiate()

As mentioned in the previous section, to enable replication, we first need to create a replica set of MongoDB instances.

Let’s assume that for our example, we have 3 servers called ServerA,

ServerB, and ServerC. In this configuration, ServerA will be our Primary server and ServerB and ServerC will be our secondary servers. Below screenshot will give a better idea on it.



Below are the steps which need to be followed for creating the replica set along with the addition of the first member to the set.

Step 1) Ensure that all mongod.exe instances which will be added to the replica set are installed on different servers. This is to ensure that even if one server goes down, the others will be available and hence other instances of MongoDB will be available.

Step 2) Ensure that all mongo.exe instances can connect to each other. From ServerA, issue the below 2 commands

```
mongo –host ServerB –port 27017
```

```
mongo –host ServerC –port 27017
```

Similarly, do the same thing from the remaining servers.

Step 3) Start the first mongod.exe instance with the replSet option. This option provides a grouping for all servers which will be part of this replica set.

```
mongo –replSet "Replica1"
```

Where “Replica1” is the name of your replica set. You can choose any meaningful name for your replica set name.

Step 4) Now that the first server is added to the replica set, the next step is to initiate the replica set by issuing the following command rs.initiate ()

Step 5) Verify the replica set by issuing the command rs.conf() to ensure the replica set up properly

Replica Set: Adding a Secondary using rs.add()

The secondary servers can be added to the replica set by just using the rs.add command. This command takes in the name of the secondary servers and adds the servers to the replication set.

Step 1) Suppose if you have ServerA, ServerB, and ServerC, which are required to be part of your replica set and ServerA, is defined as the primary server in the replica set.

To add ServerB and ServerC to the replica set issue the commands

```
rs.add("ServerB")
rs.add("ServerC")
```

Replica Set: Reconfiguring or Removing using rs.remove()

To remove a server from the configuration set, we need to use the “rs.remove” command

Step 1) First perform a shutdown of the instance which you want to remove. One can do this by issuing the db.shutdownserver command from the mongo shell.

Step 2) Connect to the primary server

Step 3) Use the rs.remove command to remove the required server from the replica set. So suppose if you have a replica set with ServerA,

ServerB, and ServerC, and you want to remove ServerC from the replica set, issue the command

```
rs.remove("ServerC")
```

Troubleshooting Replica Sets

The following steps are same ways one can troubleshoot when issues are encountered with the usage of replica sets.

1. Ensure that all mongo.exe instances can connect to each other. Suppose if you have 3 servers called ServerA, ServerB, and ServerC. From Server A, issue the below 2 commands

```
mongo –host ServerB –port 27017
mongo –host ServerC –port 27017
```

2. Run the rs.status command. This command gives the status of the replica set. By default, each member will send messages to each other called “heartbeat” messages which just indicates that the server is alive and working. The “status” command get the status of these messages and shows if there are any issues with any members in the replica set.
3. Check the size of the Oplog – The Oplog is a collection in MongoDB that stores the history of writes which were done to the MongoDB database. MongoDB then uses this Oplog to replicate the writes to the other members in the replica set. To check the Oplog, connect to the required member instance and run the rs.printReplicationInfo command. This command will show the size of the log and how long it can hold transactions in its log file before it becomes full.

Summary:

- Replication is referred to the process of ensuring that the same data is available on more than one Mongo DB Server. Many

members (MongoDB instances) can be added to the Replica set depending on the requirements.

Chapter 17: MongoDB Sharding: Step by Step Tutorial with Example

What is Sharding in MongoDB?

Sharding is a concept in MongoDB, which splits large data sets into small data sets across multiple MongoDB instances.

Sometimes the data within MongoDB will be so huge, that queries against such big data sets can cause a lot of CPU utilization on the server. To tackle this situation, MongoDB has a concept of Sharding, which is basically the splitting of data sets across multiple MongoDB instances.

The collection which could be large in size is actually split across multiple collections or Shards as they are called. Logically all the shards work as one collection.

How to Implement Sharding

Shards are implemented by using clusters which are nothing but a group of MongoDB instances.

The components of a Shard include

1. **A Shard** – This is the basic thing, and this is nothing but a MongoDB instance which holds the subset of the data. In production environments, all shards need to be part of replica sets.
2. **Config server** – This is a mongodb instance which holds metadata about the cluster, basically information about the various mongodb instances which will hold the shard data.

3. **A Router** – This is a mongodb instance which basically is responsible to re-directing the commands send by the client to the right servers.

Step by Step Sharding Cluster Example

Step 1) Create a separate database for the config server.

```
mkdir /data/configdb
```

Step 2) Start the mongodb instance in configuration mode. Suppose if we have a server named Server D which would be our configuration server, we would need to run the below command to configure the server as a configuration server.

```
mongod –configdb ServerD: 27019
```

Step 3) Start the mongos instance by specifying the configuration server

```
mongos –configdb ServerD: 27019
```

Step 4) From the mongo shell connect to the mongo's instance

```
mongo –host ServerD –port 27017
```

Step 5) If you have Server A and Server B which needs to be added to the cluster, issue the below commands

```
sh.addShard("ServerA:27017")
sh.addShard("ServerB:27017")
```

Step 6) Enable sharding for the database. So if we need to shard the Employeedb database, issue the below command

```
sh.enableSharding(Employeedb)
```

Step 7) Enable sharding for the collection. So if we need to shard the Employee collection, issue the below command

```
Sh.shardCollection("db.Employee" , { "Employeeid" : 1 ,
"EmployeeName" : 1})
```

Summary:

- As explained in tutorial, Sharding is a concept in MongoDB, which splits large data sets into small data sets across multiple MongoDB instances.

Chapter 18: MongoDB Indexing Tutorial - createIndex(), dropindex() Example

Indexes are very important in any database, and with MongoDB it's no different. With the use of Indexes, performing queries in MongoDB becomes more efficient.

If you had a collection with thousands of documents with no indexes, and then you query to find certain documents, then in such case MongoDB would need to scan the entire collection to find the documents. But if you had indexes, MongoDB would use these indexes to limit the number of documents that had to be searched in the collection.

Indexes are special data sets which store a partial part of the collection's data. Since the data is partial, it becomes easier to read this data. This partial set stores the value of a specific field or a set of fields ordered by the value of the field.

Understanding Impact of Indexes

Now even though from the introduction we have seen that indexes are good for queries, but having too many indexes can slow down other operations such as the Insert, Delete and Update operation.

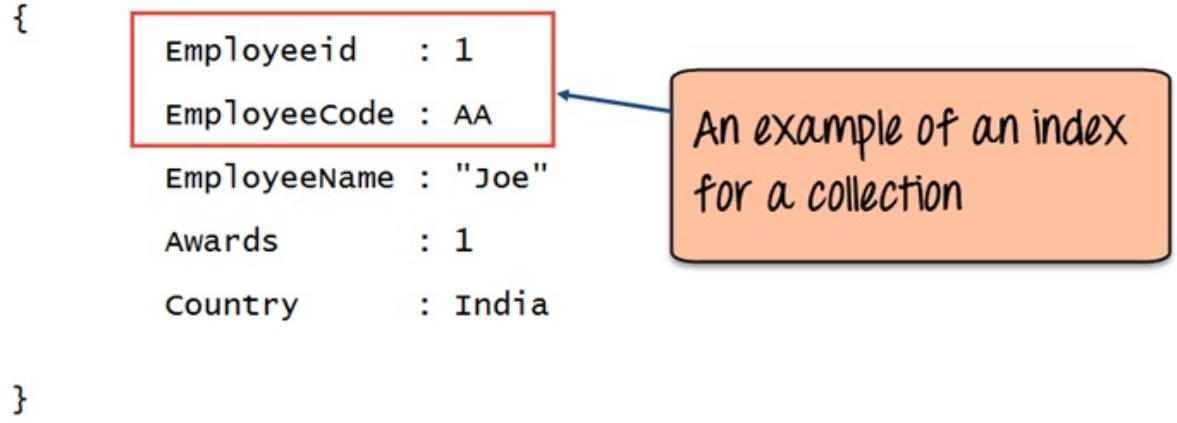
If there are frequent insert, delete and update operations carried out on documents, then the indexes would need to change that often, which would just be an overhead for the collection.

The below example shows an example of what field values could constitute an index in a collection. An index can either be based on just

one field in the collection, or it can be based on multiple fields in the collection.

In the example below, the Employeeid “1” and EmployeeCode “AA” are used to index the documents in the collection. So when a query search is made, these indexes will be used to quickly and efficiently find the required documents in the collection.

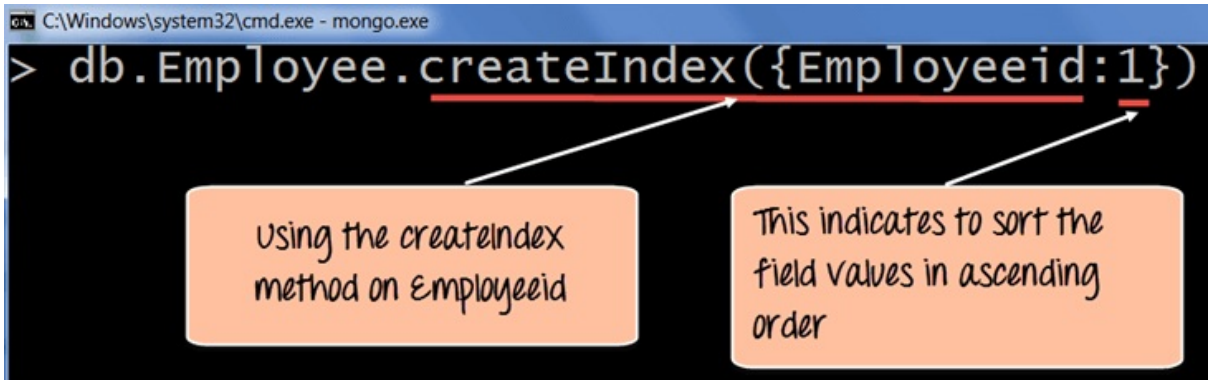
So even if the search query is based on the EmployeeCode “AA”, that document would be returned.



How to Create Indexes: createIndex()

Creating an Index in MongoDB is done by using the “**createIndex**” method.

The following example shows how add index to collection. Let’s assume that we have our same Employee collection which has the Field names of “Employeeid” and “EmployeeName”.



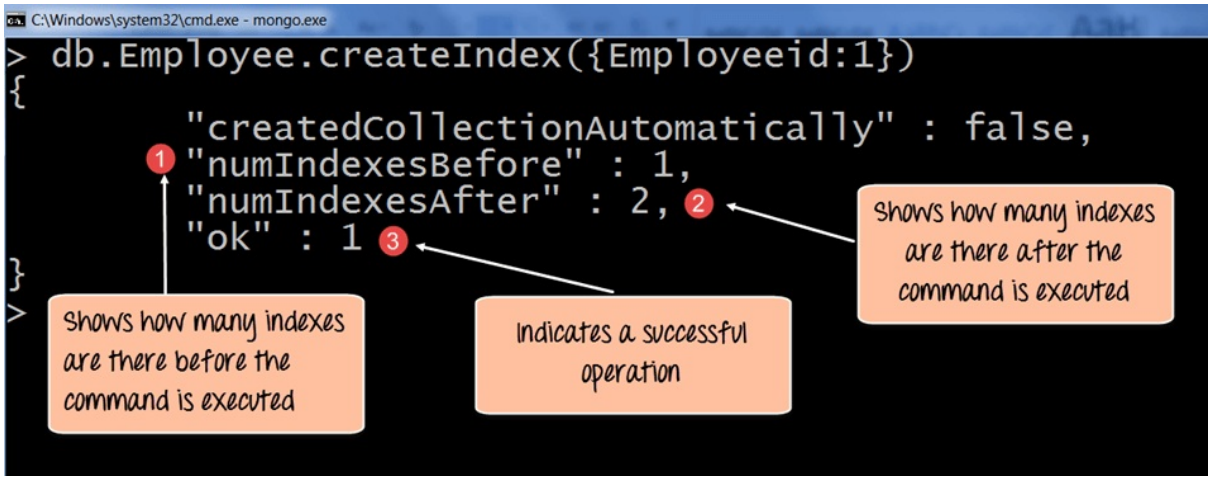
```
db.Employee.createIndex({Employeeid:1])
```

Code Explanation:

1. The **createIndex** method is used to create an index based on the "Employeeid" of the document.
2. The '1' parameter indicates that when the index is created with the "Employeeid" Field values, they should be sorted in ascending order. Please note that this is different from the _id field (The id field is used to uniquely identify each document in the collection) which is created automatically in the collection by MongoDB. The documents will now be sorted as per the Employeeid and not the _id field.

If the command is executed successfully, the following Output will be shown:

Output:

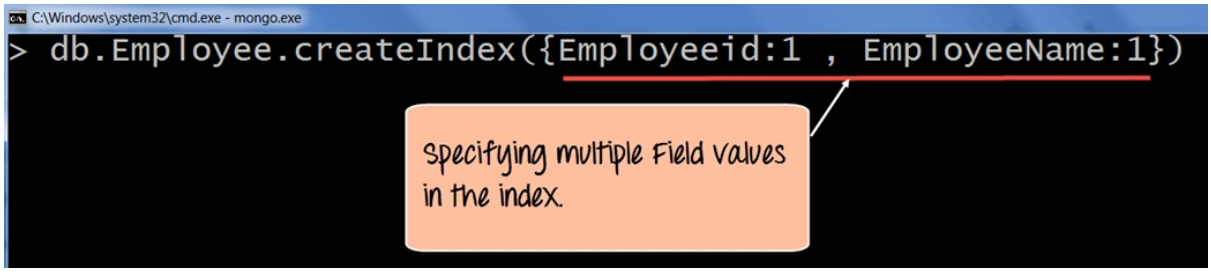


1. The numIndexesBefore: 1 indicates the number of Field values (The actual fields in the collection) which were there in the indexes before the command was run. Remember that each collection has the _id field which also counts as a Field value to the index. Since the _id index field is part of the collection when it is initially created, the value of numIndexesBefore is 1.
2. The numIndexesAfter: 2 indicates the number of Field values which were there in the indexes after the command was run.

3. Here the “ok: 1” output specifies that the operation was successful, and the new index is added to the collection.

The above code shows how to create an index based on one field value, but one can also create an index based on multiple field values.

The following example shows how this can be done;




```
db.Employee.createIndex({Employeeid:1, EmployeeName:1])
```

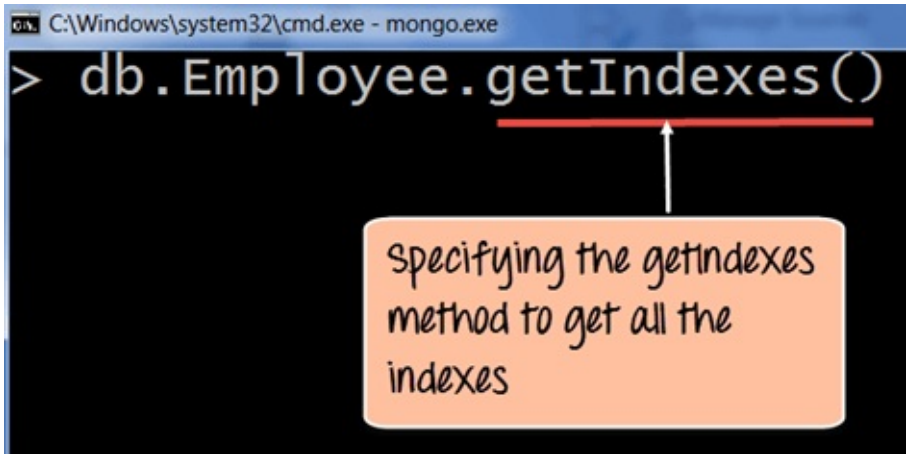
Code Explanation:

1. The createIndex method now takes into account multiple Field values which will now cause the index to be created based on the “Employeeid” and “EmployeeName”. The Employeeid:1 and EmployeeName:1 indicates that the index should be created on these 2 field values with the :1 indicating that it should be in ascending order.

How to Find Indexes: getindexes()

Finding an Index in MongoDB is done by using the **“getIndexes”** method.

The following example shows how this can be done;



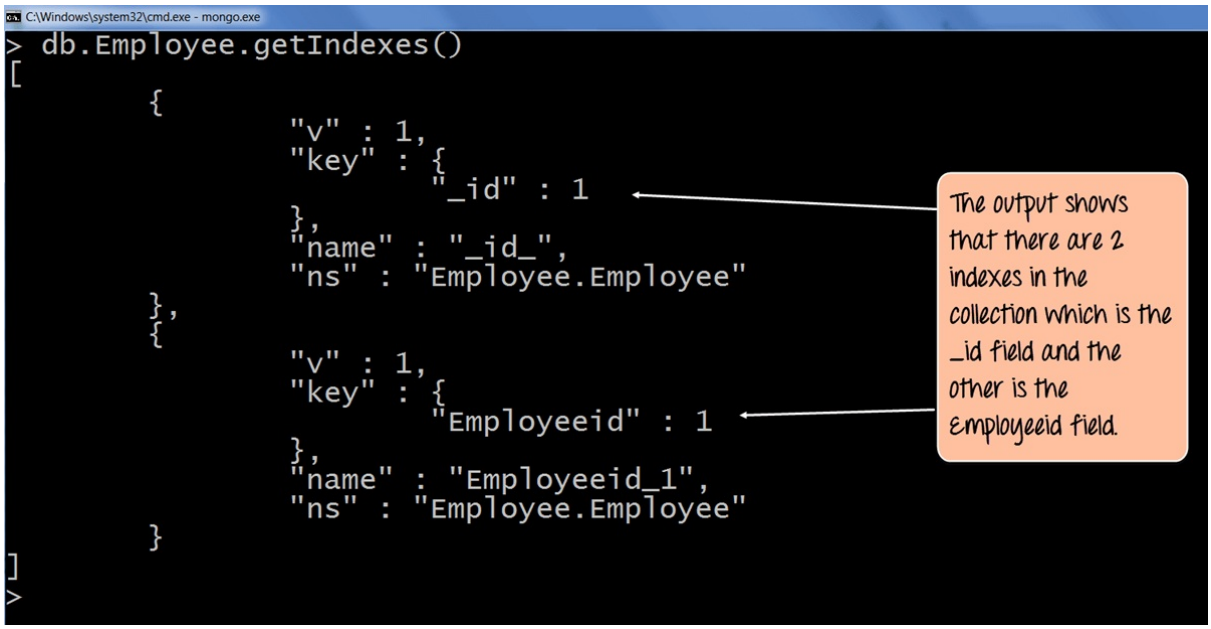

```
db.Employee.getIndexes()
```

Code Explanation:

1. The getIndexes method is used to find all of the indexes in a collection.

If the command is executed successfully, the following Output will be shown:

Output:

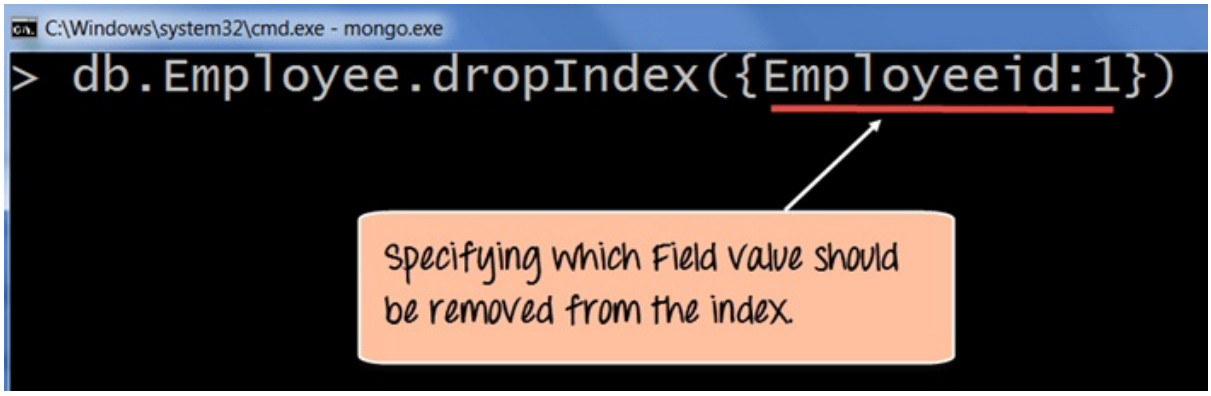


1. The output returns a document which just shows that there are 2 indexes in the collection which is the _id field, and the other is the Employee id field. The :1 indicates that the field values in the index are created in ascending order.

How to Drop Indexes: dropindex()

Removing an Index in MongoDB is done by using the dropIndex method.

The following example shows how this can be done;



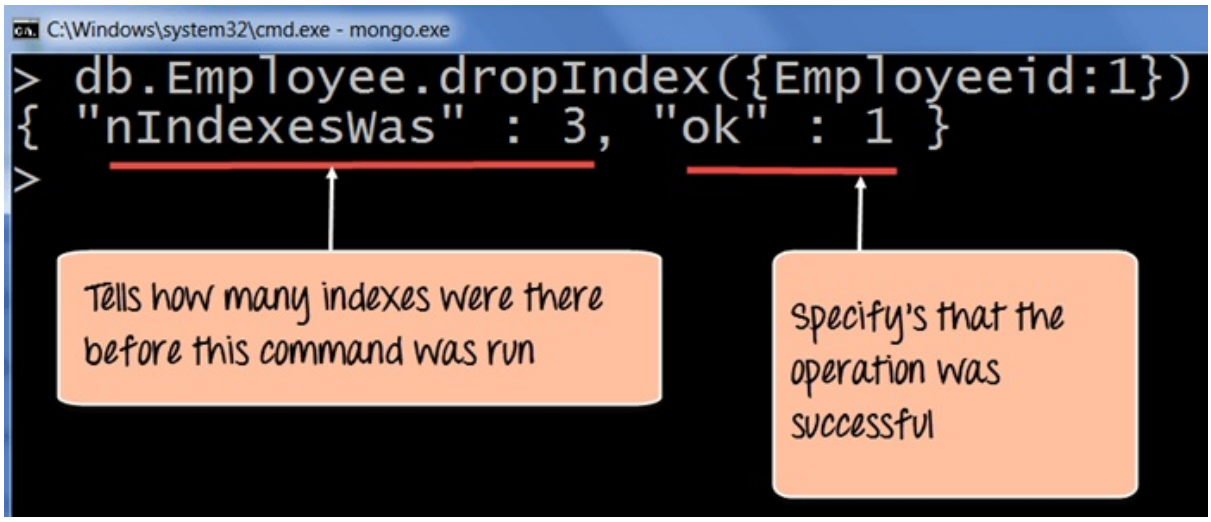

```
db.Employee.dropIndex(Employeeid:1)
```

Code Explanation:

1. The dropIndex method takes the required Field values which needs to be removed from the Index.

If the command is executed successfully, the following Output will be shown:

Output:



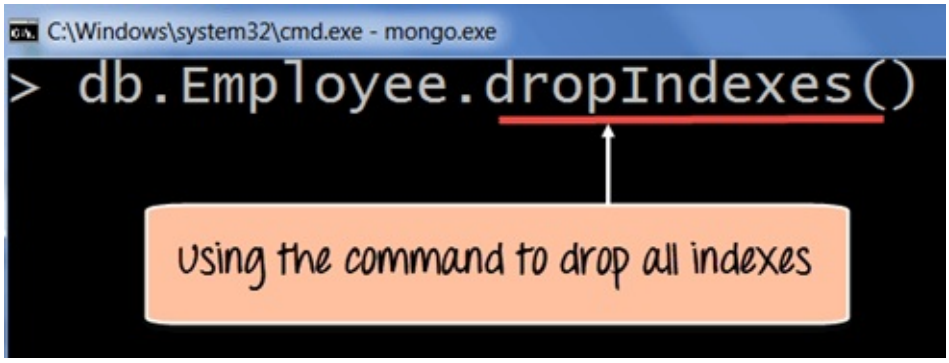
1. The nIndexesWas: 3 indicates the number of Field values which

were there in the indexes before the command was run. Remember that each collection has the _id field which also counts as a Field value to the index.

2. The ok: 1 output specifies that the operation was successful, and the “Employeeid” field is removed from the index.

To remove all of the indexes at once in the collection, one can use the dropIndexes command.

The following example shows how this can be done.



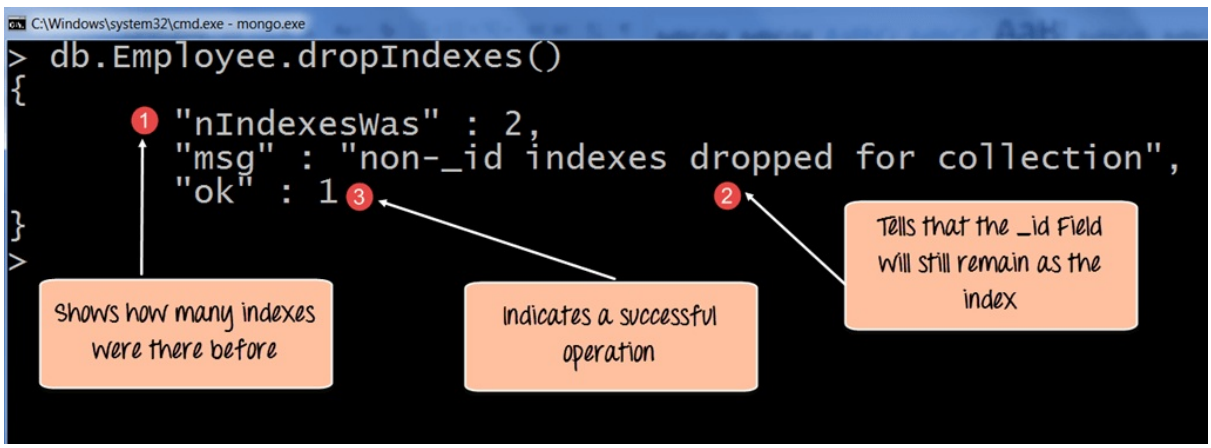

```
db.Employee.dropIndex()
```

Code Explanation:

1. The dropIndexes method will drop all of the indexes except for the _id index.

If the command is executed successfully, the following Output will be shown:

Output:



1. The nIndexesWas: 2 indicates the number of Field values which were there in the indexes before the command was run.
2. Remember again that each collection has the _id field which also counts as a Field value to the index, and that will not be removed by MongoDB and that is what this message indicates.
3. The ok: 1 output specifies that the operation was successful.

Summary

- Defining indexes are important for faster and efficient searching of documents in a collection.
- Indexes can be created by using the createIndex method. Indexes can be created on just one field or multiple field values.
- Indexes can be found by using the getIndexes method.
- Indexes can be removed by using the dropIndex for single indexes or dropIndexes for dropping all indexes.

Chapter 19: MongoDB Regular Expression (Regex) with Examples

Regular expressions are used for pattern matching, which is basically for findings strings within documents.

Sometimes when retrieving documents in a collection, you may not know exactly what the exact Field value to search for. Hence, one can use regular expressions to assist in retrieving data based on pattern matching search values.

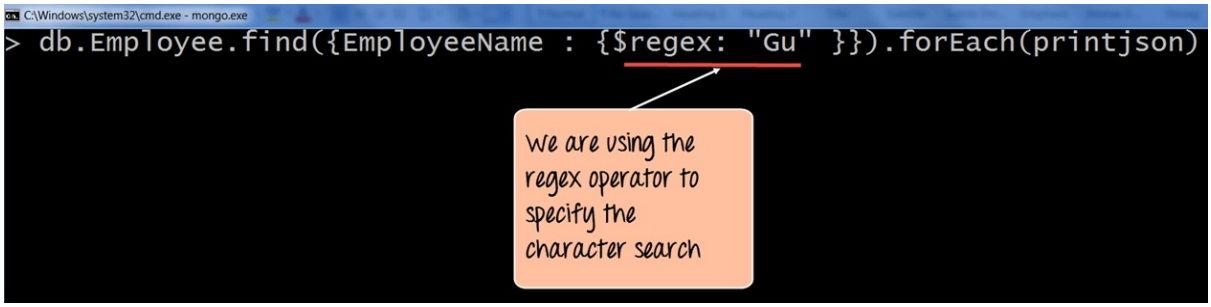
Using $regex operator for Pattern matching

The regex operator in MongoDB is used to search for specific strings in the collection. The following example shows how this can be done.

Let's assume that we have our same Employee collection which has the Field names of "Employeeid" and "EmployeeName". Let' also assume that we have the following documents in our collection.

Employee id	Employee Name
22	NewMartin
2	Mohan
3	Joe
4	MohanR
100	Guru99
6	Gurang

Here in the below code we have used regex operator to specify the search criteria.



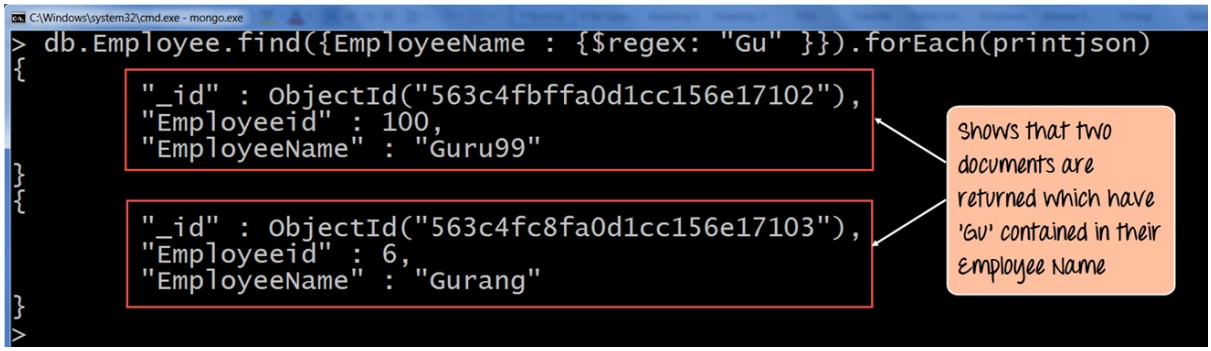

```
db.Employee.find({EmployeeName : {$regex: "Gu"
}}).forEach(printjson)
```

Code Explanation:

1. Here we want to find all Employee Names which have the characters 'Gu' in it. Hence, we specify the $regex operator to define the search criteria of 'Gu'
2. The printjson is being used to print each document which is returned by the query in a better way.

If the command is executed successfully, the following Output will be shown:

Output:



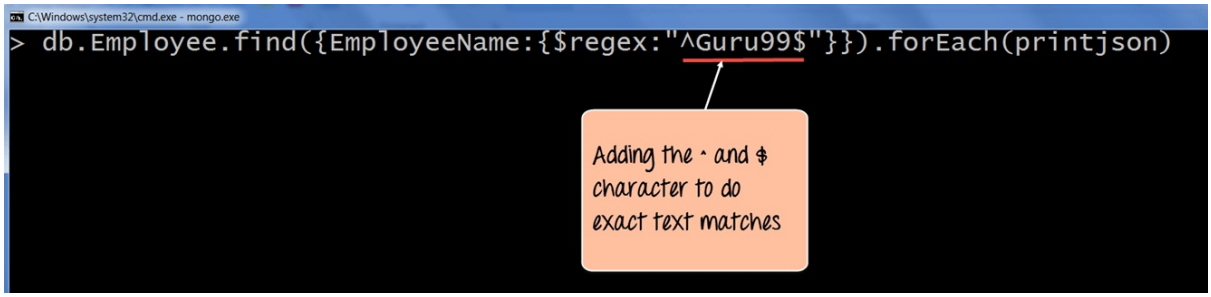
The output clearly shows that those documents wherein the Employee Name contains the 'Gu' characters are returned.

If suppose your collection has the following documents with an additional document which contained the Employee Name as "Guru999". If you entered the search criteria as "Guru99", it would also return the document which had "Guru999". But suppose if we didn't want this and only wanted to return the document with "Guru99". Then we can do this with exact pattern matching. To do an

exact pattern matching, we will use the ^ and $ character. We will add the ^ character in the beginning of the string and $ at the end of the string.

Employee id	Employee Name
22	NewMartin
2	Mohan
3	Joe
4	MohanR
100	Guru99
6	Gurang
8	Guru999

The following example shows how this can be done.



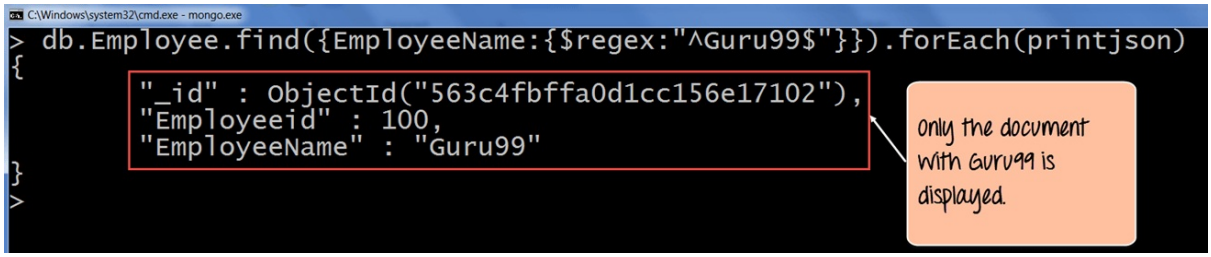

```
db.Employee.find({EmployeeName : {$regex:
"^Guru99$"}}).forEach(printjson)
```

Code Explanation:

1. Here in the search criteria, we are using the ^ and $ character. The ^ is used to make sure that the string starts with a certain character, and $ is used to ensure that the string ends with a certain character. So when the code executes it will fetch only the string with the name “Guru99”.
2. The printjson is being used to print each document which is returned by the query in a better way.

If the command is executed successfully, the following Output will be shown:

Output:



In the output, it is clearly visible that string "Guru99" is fetched.

Pattern Matching with $options

When using the regex operator one can also provide additional options by using the **$options** keyword. For example, suppose you wanted to find all the documents which had 'Gu' in their Employee Name, irrespective of whether it was case sensitive or insensitive. If such a result is desired, then we need to use the **$options** with case insensitivity parameter.

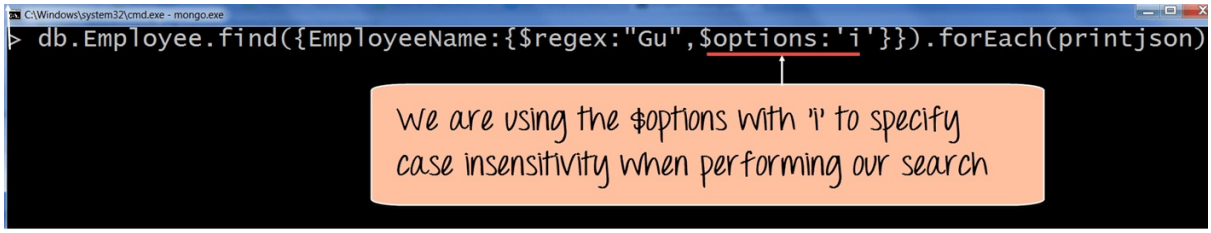
The following example shows how this can be done.

Let's assume that we have our same Employee collection which has the Field names of "Employeeid" and "EmployeeName".

Let' also assume that we have the following documents in our collection.

Employee id	Employee Name
22	NewMartin
2	Mohan
3	Joe
4	MohanR
100	Guru99
6	Gurang
7	GURU99

Now if we run the same query as in the last topic, we would never see the document with "GURU99" in the result. To ensure this comes in the result set, we need to add the $options "I" parameter.



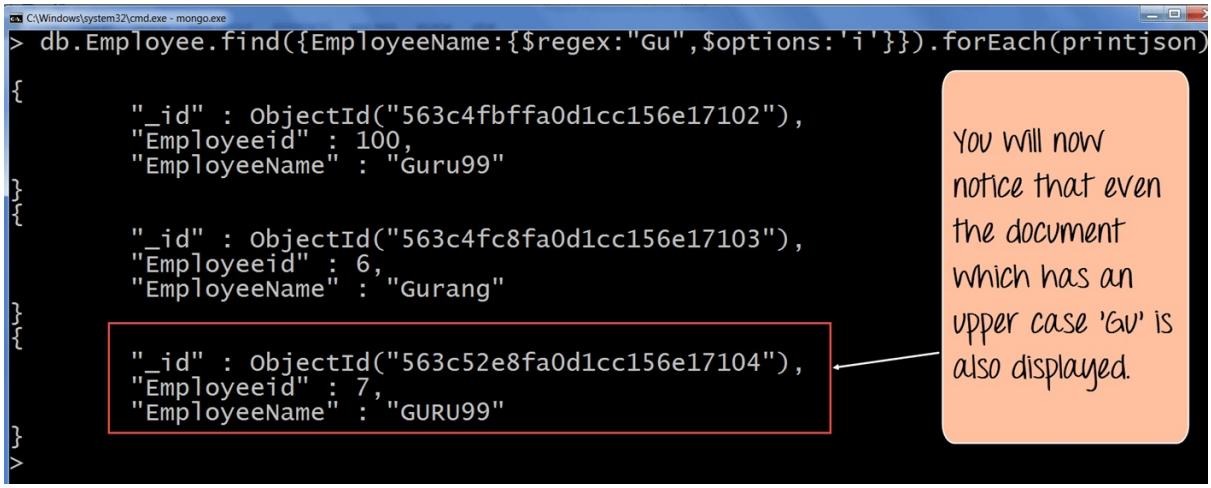

```
db.Employee.find({EmployeeName:{$regex:
"Gu",$options:'i'}}).forEach(printjson)
```

Code Explanation:

1. The $options with 'I' parameter (which means case insensitivity) specifies that we want to carry out the search no matter if we find the letters 'Gu' in lower or upper case.

If the command is executed successfully, the following Output will be shown:

Output:

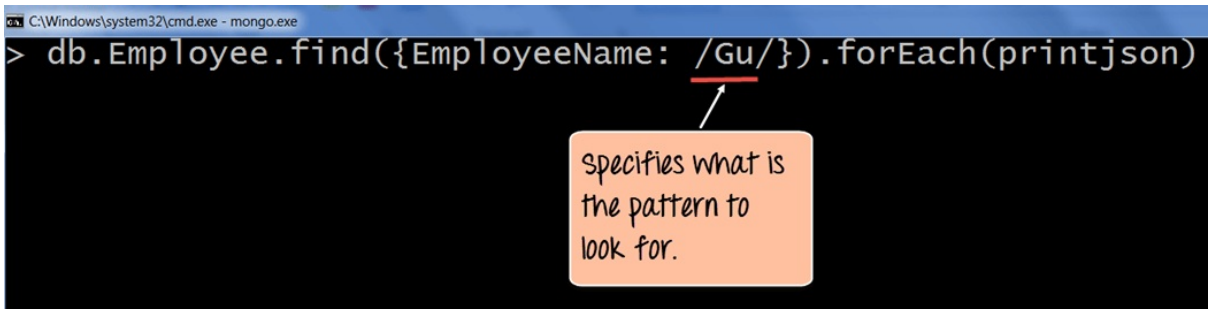


1. The output clearly shows that even though one document has the upper case 'Gu' , the document still gets displayed in the result set.

Pattern matching without the regex operator

One can also do pattern matching without the regex operator. The

following example shows how this can be done.



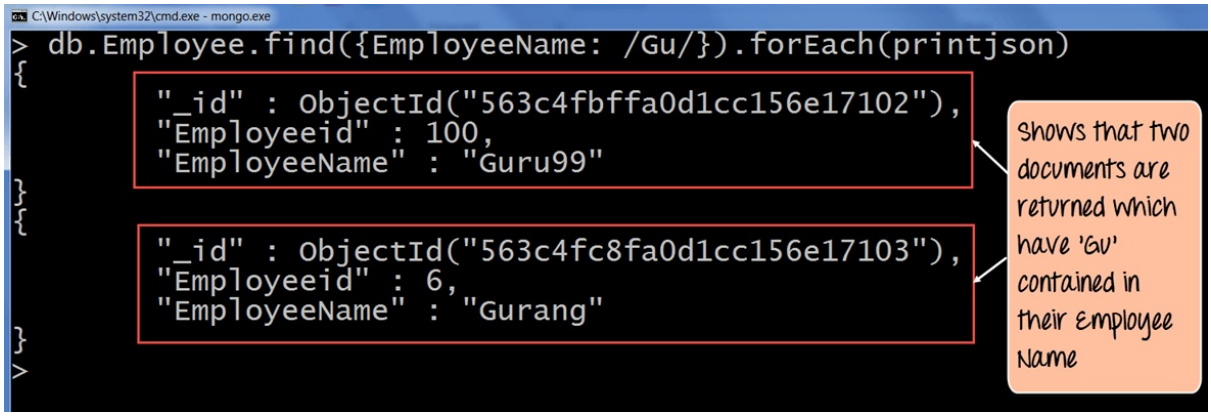

```
db.Employee.find({EmployeeName: /Gu/'}).forEach(printjson)
```

Code Explanation:

1. The "//" options basically means to specify your search criteria within these delimiters. Hence, we are specifying /Gu/ to again find those documents which have 'Gu' in their EmployeeName.

If the command is executed successfully, the following Output will be shown:

Output:



The output clearly shows that those documents wherein the Employee Name contains the 'Gu' characters are returned.

Fetching last 'n' documents from a collection

There are various ways to get the last n documents in a collection.

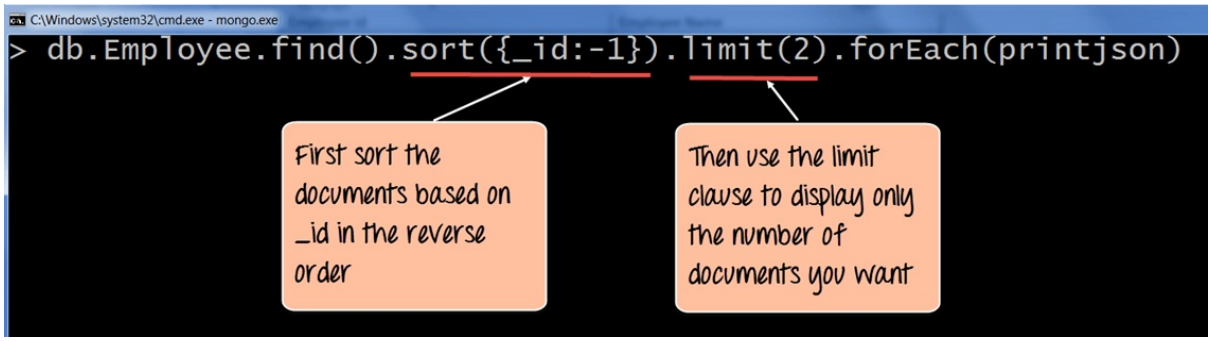
Let's look at one of the ways via the following steps

The following example shows how this can be done.

Let's assume that we have our same Employee collection which has the Field names of "Employeeid" and "EmployeeName".

Let' also assume that we have the following documents in our collection:

Employee id	Employee Name
22	NewMartin
2	Mohan
3	Joe
4	MohanR
100	Guru99
6	Gurang
7	GURU99




```
db.Employee.find().sort({_id:-1}).limit(2).forEach(printjson)
```

Code Explanation:

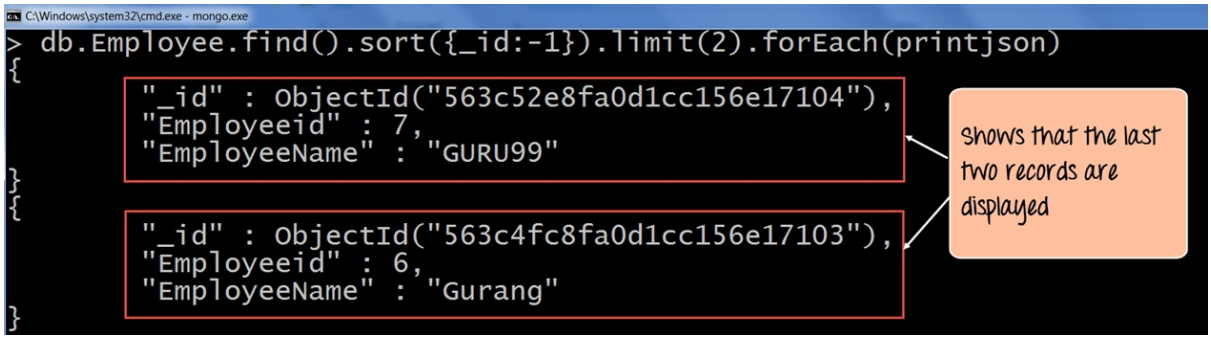
1) When querying for the documents, use the sort function to sort the records in reverse order based on the _id field value in the collection. The -1 basically indicates to sort the documents in reverse order or descending order so that the last document becomes the first document to be displayed.

2) Then use the limit clause to just display the number of records you want. Here we have set the limit clause (2), so it will fetch the last two documents.

If the command is executed successfully, the following Output will be

shown:

Output:



The output clearly shows that the last two documents in the collection are displayed. Hence we have clearly shown that to fetch the last 'n' documents in the collection, we can first sort the documents in descending order and then use the limit clause to return the 'n' number of documents which are required.

Note: If the search is performed on a string which is greater than say 38,000 characters, it will not display the right results.

Summary:

- Pattern matching can be achieved by the $regex operator. This operator can be used to find for certain strings in the collection.
- The ^ and $ symbol can be used for exact text searches with ^ being used to make sure that the string starts with a certain character and $ used to ensure that the string ends with a certain character.
- The 'i' along with the $regex operator can be used to specify case insensitivity so that strings can be searched whether they are in lower case or upper case.
- The delimiters // can also be used for pattern matching.
- Use a combination of sort and the limit function to return the last n documents in the collection. The sort function can be used to return the documents in descending order after which the limit clause can be used to limit the number of documents being returned.